Analytic Underpinnings of the Symbolic Answer-Reduction Pipeline

4 September 2026

Abstract

The objects manipulated by answer reduction are finite descriptions of machines, circuits, formulas, polynomials, samplers, and verifiers. The Julia prototype realizes part of the corresponding analytic chain as typed transformations of explicit intermediate representations. It constructs finite-field polynomials and conditionally linear samplers, and it checks coefficient identities, degree accounts, dependency blocks, marginal laws, and finite-instance histograms. The operator-algebraic and compression theorems remain imported. This note makes the description-level Turing-machine–lambda-calculus correspondence precise, identifies the fixed point, and records the evidence boundary at every rung. Part I is a self-contained computability refresher for readers returning to Turing machines and lambda calculus after a long interval; Part II then puts exact resource bounds on the same ideas.

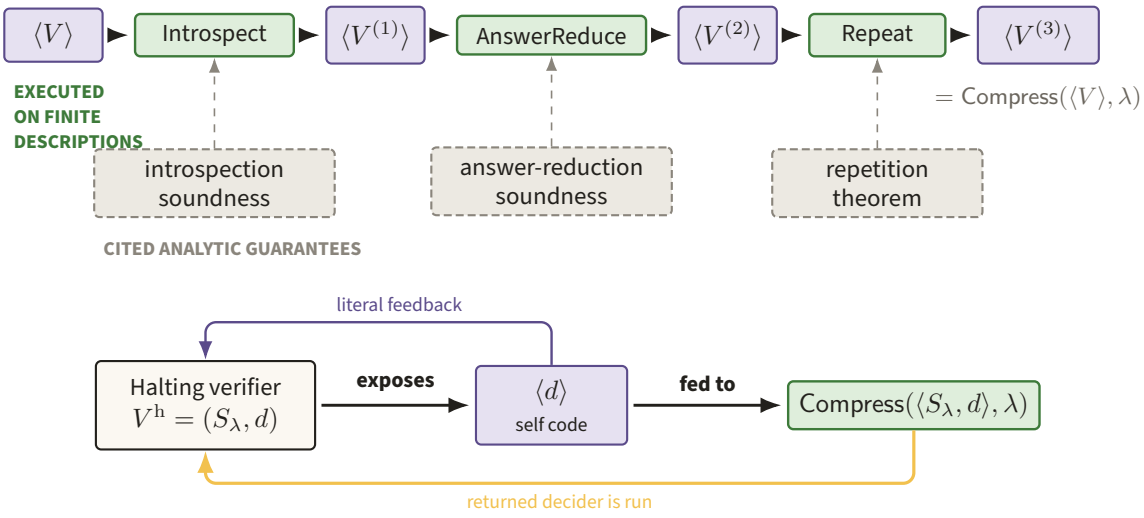


Figure 1. Green stages are finite source transformations, while dashed slate boxes mark the analytic guarantees they invoke and the amber return path closes the self-code loop. The split keeps executable description manipulation visibly separate from cited soundness in the compression argument (after `fig:compress`).

NOTATION: THE NINE QUANTITIES THAT TRAVEL THROUGH BOTH PARTS

symbol	reading	fixed at
TIME	machine steps to halt: $\mathrm{TIME}_M(x)$ on one input, and $\mathrm{TIME}_D(n) = \max_{x,y,a,b} \mathrm{TIME}_{D,(n,x,y,a,b)}$ for a decider	§1.1; Definition 7.2
$\lvert M\rvert, \lvert t\rvert$	description size in bits, with $\lvert t\rvert = \lvert \mathrm{ser}(t)\rvert$ for a term	§1.4; Definition 8.1
f, F	remaining fuel, and a fuel budget supplied to one bounded run; F plays the role of the paper's time bound T in `prop: standard-succinct-sat`	§8.3
κ_p	the *charge* of one primitive step; every microstep spends one fuel unit	§8.3
level	the CL nesting depth; $\mathrm{level}(V) = \mathrm{level}(S)$, and it is an upper bound, never a minimum	Definitions 7.4 and 13.1
λ	the resource bound: $\lvert V\rvert \le \lambda$ and $\mathrm{TIME}_S(n), \mathrm{TIME}_D(n) \le n^\lambda$; also Compress's second argument	Definition 7.4
Q	a bound on the public question lengths $\lvert x\rvert, \lvert y\rvert$; the answer-reduction contract sets $Q(n) = (\lambda n)^\mu$	Theorem 11.2
R	$R = 2^{r_0}$, the succinct index range of the three tableau blocks of a decoupled 5SAT instance	§11.3
$k(n)$	$k(n) = (\lambda n)^{(1+c'_3)\tau}$, the number of copies Repeat runs	Figure 43

The remaining constants are collected where they are first needed: μ, γ, τ, C_0 and the level 9 in Figure 43; $h(d,u) = 3 + \lvert d\rvert + \lvert \mathrm{enc}(u)\rvert$ and $c_Y = 3$ in §8.4 and §10.2; C_L and the exponent 8 in Theorem 9.2; A in Theorem 9.3; $q = 2^k$, m, m', d and $\deg_F$ in §11.5 and §12. Two names are deliberately kept apart: τ is always the repetition constant, and the transition-window function of Lemma 11.1 is ω.

Figure 2. Nine quantities recur in both parts and are given here once, with the place each is fixed. Reading the notation before the table of contents is what lets Part I's qualitative statements and Part II's bounds be recognised as the same quantities.

Contents

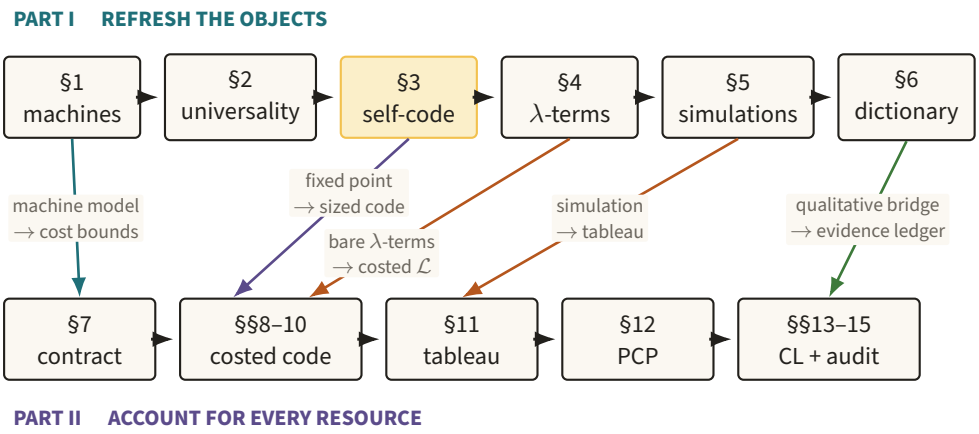


Figure 3. The upper path introduces machines, self-description, and lambda simulation before the lower path assigns sizes, costs, tableaux, and evidence grades. The vertical arrows name which qualitative refresher each resource-accounted construction upgrades; §2's universal machine is used throughout rather than upgraded at one place, and §12's PCP algebra is new material with no Part I counterpart.

Part I

A refresher on computability, from the paper's viewpoint

The outermost layer of the argument is classical computation. Before any low-degree test or entangled strategy appears, the proof must be able to take the finite text of one verifier, transform that text into another verifier, and arrange for a verifier to use a description of itself. This part rebuilds that vocabulary from the ground up. It is deliberately qualitative about polynomial overheads; the exact costed statements begin in Part II. Figure 1 gives the whole description-level pipeline and its fixed-point feedback at a glance, while Figure 3 maps the refresher objects to their resource-accounted counterparts.

1 Turing machines in the form used by the paper

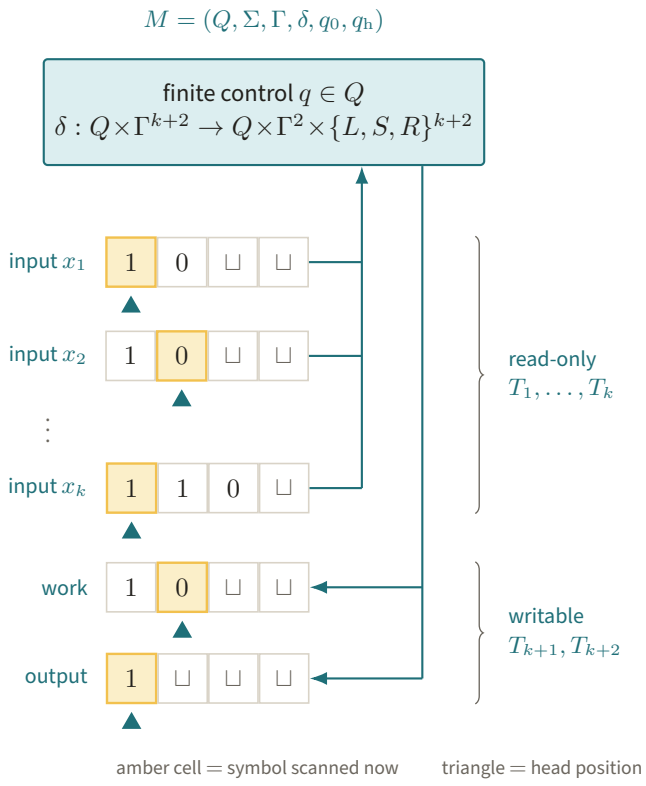


Figure 4. Amber cells are the symbols read by the finite control, with arrows returning updates only to the work and output tapes. This concrete separation of finite rule book from finitely used memory is the machine model simulated throughout the document.

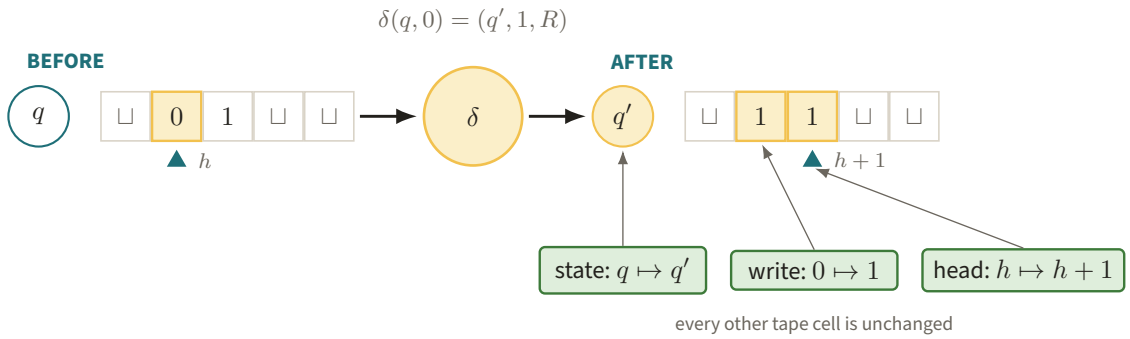


Figure 5. The amber transition rewrites the scanned zero, changes q to q', and advances the head from h to $h + 1$. Exposing exactly these three local changes is what later permits a computation history to be checked by bounded windows.

1.1 Programs, configurations, and one step

A Turing machine separates two things which ordinary mathematical notation often blends together: a finite rule book and an unbounded but finitely used memory. For one tape, a convenient formal tuple is

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\mathrm{h}}),$$

where Q is a finite set of control states, $q_0 \in Q$ is the initial state, $q_{\mathrm{h}} \in Q$ is the halting state, $\Sigma = \{0, 1\}$ is the input alphabet, $\Gamma \supseteq \Sigma \cup \{\sqcup\}$ is a finite tape alphabet with blank symbol $\sqcup \notin \Sigma$, and

$$\delta : (Q \setminus \{q_{\mathrm{h}}\}) \times \Gamma \longrightarrow Q \times \Gamma \times \{L, S, R\}$$

is the transition function. We take tapes to be indexed by $\mathbb{N}$, so a left move at cell zero leaves the head at zero. The paper uses a multitape version: a k-input machine has k input tapes, one work tape, and one output tape. Its transition reads all scanned symbols, changes the state, writes on the work and output tapes, and moves each head by at most one cell. The input tapes are read-only. Adding separate accept and reject halting states, or allowing writes on input tapes, changes none of the computability or polynomial-overhead statements below. Figure 4 fixes this multitape anatomy and the semantic roles of its heads, tapes, and finite control.

A *configuration* is a complete instantaneous description

$$C = (q, T_1, h_1, \ldots, T_{k+2}, h_{k+2}) :$$

the current state, the contents $T_i : \mathbb{N} \to \Gamma$ of every tape, and the head locations $h_i \in \mathbb{N}$. Only finitely many cells are nonblank in every reachable configuration. The notation $C \vdash_M C'$ means that one application of δ changes C into C'. Thus a computation is not a metaphor: it is the uniquely determined path $C_0 \vdash_M C_1 \vdash_M C_2 \vdash_M \cdots$ in the graph of configurations.

Why this matters here. Later, Cook–Levin replaces a bounded computation by local constraints on adjacent configurations. That replacement works because one step looks only at the finite control and the cells under the heads. The enormous object is the history; the local dynamical law δ is finite. This is close to a lattice model with a fixed local update rule, except that the update is deterministic and the head positions are part of the state.

Worked one-step example. If $\delta(q, 0) = (q', 1, R)$, then

$$(q, \ldots \sqcup \underline{0}\, 1 \sqcup \ldots, h) \vdash_M (q', \ldots \sqcup 1\, \underline{1} \sqcup \ldots, h + 1).$$

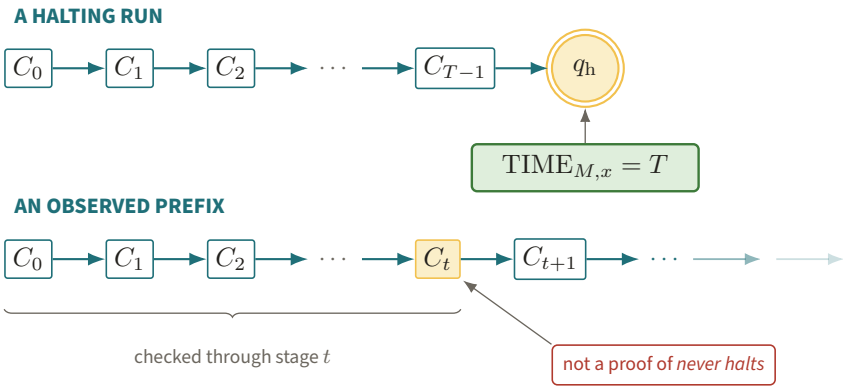


Figure 6. The upper deterministic path ends in q_h and certifies the exact time T, whereas the lower path fades beyond the last observed configuration C_t. The missing halt in a finite prefix cannot be promoted to a proof that the run never halts.

Only the state, the scanned cell, and the head location have changed. Figure 5 isolates those three changes in a literal tape move.

The machine *halts* on input x if some C_T has state q_h. Its output is the longest nonblank initial string on the output tape. We write $M(x)$ for that string and $M(x) = \perp$ if the path never reaches q_h. The instance running time is

$$\mathrm{TIME}_{M,x} := \min\{T : C_T \text{ has state } q_h\},$$

with value ∞ if the set is empty. We sometimes abbreviate this as $\mathrm{TIME}_M(x)$. Notice the distinction between *has not halted yet* and *will never halt*; no finite observation of a run decides the latter in general. Figure 6 contrasts the finite witness supplied by a halting configuration with an open-ended observed prefix.

Why this matters here. The compression argument repeatedly says that a generated verifier finishes within a supplied bound. That is a statement about the number of transitions on every relevant input, not merely about the mathematical function obtained after ignoring time. A timeout wrapper may safely reject after T steps, because it is counting the very transitions used in the definition above.

Worked timing example. For the transition just displayed, if the new state q' is the halt state, then the initial configuration is assigned time one. A timeout of zero rejects without applying the transition; a timeout of one obtains the output.

1.2 A complete two-state equality machine

Here is a small instance in the paper's multitape style. The machine has two one-bit input tapes A, B, a work tape W, and an output tape O. It has two nonhalting states q_0, q_1, plus the designated state q_h. A table entry lists (next state; new W, new O; moves of A, B, W, O). Rows are tested from top to bottom; they are disjoint except for the last catch-all row, so the table specifies every possible scanned tuple.

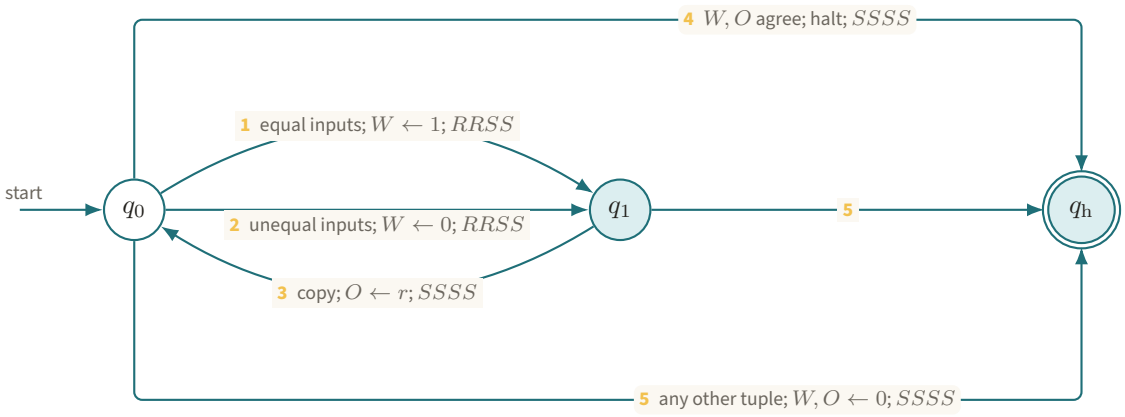


Figure 7. The five numbered paths are exactly the equal-input, unequal-input, copy, halt, and catch-all rows of $M_{=}$. Reading the table as control flow makes totality and the three-step legal-input route visible at once.

State	Scanned (A, B, W, O)	Transition
q_0	$(0, 0, \sqcup, \sqcup)$ or $(1, 1, \sqcup, \sqcup)$	$(q_1; 1, \sqcup; R, R, S, S)$
q_0	$(0, 1, \sqcup, \sqcup)$ or $(1, 0, \sqcup, \sqcup)$	$(q_1; 0, \sqcup; R, R, S, S)$
q_1	$(\sqcup, \sqcup, r, \sqcup)$, $r \in \{0, 1\}$	$(q_0; r, r; S, S, S, S)$
q_0	$(\sqcup, \sqcup, r, r)$, $r \in \{0, 1\}$	$(q_h; r, r; S, S, S, S)$
q_0, q_1	every tuple not covered above	$(q_h; 0, 0; S, S, S, S)$

Figure 7 turns the five rows into a state diagram, including the totalizing catch-all path.

The final row makes the table total and rejects malformed inputs. On legal one-bit inputs the first step computes the equality bit on W, the second copies it to O, and the third enters the halt state. For $A = B = 1$, the full three-step trace, showing the scanned cells, is

i	q	A	B	W	O	rule used
0	q_0	$\underline{1}$	$\underline{1}$	$\underline{\sqcup}$	$\underline{\sqcup}$	equal-input row
1	q_1	$1\underline{\sqcup}$	$1\underline{\sqcup}$	$\underline{1}$	$\underline{\sqcup}$	copy row with $r = 1$
2	q_0	$1\underline{\sqcup}$	$1\underline{\sqcup}$	$\underline{1}$	$\underline{1}$	halt row with $r = 1$
3	q_h	$1\underline{\sqcup}$	$1\underline{\sqcup}$	$\underline{1}$	$\underline{1}$	halted.

Thus $M_{=}(1, 1) = 1$ and $\mathrm{TIME}_{M_{=},(1,1)} = 3$. The same trace with work/output bit zero gives $M_{=}(0, 1) = 0$. The tape-by-tape rendering in Figure 8 keeps every scanned location visible through the full run.

Why this matters here. The example is intentionally mundane: a paper verifier is just a much larger finite transition table whose output bit says accept or reject. Statements about a verifier's code, time, or simulation refer to this concrete object, even when the paper describes it in higher-level pseudocode.

1.3 Several inputs and the verifier convention

Writing $M(x_1, \ldots, x_k)$ means that x_i initially occupies the initial cells of input tape i, with all other cells blank and all heads at zero. Multiple tapes are convenient interfaces, not a stronger model: a one-tape machine can store a self-delimiting encoding of all tapes and their head markers and simulate one multitape step with polynomial overhead.

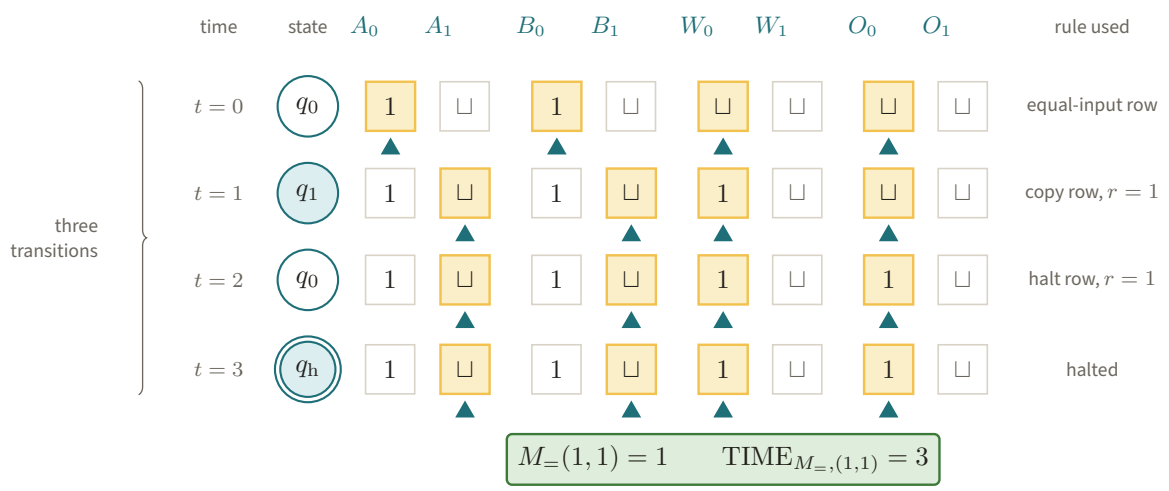


Figure 8. Four time rows track the A, B, W, O tapes of $M_=$, with amber cells and triangles marking every scanned symbol and head. The rule labels expose why equal inputs produce output one and reach q_h after exactly three transitions.

The paper's *decider* is a total five-input machine

$$D(n, x, y, a, b) \in \{0, 1\}.$$

Here n is an index written in binary, x, y are the two questions, and a, b are the two answers. Total means that D halts on every such five-tuple. Its paper-level time at index n is the worst case over the other four strings. A normal-form verifier is a pair $V = (S, D)$, where the sampler S produces the questions and D checks the answers. Figure 9 draws that interface without bundling away any of the decider's five inputs.

Why this matters here. The five inputs are not later bundled by sleight of hand. When we translate a decider to a lambda term, its argument is a five-component encoded tuple; when we translate back, those components initialize five separate tapes. The worst-case convention is what lets a resource bound hold independently of unread suffixes of arbitrarily long answer strings.

Worked verifier example. Let S always ask the one-bit questions $x = y = 0$, and let D ignore n, x, y and run $M_=$ on the first bits of a, b. Then the players are accepted exactly when their one-bit answers agree, and the decision stage takes three transitions plus fixed input-parsing overhead.

1.4 Descriptions are finite data

Because Q, Γ, and δ are finite, we may serialize them with a fixed prefix-free convention. The resulting bit string is the machine description $\langle M \rangle$; the paper also writes $\overline{M}$. Its length in bits is

$$|M| := |\langle M \rangle|.$$

Malformed strings are assigned a fixed rejecting machine, so every bit string still has a defined interpretation. The encoding is syntactic: extensionally equivalent machines may have different codes, sizes, and running times. A useful physics analogy is that a description is like a written Hamiltonian, not its spectrum. The actual definition is the serialized state set and transition table, and an algorithm may inspect those bits without solving the machine's behavior. Figure 10 makes the syntax–behavior distinction concrete for the five-row equality machine.

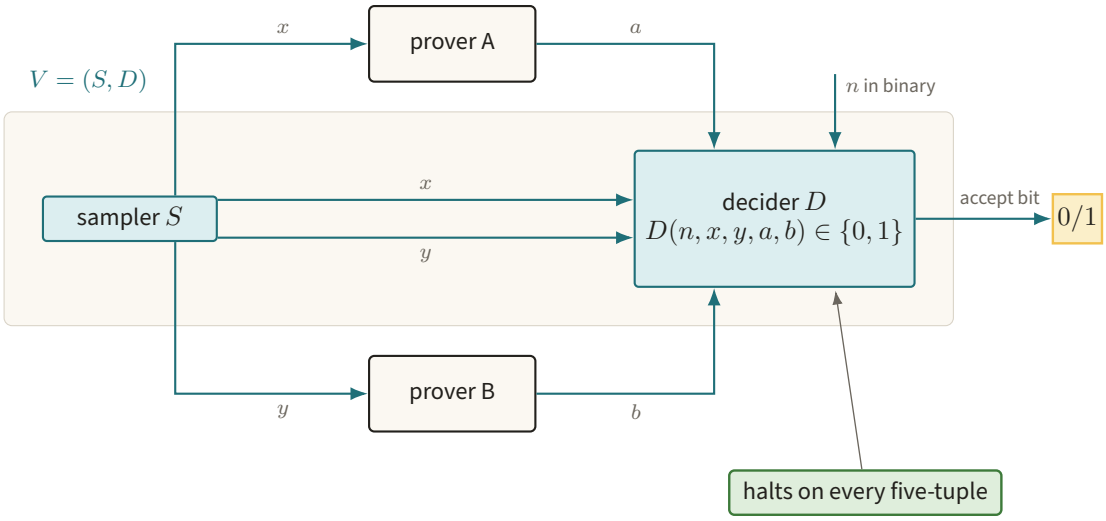


Figure 9. The sampler sends x, y to the two provers, while the total decider receives the copied questions together with n and returned answers a, b. This five-port interface is the normal form preserved by the later source transformations.

Why this matters here. The size bound $|V| \leq \lambda$ constrains the verifier's source code, not the number of functions extensionally equivalent to it. Transformations such as introspection and answer reduction copy, scan, hardwire, and simulate that source. They cannot be defined merely on a black-box input/output relation.

Worked description example. The code of $M_{=}$ contains tags for the three states and the five rows above. A compiler can prepend a wrapper saying "run this code, but reject after three simulated steps." It produces new finite code without having to decide any semantic property of $M_{=}$. The compilation itself is drawn in Figure 11.

A verifier description is the ordered pair $\langle V \rangle = (\langle S \rangle, \langle D \rangle)$. In `fig:compress`, Compress literally receives $(\langle V \rangle, \lambda)$, constructs descriptions for introspection, answer reduction, and parallel repetition, and returns the description of a new verifier. This is why it is meaningful for Compress to take "a verifier" as input: it takes the finite string naming the verifier's rules, not an infinite table of all possible plays.

2 Universality: one machine can interpret all machines

2.1 The universal machine

Fix an encoding of tuples and programs. A universal machine U_k is a single machine satisfying

$$U_k(\langle M \rangle, x_1, \ldots, x_k) = M(x_1, \ldots, x_k)$$

whenever the right side halts, and diverging when that simulated run diverges. Operationally, U_k keeps the code of M on a read-only region, stores an encoded simulated configuration, finds the matching transition-table entry, and updates the encoding. Universality therefore says that descriptions can be used as ordinary input data.

The paper needs the quantitative version. With its dual-rail tuple encoding, there are universal constants $C_U, c_U \geq 1$ such that a T-step run of a k-input machine is reproduced within

$$C_U \big(k|\langle M \rangle||x|T\big)^{c_U}, \qquad |x| = \sum_i |x_i|.$$

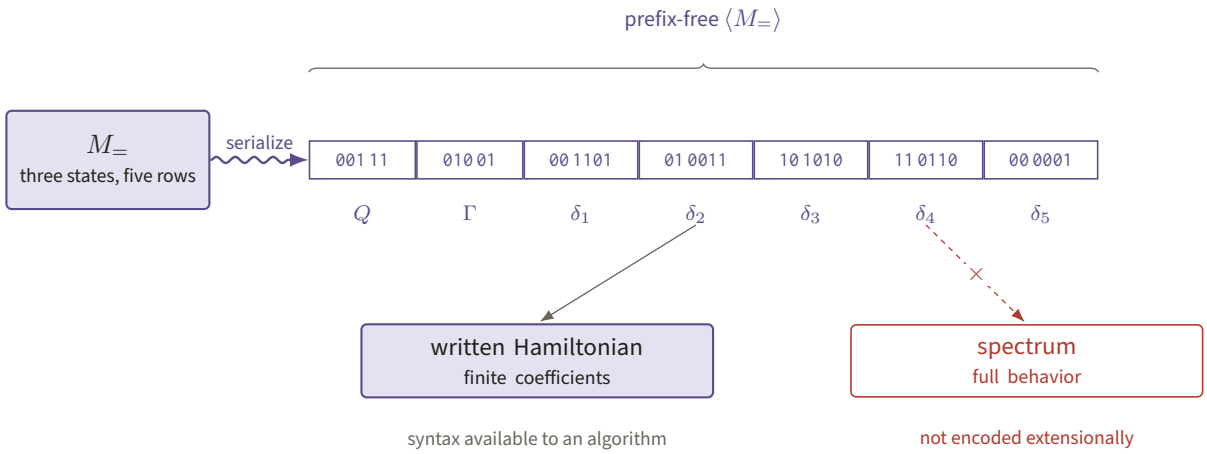


Figure 10. A prefix-free strip stores the state alphabet and all five transition rows of $M_{=}$, with field boundaries remaining available to a parser. Like a written Hamiltonian rather than its spectrum, $\langle M_{=} \rangle$ records finite syntax rather than the machine's full behavior.

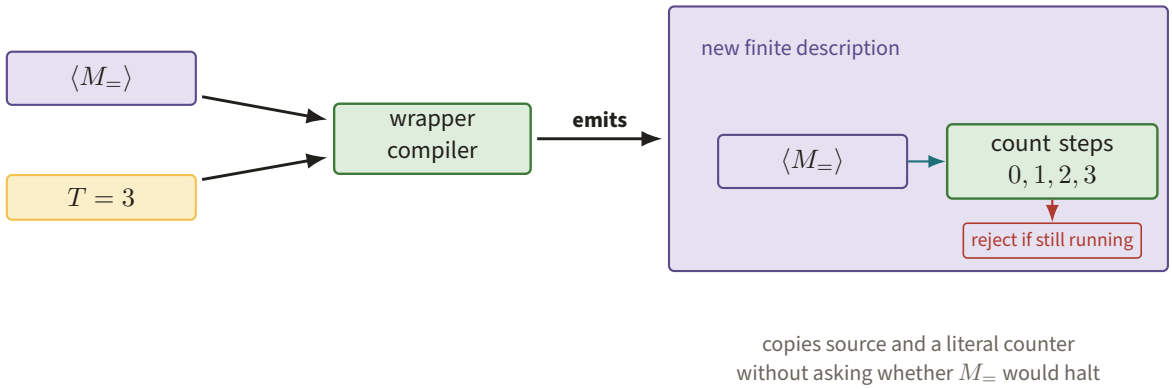


Figure 11. The compiler copies $\langle M_{=} \rangle$ beside a literal three-step counter whose exhausted branch rejects. Producing this bounded wrapper is a terminating syntactic operation and never requires deciding the wrapped machine's unbounded behavior.

No particular exponent is sacred. What matters is that the overhead is one fixed polynomial after the encodings have been fixed. Figure 12 exposes the read–lookup–update loop behind this quantitative simulation statement.

Why this matters here. Generated verifiers call old verifiers as subroutines. Their finite source contains the universal interpreter plus the old description; it does not contain a separate hand-compiled copy of every possible transition. The polynomial overhead is later absorbed into a new resource exponent.

Worked universal simulation. On input $(\langle M_{=} \rangle, 1, 0)$, U_2 parses the five table rows, builds the initial four-tape configuration, and simulates the three transitions above. It outputs zero. The interpreter is fixed; changing the two input bits or replacing $\langle M_{=} \rangle$ changes only its data.

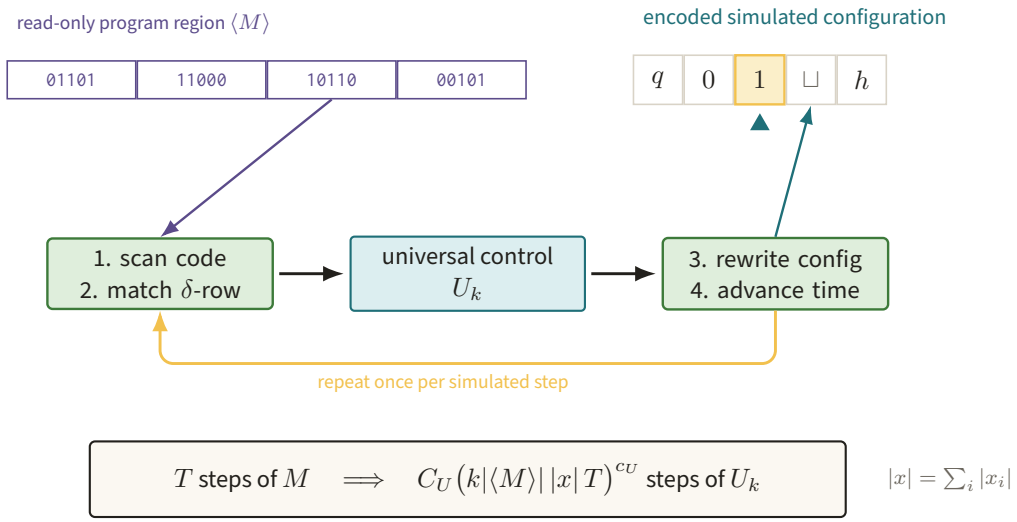


Figure 12. The fixed control of U_k alternates between a read-only program region and the encoded simulated configuration, matching one δ-row before each update. Iterating this real lookup loop yields the single fixed-polynomial overhead used by generated verifiers.

2.2 Parameter fixing: the s-m-n theorem

The notation s-m-n packages a simple compiler operation. If program e expects $r + k$ arguments, there is a total computable function $s_{r,k}$ on bit strings such that

$$\varphi^{(k)}_{s_{r,k}(e,z_1,\ldots,z_r)}(x_1,\ldots,x_k) = \varphi^{(r+k)}_e(z_1,\ldots,z_r,x_1,\ldots,x_k).$$

The left program has the first r values hardwired into its source.

Proof idea in two lines. Print a wrapper containing one literal copy of $e, z_1, \ldots, z_r$. On input x, the wrapper forms the tuple $(e, z_1, \ldots, z_r, x)$ and invokes the universal machine; printing this wrapper is itself a terminating, effective source transformation. Figure 13 shows exactly which slots become source literals and which remain live ports.

Why this matters here. Hardwiring is the bridge from "this value is supplied at run time" to "this value is part of the verifier description." Compression constantly fixes a verifier, a level, or a time exponent while leaving (n, x, y, a, b) as the live inputs. Part II calls the corresponding operation Specialize.

Worked specialization. Apply $s_{1,1}$ to the two-input equality machine and the static bit 1. The resulting one-input program has code $e_1 = s_{1,1}(\langle M_= \rangle, 1)$ and satisfies $\varphi_{e_1}(a) = M_=(1, a)$. Thus it is the one-bit identity test, even though its source is a universal-simulation wrapper rather than a newly invented transition table.

2.3 Why the Halting problem is undecidable

Let

$$\mathsf{HALT} = \{(\langle M \rangle, x) : M(x) \neq \bot\}.$$

The Halting problem asks for a total decider H for this set. The obstruction is diagonalization, and it is worth seeing every step.

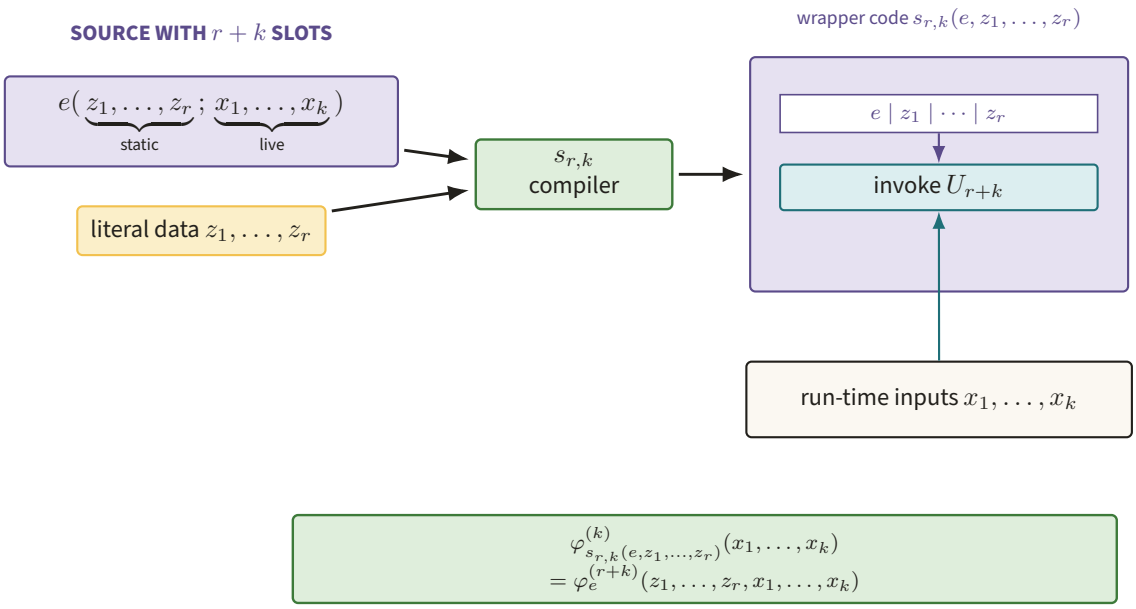


Figure 13. The $s_{r,k}$ compiler moves $e, z_1, \ldots, z_r$ into a literal byte region while preserving $x_1, \ldots, x_k$ as run-time inputs. The resulting wrapper realizes parameter fixing by one finite source transformation followed by universal simulation.

Diagonal proof. Assume, for contradiction, that $H(e, x)$ always halts and returns one exactly when the program described by e halts on x. Construct a one-input machine D as follows on input z:

1. Run $H(z, z)$.
2. If H returns zero, halt.
3. If H returns one, loop forever.

This is a legitimate finite program because H was assumed to be a total subroutine. Let $d = \langle D \rangle$ and run $D(d)$.

If $H(d, d) = 0$, then by the specification of H, $D(d)$ does not halt; but Step 2 says that it halts. If $H(d, d) = 1$, then the specification says that $D(d)$ halts; but Step 3 says that it loops forever. The two possible output bits both yield contradictions. Therefore no such total H exists. □

Figure 14 places the two contradictory cases directly beside the self-input entry that D flips.

This proof does not say that a particular long run can never be understood. It says there is no single terminating algorithm which answers the halting question correctly for every program-description/input pair.

Why this matters here. The $\mathrm{MIP}^* = \mathrm{RE}$ construction reduces the behavior of an arbitrary machine to the value of a nonlocal game. Its Halting verifier is given $\langle M \rangle$, embeds and simulates that description, and asks whether bounded prefixes of the computation are consistent. It does not contain an impossible halting decider; rather, the infinite family of game instances reflects the recursively enumerable process of eventually seeing a halt.

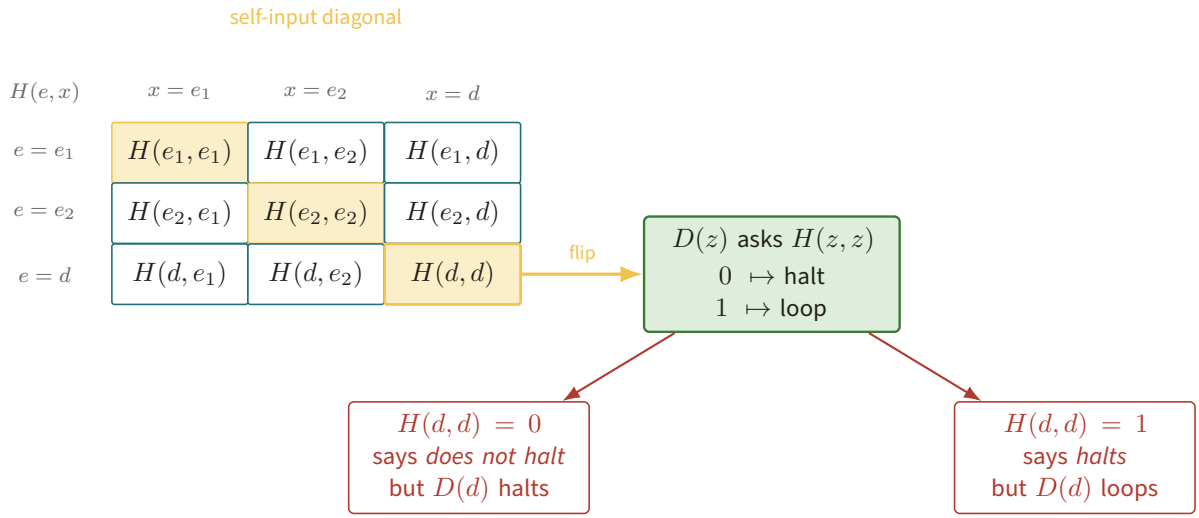


Figure 14. The amber diagonal specializes $H(e,x)$ to self-inputs, and D reverses the highlighted value $H(d,d)$. Both possible bits terminate in a red contradiction, so no total table-filling decider H can exist.

Worked semidecision. For a fixed M, x, simulate one more step at each stage $t = 0, 1, 2, \ldots$. If $M(x)$ halts after T steps, stage T witnesses it. If it never halts, this search itself never returns. That one-sided behavior is the "RE" side of the theorem's computability interface.

Figure 15 separates the finite yes-witness from the deliberately absent no-return.

3 Programs which obtain their own descriptions

The diagonal argument used a program on its own code. Kleene's second recursion theorem turns that move into a general construction. In its most useful form here:

> Every total computable transformation of programs has a program that behaves like its own transform.

More formally, if Q is a total computable map from descriptions of k-input programs to descriptions of k-input programs, then there is a description e_Q such that

$$\varphi^{(k)}_{e_Q} = \varphi^{(k)}_{Q(e_Q)}.$$

This is behavioral equality, not necessarily literal equality of bit strings.

For the paper, this theorem buys a finite verifier description whose behavior may depend on that very description.

Theorem 3.1 (Kleene's second recursion theorem, refresher form)**.** *Every total computable transformation Q of k-input program codes has an effectively constructible behavioral fixed point e_Q.*

The self-specialization square and its four equalities are laid out in Figure 16.

Proof. Use parameter fixing for one static and k live arguments. Write $s(z,w)$ for the code obtained by fixing the first input of a suitable $(k+1)$-input program z to the literal w; hence

$$\varphi^{(k)}_{s(z,w)}(x) = \varphi^{(k+1)}_{z}(w,x).$$

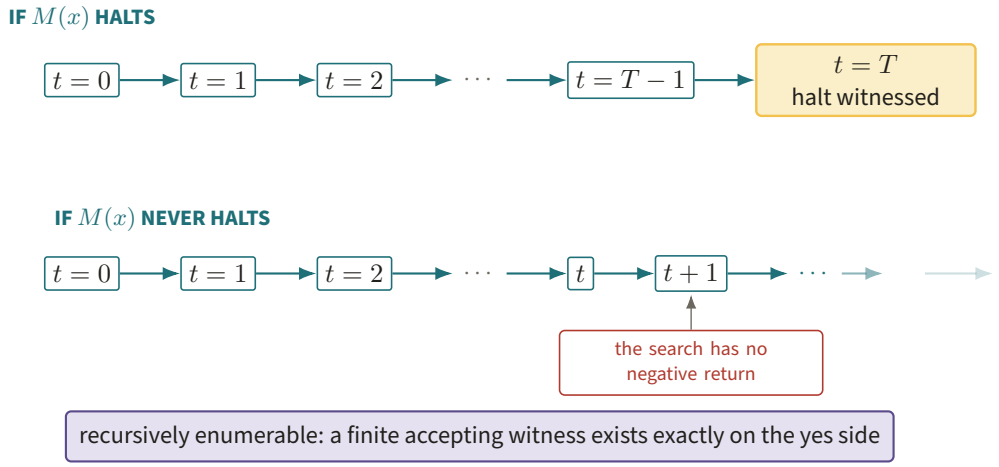


Figure 15. Stage T supplies a finite halting witness on the upper timeline, while the lower search extends forever without producing a negative answer. This asymmetric stopping behavior is exactly the recursively enumerable interface used by the Halting verifier.

Now define the partial computable function

$$g(z, x) := \varphi^{(k)}_{Q(s(z,z))}(x).$$

There is a finite $(k+1)$-input program p for g: on (z, x), it computes $s(z, z)$, runs the total transformer Q on that string, and universally simulates the resulting program on x. Set $e_Q := s(p, p)$. Then, for every x,

$$\begin{aligned}\varphi^{(k)}_{e_Q}(x) &= \varphi^{(k)}_{s(p,p)}(x) \\ &= \varphi^{(k+1)}_{p}(p, x) \\ &= \varphi^{(k)}_{Q(s(p,p))}(x) \\ &= \varphi^{(k)}_{Q(e_Q)}(x).\end{aligned}$$

If the last computation diverges, each displayed partial computation diverges for the same reason; if it halts, all return the same output. Thus the partial functions are equal on every input. Every source-manipulating step used to construct e_Q terminates, so the fixed point is effective. □

Why this matters here. The Halting verifier must pass its own decider description into Compress. The theorem says that this is ordinary program construction, not a circular definition requiring an infinite source string. It also separates two facts: constructing the fixed-point code terminates, while running the partial program denoted by that code may or may not terminate.

Worked quine-like example. Let $Q(z)$ be the code of the zero-input program "print the literal string z." The transformation Q is total because it merely prints a wrapper. Its fixed point e_Q satisfies $\varphi_{e_Q} = \varphi_{Q(e_Q)}$, so running e_Q prints e_Q itself. The theorem has built a quine without requiring the source printer to know its own final address in advance. Figure 17 traces the identical finite string from code to output.

3.1 The paper's $\mathcal{F}(\mathcal{F}, M, \lambda, \ldots)$ construction

The paper writes the fixed point in an unrolled form. Begin with an eight-input machine

$$\mathcal{F}(f, m, \lambda, n, x, y, a, b).$$

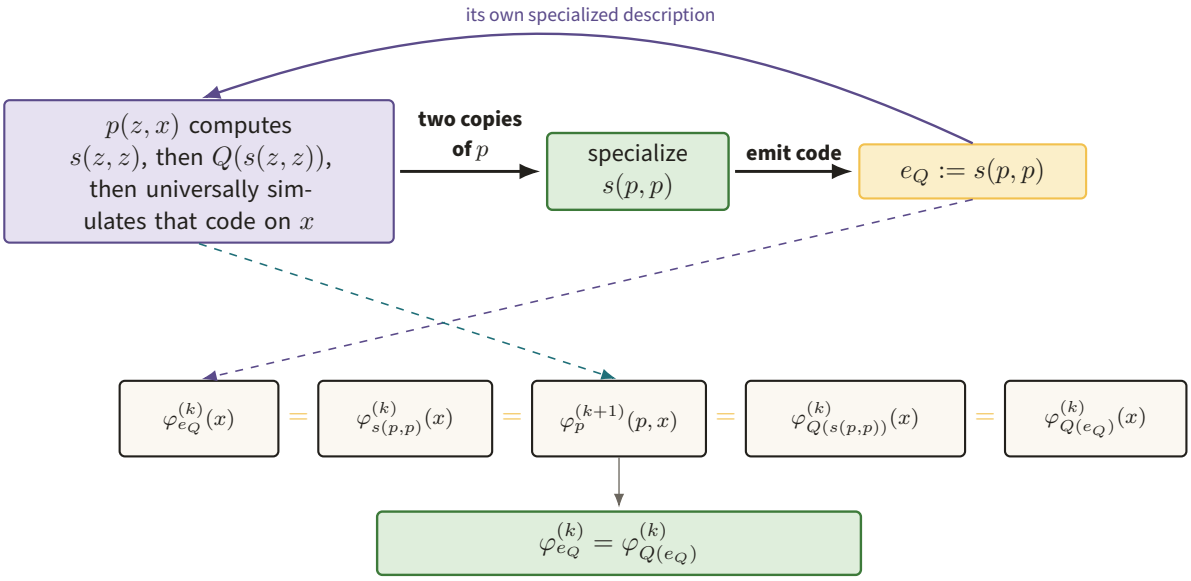


Figure 16. Two literal copies of p enter specialization to create $e_Q = s(p, p)$, whose run follows the five expressions joined by four amber equalities. The commuting construction turns apparent circularity into finite code generation plus ordinary universal simulation.

After its initial n-step simulation has not halted, its first source action is computable: hardwire (f, m, λ) into the first three slots of $\mathcal{F}$'s template, obtaining a five-input decider description d. It pairs d with a sampler description, invokes Compress on that verifier description, and runs the resulting decider on (n, x, y, a, b). The published call is

$$\mathcal{F}(\langle\mathcal{F}\rangle, \langle M\rangle, \lambda, n, x, y, a, b).$$

After the first three arguments have been fixed, the produced d is exactly the description of the five-input program currently being run. Hence the verifier passed to Compress contains its own decider code. Figure 18 follows both the bounded-halting shortcut and the complete feedback path through Compress.

Why this matters here. This is Kleene's construction with the diagonal substitution exposed as pseudocode. The transformation Q says "insert the candidate code into a verifier, compress it, then run the returned decider." The $\mathcal{F}(\mathcal{F}, \ldots)$ trick supplies $s(p, p)$ directly. An earlier version of the paper invoked Kleene's recursion theorem explicitly; the published presentation changes the notation, not the computability fact.

Worked schematic expansion. Suppress the fixed parameters M, λ and denote the five live inputs by u. For a proposed self-description f, put $d_f = \text{hardwire}(\langle\mathcal{F}\rangle, f)$. Substituting $f = \langle\mathcal{F}\rangle$ gives

$$d_{\langle\mathcal{F}\rangle}(u) = \mathcal{F}(\langle\mathcal{F}\rangle, u) = Q(d_{\langle\mathcal{F}\rangle})(u),$$

which is the recursion-theorem equation with the compiler operation written out rather than hidden under the symbol s.

4 Lambda calculus from scratch

Turing machines make state and local time explicit. Lambda calculus makes substitution and composition explicit. We start with its untyped core, because fixed-point combinators live there naturally, and only later

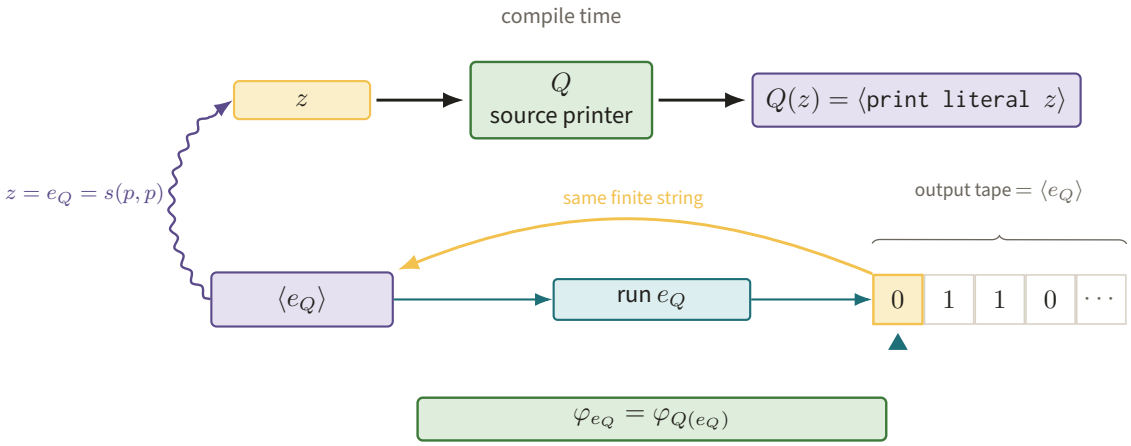


Figure 17. Specializing the source printer at its own generated code produces e_Q, whose output tape contains the same finite description $\langle e_Q \rangle$. The returning amber arrow emphasizes behavioral self-reproduction without requiring a source string of infinite regress.

return to the typed, fuelled language used by the project.

4.1 Terms, scope, and beta reduction

A lambda term is generated by

$$t ::= x \mid (\lambda x.t) \mid (t\ t).$$

A variable occurrence is *bound* if it lies inside a matching λx; otherwise it is *free*. For example, in $\lambda x.(x\ y)$, the occurrence of x is bound and y is free. Changing a bound name consistently is *alpha-renaming*: $\lambda x.x =_\alpha \lambda z.z$. It changes typography, not the term's binding structure. Figure 19 makes that unchanged binding structure visible as an edge from binder to occurrence.

Application associates to the left and abstraction extends to the right, so $f\ a\ b$ means $(f\ a)\ b$, while $\lambda x.f\ x$ means $\lambda x.(f\ x)$. The computational rule is beta reduction,

$$(\lambda x.t)\ u \longrightarrow_\beta t[x := u],$$

where substitution replaces the free occurrences of x in t and first alpha-renames binders when necessary to avoid capture. Figure 20 reads beta reduction literally as tree surgery.

Why this matters here. Hardwiring a machine input and applying a function are both forms of substitution. Our later Specialize operation is the source-code version: it fills typed holes while preserving scope. Being precise about bound and free variables is what prevents a verifier description from changing meaning when code is inserted under a binder.

Worked reduction 1. The identity combinator $I = \lambda x.x$ satisfies

$$(\lambda x.x)\ ((\lambda z.z)\ w) \longrightarrow_\beta (\lambda z.z)\ w \longrightarrow_\beta w.$$

Worked reduction 2: avoiding capture. Naively substituting y for x in $(\lambda x.\lambda y.x)\ y$ would incorrectly bind the free y. Rename the inner binder first:

$$(\lambda x.\lambda y.x)\ y =_\alpha (\lambda x.\lambda z.x)\ y \longrightarrow_\beta \lambda z.y.$$

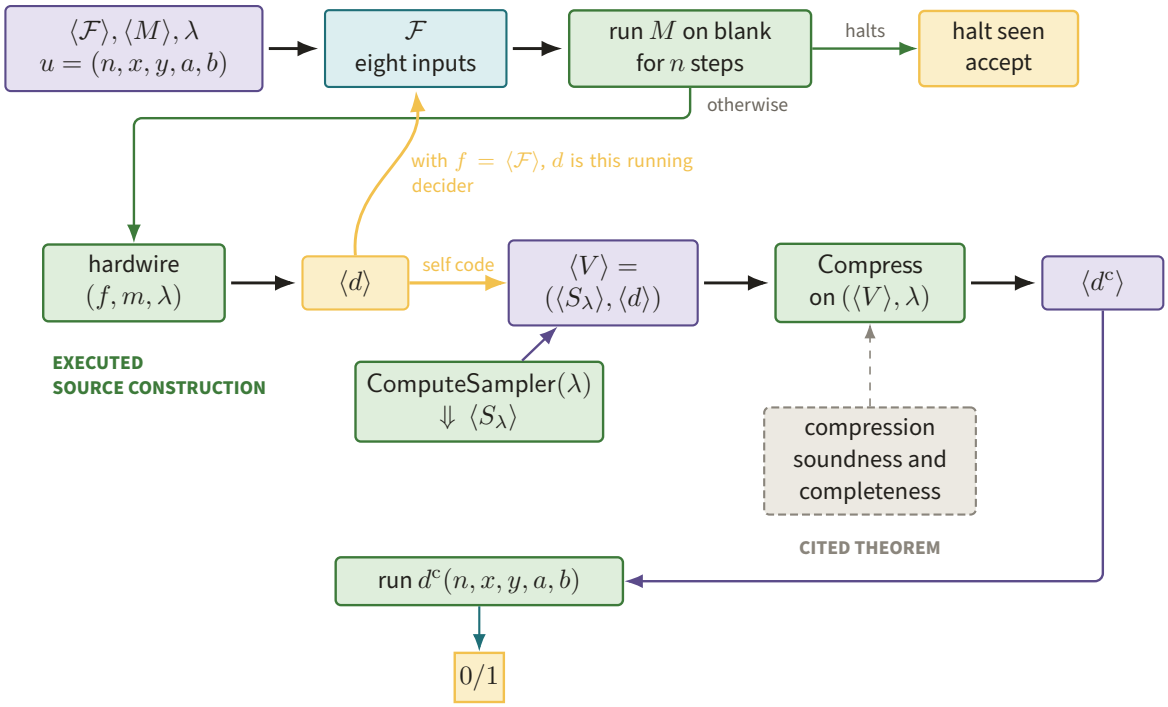


Figure 18. The eight-input machine first accepts a witnessed n-step halt; otherwise it hardwires (f, m, λ), feeds the resulting self-code d into (S_λ, d), compresses, and runs the returned decider. Green boxes are executed source operations while the dashed slate box is the cited game-theoretic guarantee (after `fig:halt_f`).

The surviving y is free, as it must be. The two outcomes are contrasted in Figure 21.

A *beta-redex* is a subterm of the form $(\lambda x.t)u$. A *normal form* contains no beta-redex. Some terms have none: with $\Omega = (\lambda x.xx)(\lambda x.xx)$, one has $\Omega \to_\beta \Omega$ forever. The Church–Rosser, or confluence, theorem states that if one term reduces by different paths to u and v, then u and v have a common reduct. In particular, a normal form, if it exists, is unique up to alpha-renaming. We use this theorem as orientation and do not prove it here. The cited joining property is isolated in Figure 22.

Why this matters here. Confluence says that the mathematical result does not depend on which redex is chosen, provided reduction reaches a normal form. Resource accounting is stricter: different strategies may take very different numbers of steps, and one may diverge while another reaches the answer. The project therefore fixes an evaluator instead of treating all beta paths as operationally equal.

Worked strategy example. For $K = \lambda x.\lambda y.x$,

$$K\ I\ \Omega \longrightarrow_\beta^* I$$

under normal order, which reduces the leftmost outermost redex first and never evaluates the unused argument. Call-by-value first tries to turn Ω into a value and therefore loops. This difference is exactly why the usual Y combinator needs an eta-delayed cousin under call-by-value. Figure 23 puts the terminating and looping schedules on the same starting term.

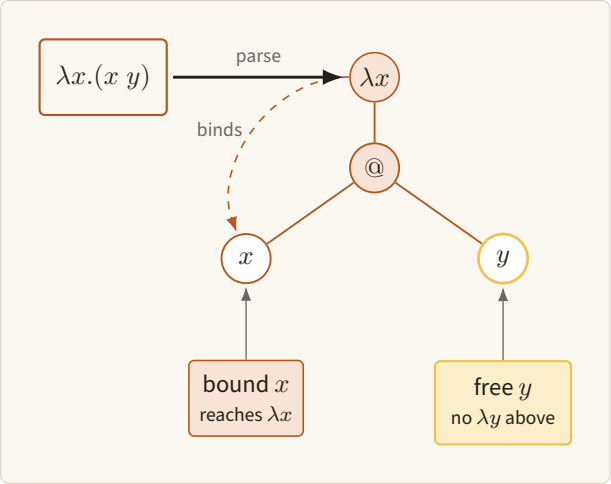


Figure 19. The tree for $\lambda x.(x\ y)$ links the binder to its bound x while amber isolates the free y. Scope is structural, so consistent renaming changes labels without changing this graph.

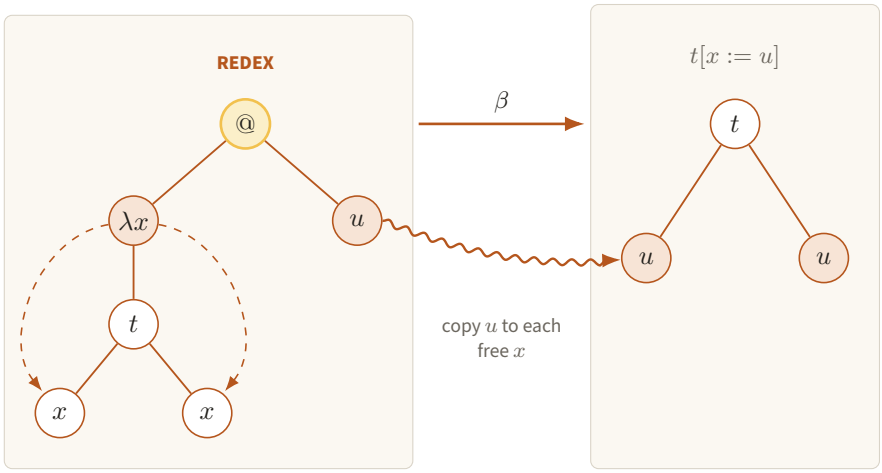


Figure 20. The amber application is cut away and the argument subtree u is spliced into every free x-leaf of t. This concrete operation is the term-level model for later code specialization.

4.2 Church data: behavior as representation

Pure lambda calculus has no primitive bits, tuples, or integers. Church encodings represent data by what eliminator it applies. These encodings are not the project's efficient runtime representation; they show that the bare calculus is computationally expressive.

Booleans. Define

$$\mathsf{true} = \lambda t.\lambda f.t, \qquad \mathsf{false} = \lambda t.\lambda f.f.$$

They choose the first or second continuation. For example, $\mathsf{true}\ A\ B \to_\beta^* A$, whereas $\mathsf{false}\ A\ B \to_\beta^* B$ — two steps each, since both arguments must be consumed.

Why booleans matter here. A verifier ultimately returns one acceptance bit. In the resource language we use a primitive bit sort, but its conditional has precisely the selection behavior exhibited by these terms.

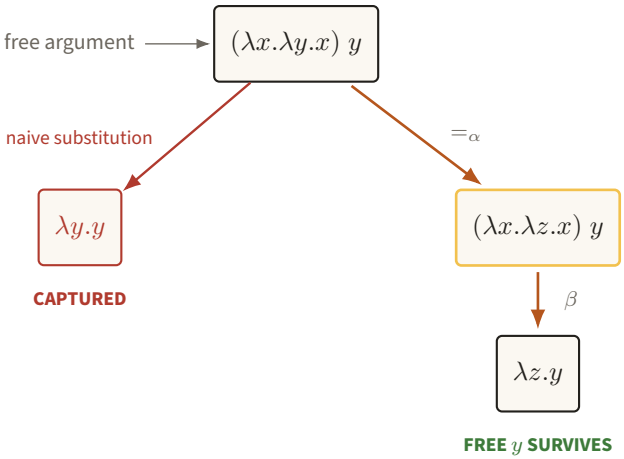


Figure 21. Naive substitution captures the argument's free y in red, whereas alpha-renaming the inner binder preserves it. Capture avoidance is the scope invariant required when code is inserted below binders.

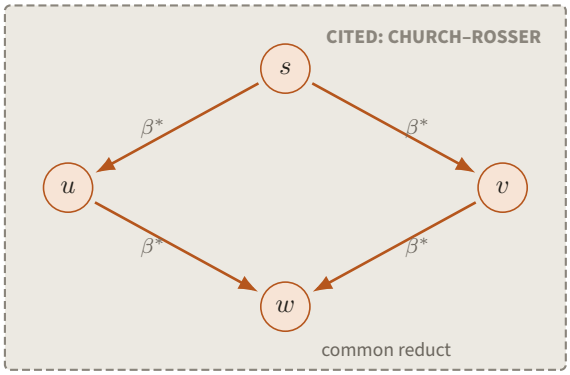


Figure 22. Two reduction paths from s close to a common reduct w in the Church–Rosser diamond. The dashed slate boundary records that confluence is cited orientation, not an executed project check.

If–then–else. Define $\mathsf{if} = \lambda b.\lambda t.\lambda e.b\ t\ e$. Then

$$\mathsf{if}\ \mathsf{false}\ A\ B \to_\beta^* \mathsf{false}\ A\ B \to_\beta^* B.$$

Figure 24 draws both Church booleans as two-port selectors and follows the false branch of this conditional.

Why the conditional matters here. Every decider is built from branches on parsed bits, timeout tests, and local predicates. Fixing their evaluation order makes "the branch not taken" operationally meaningful, especially when it contains a recursive call.

Pairs. Let

$$\mathsf{pair} = \lambda a.\lambda b.\lambda p.p\ a\ b, \quad \mathsf{fst} = \lambda p.p\ \mathsf{true}, \quad \mathsf{snd} = \lambda p.p\ \mathsf{false}.$$

Writing $\langle A, B\rangle = \mathsf{pair}\ A\ B$,

$$\mathsf{fst}\ \langle A, B\rangle \to_\beta \mathsf{true}\ A\ B \to_\beta^* A.$$

The packing and two projections appear together in Figure 25.

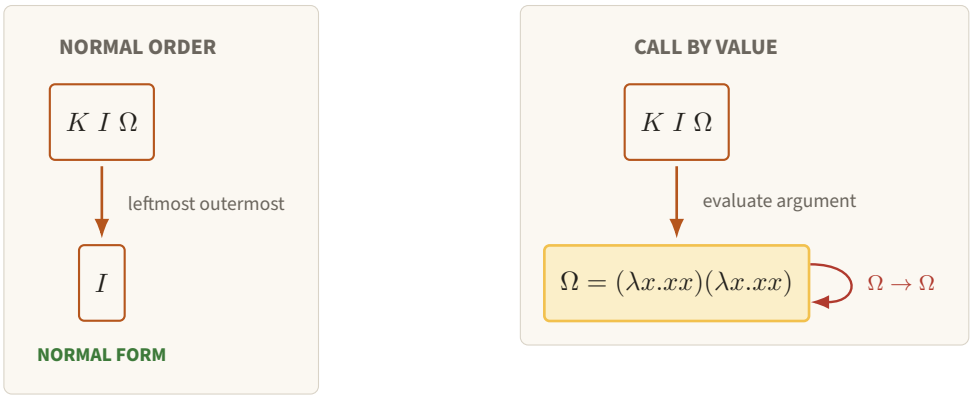


Figure 23. Normal order discards the unused Ω and reaches I, while call-by-value enters the self-loop $\Omega \to \Omega$. Evaluation strategy therefore changes the operational resource story even when confluence governs completed reductions.

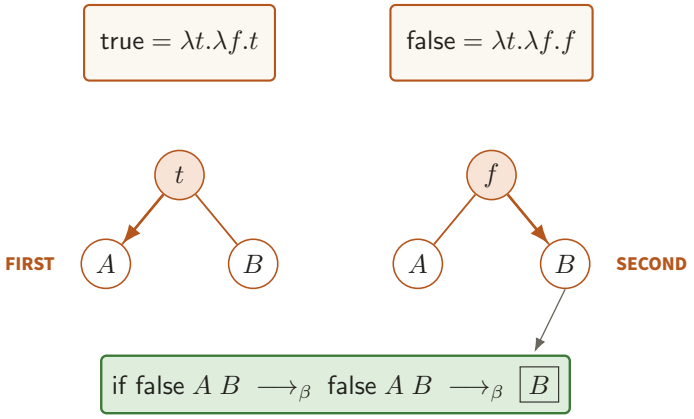


Figure 24. true and false route the same two continuations to opposite outputs, with if false $A\ B$ selecting B. A Church bit is therefore its elimination behavior, exactly the interface later implemented by a primitive conditional.

Why pairs matter here. Machine configurations, verifier descriptions, and sampler/decider outputs are finite products. The project gives tuples primitive storage so that projection has an explicit small cost, but the lambda interface is the same.

Numerals. The Church numeral n is

$$\overline{n} = \lambda f.\lambda x.f^n x, \quad \overline{0} = \lambda f.\lambda x.x, \quad \overline{2} = \lambda f.\lambda x.f(fx).$$

It represents iteration: $\overline{2}\ s\ z \to_\beta^* s(sz)$.

Why numerals matter here. An index n, a timeout, and a fuel counter are all iterated control data. Church numerals prove representability, while binary primitive naturals in Part II prevent a unary blow-up in the resource bounds.

Successor. Define succ $= \lambda n.\lambda f.\lambda x.f(nfx)$. Then

$$\text{succ}\ \overline{2} \to_\beta \lambda f.\lambda x.f(\overline{2}fx) \to_\beta^* \lambda f.\lambda x.f(f(fx)) = \overline{3}.$$

Figure 26 exposes the iteration spine and the one extra application installed by successor.

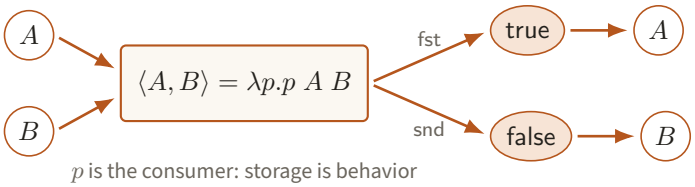


Figure 25. The Church pair stores A and B by passing both to a consumer p, while true and false project opposite components. Primitive tuples later preserve this interface while giving projection an explicit small cost.

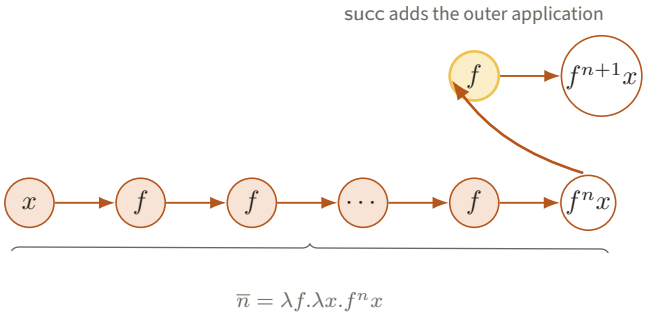


Figure 26. A Church numeral is an n-stage application spine from x to $f^n x$, and successor adds the amber outer f. The drawing separates representability by iteration from the binary storage used for efficient resource bounds.

Why successor matters here. It is the simplest example of a computable map on an encoding. Later compilers similarly build a new syntax tree around an old one and prove a size increase, rather than treating the new program as an abstract function.

Addition. Define $\mathsf{add} = \lambda m.\lambda n.\lambda f.\lambda x.mf(nfx)$. Thus

$$\mathsf{add}\ \overline{1}\ \overline{2}\ f\ x \to^*_\beta \overline{1}f(\overline{2}fx) \to^*_\beta f(f(fx)),$$

so the result is $\overline{3}$.

Why addition matters here. Fuel bounds and encoded addresses are assembled from smaller bounds. This example reminds us that arithmetic is itself computation; Part II uses efficient binary primitives and charges for their reference machines.

Predecessor by the pair trick. Predecessor is subtler because a Church numeral only says "iterate." Let

$$\Phi = \lambda p.\mathsf{pair}\ (\mathsf{snd}\ p)\ (\mathsf{succ}(\mathsf{snd}\ p)), \qquad \mathsf{pred} = \lambda n.\mathsf{fst}(n\ \Phi\ (\mathsf{pair}\ \overline{0}\ \overline{0})).$$

Each Φ-step maps $\langle j-1, j\rangle$ to $\langle j, j+1\rangle$. The complete small calculation for three is

$$\langle 0,0\rangle \xrightarrow{\Phi} \langle 0,1\rangle \xrightarrow{\Phi} \langle 1,2\rangle \xrightarrow{\Phi} \langle 2,3\rangle \xrightarrow{\mathsf{fst}} 2.$$

Hence $\mathsf{pred}\ \overline{3} \to^*_\beta \overline{2}$. The lagging-register invariant is visible step by step in Figure 27.

Why predecessor matters here. A timeout counts down, so total bounded simulation needs a predecessor and a zero test even when the simulated program diverges. The pair construction is a miniature instance of adding auxiliary state to make a desired update locally computable.

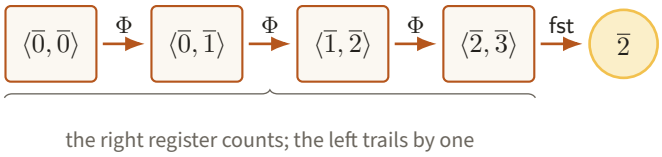


Figure 27. Three Φ-steps advance $\langle 0, 0\rangle$ to $\langle 2, 3\rangle$, after which fst exposes the predecessor 2 in amber. The auxiliary left register turns backward-looking predecessor into a local forward update.

Lists. Define a fold encoding

$$\mathsf{nil} = \lambda c.\lambda z.z, \qquad \mathsf{cons} = \lambda h.\lambda t.\lambda c.\lambda z.c\ h\ (t\ c\ z).$$

Then the list $[A, B] = \mathsf{cons}\ A\ (\mathsf{cons}\ B\ \mathsf{nil})$ satisfies

$$[A, B]\ C\ Z \longrightarrow_\beta^* C\ A\ (C\ B\ Z).$$

Figure 28 displays the resulting right-associated application spine.

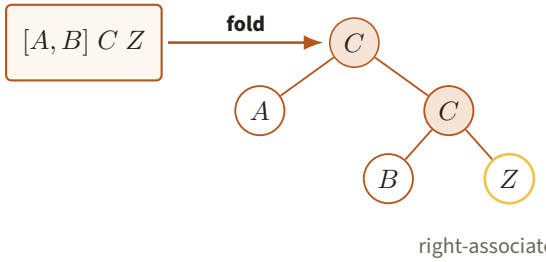


Figure 28. Folding $[A, B]$ with C and Z builds the right-leaning tree $C\ \ A\ \ (C\ \ B\ \ Z)$. This functional sequence shape becomes the tapes, environments, and continuation stacks of the resource evaluator.

Why lists matter here. Tapes, syntax children, environments, and continuation stacks are all finite sequences with potentially unbounded length. A functional list supplies the shape; the resource IR adds heap pointers and charged operations so that the cost of traversing it is visible.

4.3 Recursion from self-application

Suppose F maps a proposed recursive procedure to its body. We want a term f satisfying

$$f = Ff.$$

Think of the occurrence of f on the right as a hole. If the desired term can be written as self-application $h\ h$, then the equation asks for $h\ h = F(h\ h)$. Choose

$$h = \lambda x.F(x\ x).$$

Substituting this h into $h\ h$ yields the fixed-point combinator

$$Y = \lambda F.(\lambda x.F(x\ x))(\lambda x.F(x\ x)).$$

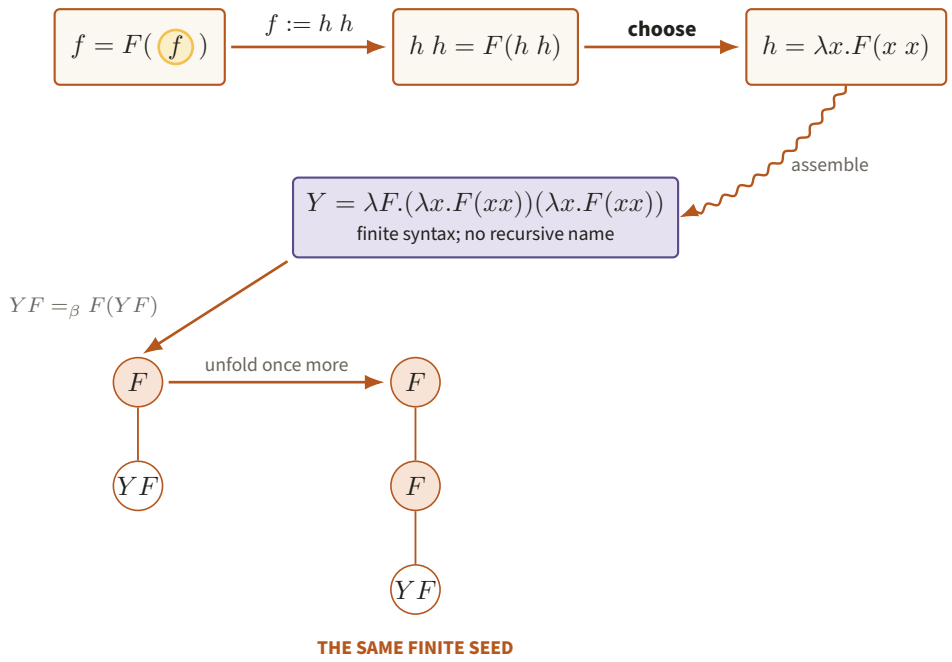


Figure 29. The fixed-point equation is solved by self-application, yielding finite Y syntax whose unfolding grows the tree $F(YF), F(F(YF)), \ldots$. Recursion is produced by running the seed rather than by storing an infinite or recursively named term.

The promised equation is a direct beta calculation:

$$\begin{aligned} Y\ F &\longrightarrow_\beta (\lambda x.F(xx))(\lambda x.F(xx)) \\ &\longrightarrow_\beta F\big((\lambda x.F(xx))(\lambda x.F(xx))\big) \\ &=_\beta F(YF). \end{aligned}$$

The last line is a beta *conversion*, not a further reduction: $(\lambda x.F(xx))(\lambda x.F(xx))$ is a reduct of YF, not YF itself, so $YF \twoheadrightarrow_\beta F(YF)$ is false for Curry's Y. Turing's combinator $\Theta = (\lambda x.\lambda y.y(xxy))(\lambda x.\lambda y.y(xxy))$ is the one that does satisfy $\Theta F \twoheadrightarrow_\beta F(\Theta F)$; the project uses the equation, never the reduction. There is no recursive name in the finite syntax of Y; recursion appears when two copies are applied. Figure 29 follows the hole equation into that finite seed and then watches the seed grow only by reduction.

Why this matters here. The lambda fixed point is the term-level analogue of Kleene's theorem. Both build a finite object whose unfolding supplies that object's own behavior. Our IR will make the analogous code link explicit and constant-size, because blindly expanding YF would obscure description length and fuel.

Worked fixed point. Take $F = \lambda r.\lambda n.\mathsf{if}\ (\mathsf{iszero}\ n)\ \overline{1}\ (\mathsf{mul}\ n\ (r(\mathsf{pred}\ n)))$. Under normal-order reduction, $YF\ \overline{3}$ unfolds only when the selected branch needs it:

$$\begin{aligned} YF\ 3 &\to F(YF)\ 3 \\ &\to 3 \cdot (YF\ 2) \to 3 \cdot 2 \cdot (YF\ 1) \\ &\to 3 \cdot 2 \cdot 1 \cdot (YF\ 0) \to 3 \cdot 2 \cdot 1 \cdot 1 = 6. \end{aligned}$$

Here the arithmetic names abbreviate their Church encodings; every displayed arrow stands for their finite beta reductions. The demanded recursive call at each level is picked out in Figure 30.

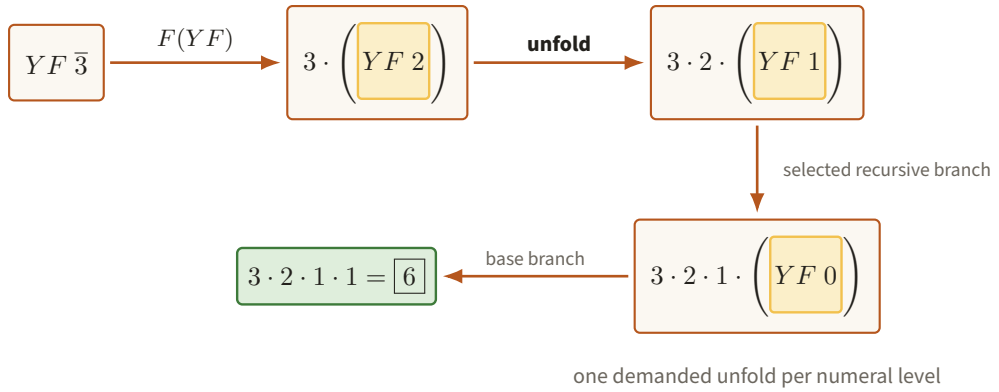


Figure 30. Normal order exposes one amber YF call per numeral level, descending from 3 to the base case before returning 6. Selective unfolding is what prevents the recursive branch from being evaluated before it is needed.

Call-by-value evaluates an argument before entering the function body, so YF tries to expand itself forever. Eta-delay the recursive occurrence:

$$Z = \lambda F.(\lambda x.F(\lambda u.xxu))(\lambda x.F(\lambda u.xxu)).$$

Now

$$ZF =_\beta F(\lambda u.(ZF)u),$$

and the recursive computation remains under a lambda until an actual argument arrives. Under call-by-value the conditional branches must likewise be thunked. With

$$F_v = \lambda r.\lambda n.\big(\mathsf{if}\ (\mathsf{iszero}\ n)\ (\lambda u.\overline{1})\ (\lambda u.\mathsf{mul}\ n\ (r(\mathsf{pred}\ n)))\big)\ I,$$

only the selected branch is applied to the dummy value I, and $ZF_v\ \overline{3} \to^*_\beta \overline{6}$ under call-by-value. Figure 31 locates the operational difference at the eta-delaying lambda.

Why evaluation strategy matters here. The project evaluator is deterministic call-by-value, so its fixed-point constructor must behave like Z, not raw Y. Once the strategy is fixed, "number of reductions" becomes an honest resource that can be translated to Turing-machine time.

4.4 De Bruijn indices and quotation

Names are pleasant for handwriting but awkward for a compiler: alpha-equal terms have many spellings, and substitution must keep inventing fresh names. De Bruijn notation replaces a bound occurrence by the number of binders between it and its binder, counting the nearest binder as zero. Thus

$$\lambda x.\lambda y.x\ y \quad \text{becomes} \quad \lambda.\lambda.(1\ 0).$$

Both $\lambda a.\lambda b.a\ b$ and $\lambda x.\lambda y.x\ y$ serialize to the same tree.

Why this matters here. The IR needs canonical bytes and capture-free specialization. De Bruijn indices remove alpha-renaming from the representation: a variable is a lexical address, and inserting a closed term cannot capture one of its free names because it has none.

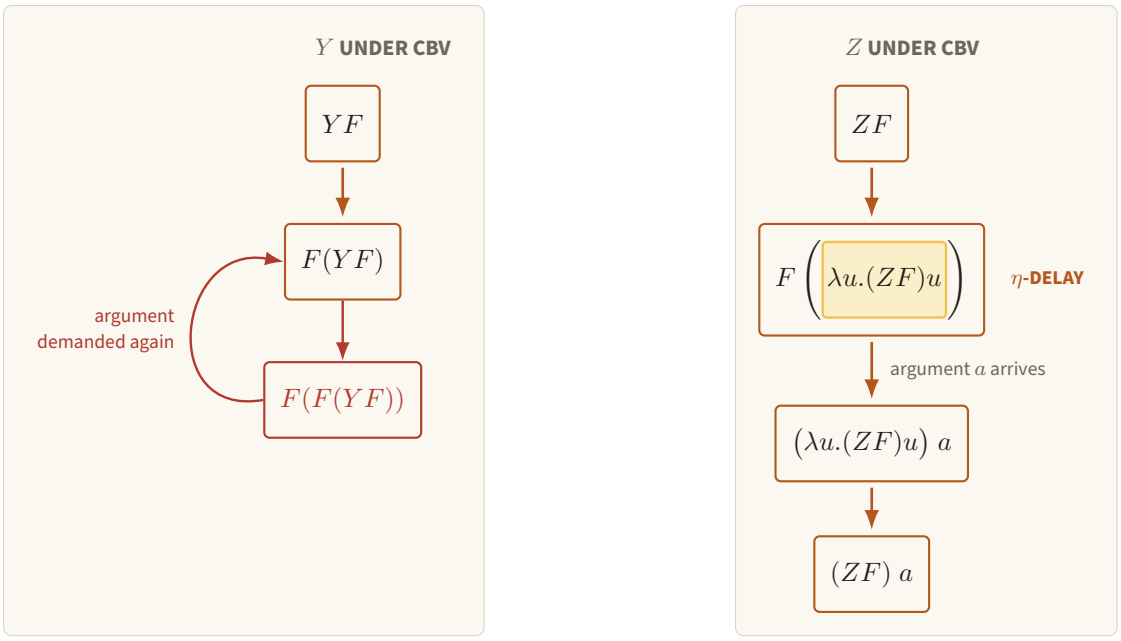


Figure 31. Under call-by-value, YF repeatedly demands its recursive argument, while ZF parks the same demand beneath the amber abstraction λu. The delayed branch unfolds only when a concrete argument arrives, making Z the appropriate fixed point for the project evaluator.

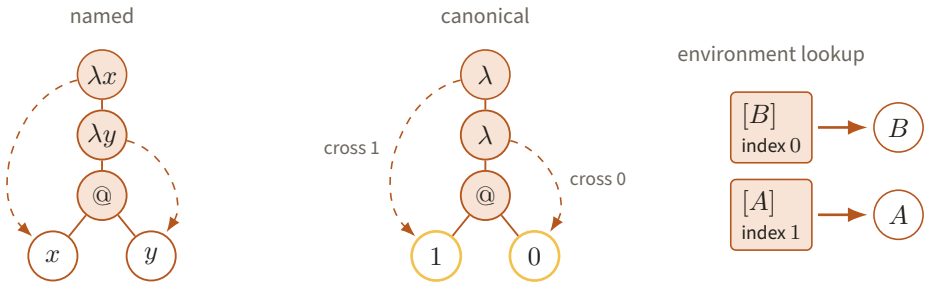


Figure 32. The named tree $\lambda x.\lambda y.x\ y$ and the canonical tree $\lambda.\lambda.(1\ 0)$ have the same binding arcs, while frames $[B], [A]$ resolve indices $0, 1$ to B, A. Lexical distance makes alpha-equivalent terms serialize identically and turns lookup into explicit frame traversal.

Worked index lookup. In $\lambda.\lambda.(1\ 0)$, applying the outer function to A and the inner function to B creates environment frames $[B], [A]$. Index zero selects B; index one crosses one frame and selects A. The body is therefore $A\ B$, just as the named term says. Figure 32 aligns the named binders, binder distances, and environment frames.

To *quote* a term is to treat its syntax rather than its value as data. A term is a finite ordered tree: each node has a constructor tag (variable, lambda, application, and so on), finite payloads, and finitely many children. Prefix-free integer and byte encodings serialize that tree to a unique bit string $\mathrm{ser}(t)$. We define

$$|t| := |\,\mathrm{ser}(t)|.$$

Quotation does not claim to compute what t will do. It merely reifies the finite tree so an evaluator or compiler can inspect it.

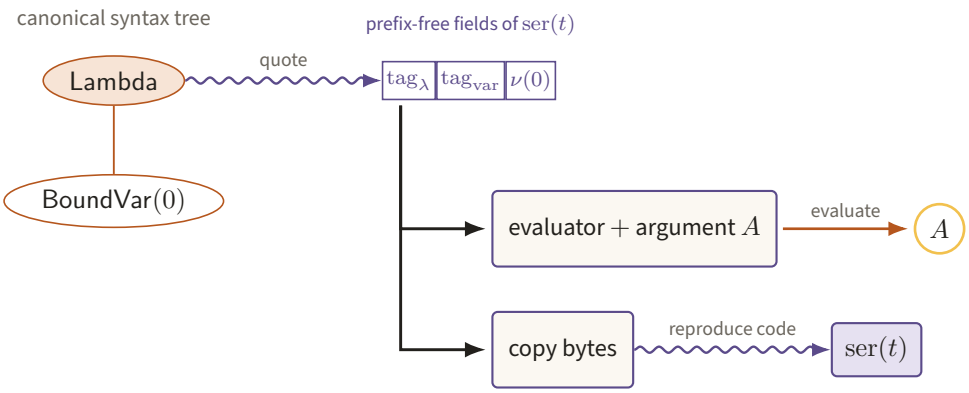


Figure 33. The identity tree quotes through a violet wavy arrow to canonical prefix-free fields, which an evaluator maps with A to A while a copier returns the fields themselves. Quotation reifies finite syntax without confusing code with the value obtained by running it.

Why this matters here. Compress must accept code, measure its length, insert static parameters, and return new code. A runtime closure with a hidden host-language environment is not adequate; a quoted, closed tree is. Quotation is the lambda-calculus version of writing $\langle M \rangle$ on a universal machine's input tape.

Worked quotation. The named identity terms $\lambda x.x$ and $\lambda z.z$ both become the de Bruijn tree Lambda(BoundVar(0)). Quoting either produces the same canonical bytes. Evaluating those bytes on A returns A; copying the bytes returns code, not A. The two consumers of the same quotation diverge exactly as Figure 33 records.

5 Church–Turing as an engineering statement

The Church–Turing thesis is often remembered as a slogan about which functions are computable. Here we need a more prosaic engineering reading: the two concrete programming media can compile into one another, and the compilers preserve both finite descriptions and feasible bounds up to fixed polynomials.

Turing machine to lambda term. Represent each tape as a functional zipper, represent a configuration as a tuple, and write a fixed recursive loop which reads the hardwired transition table and produces the next configuration. One machine step becomes a bounded collection of list, tuple, and table operations. The term contains one copy of $\langle M \rangle$, so its serialized size is linear in $|M|$; a T-step machine run becomes a polynomial number of evaluator steps. Theorem 9.1 gives the formal bound.

Why this direction matters here. It shows that replacing a paper verifier by an $\mathcal{L}$-term loses none of its behavior and makes no hidden exponential copy of its transition table.

Worked direction. For $M_=$, the recursive loop starts from the four-tape tuple, looks up the equal-input row, then the copy row, then the halt row. Three loop iterations produce the Church-level bit one, with fixed overhead for tuple and list operations. Figure 34 draws the one-cell zipper update used by each such loop iteration.

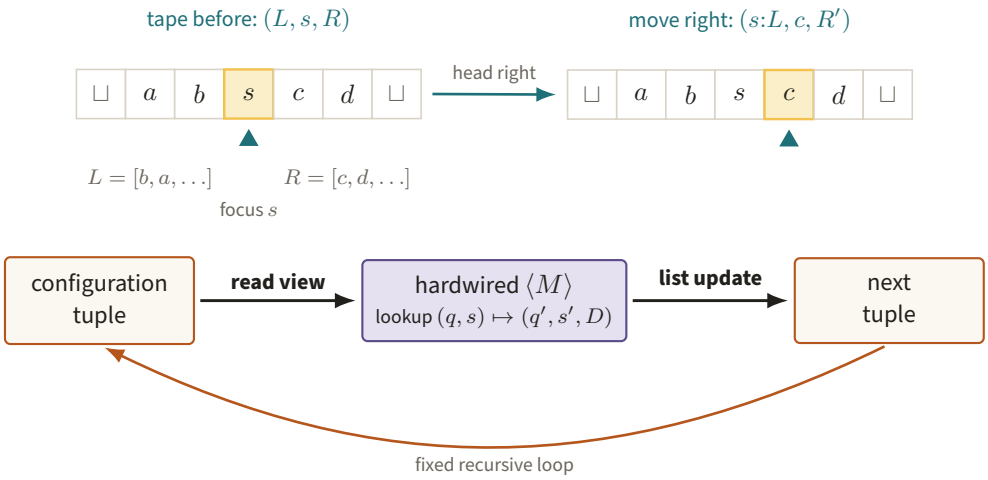


Figure 34. A right move rethreads the focused tape cell s from the zipper's right edge onto L and focuses c, while a fixed loop consults the one hardwired copy of $\langle M \rangle$. Local list updates simulate machine steps without duplicating the transition table.

Lambda term to Turing machine. Write a fixed evaluator whose work tape stores the parsed syntax tree, an environment, a continuation stack, and a fuel counter. Each evaluation rule is a finite transition routine. A run using F charged reductions is simulated in time polynomial in the code length, input length, and F. Hardwiring the quoted term into this evaluator gives a machine description linear in the serialized term. Theorem 9.2 makes this precise.

Why this direction matters here. The paper's theorems are stated for Turing-machine verifiers. Compiling a resource-accounted term back to that model is what licenses using the lambda IR without silently changing the theorem being applied.

Worked direction. For the quoted term $(\lambda x.x)\ 1$, the evaluator parses an application node, builds the identity closure, places the bit in its environment, looks up index zero, and emits one. A Turing machine can represent every one of those finite records on its work tape. Those records occupy explicit regions in Figure 35.

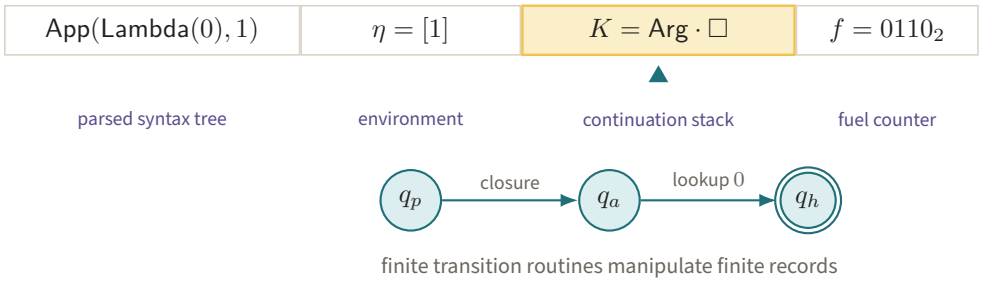


Figure 35. The evaluator's work tape contains a parsed application, environment, focused continuation frame, and binary fuel counter as four concrete finite regions. Fixed state routines can therefore simulate every charged evaluation rule with polynomial overhead.

The promised dictionary is therefore:

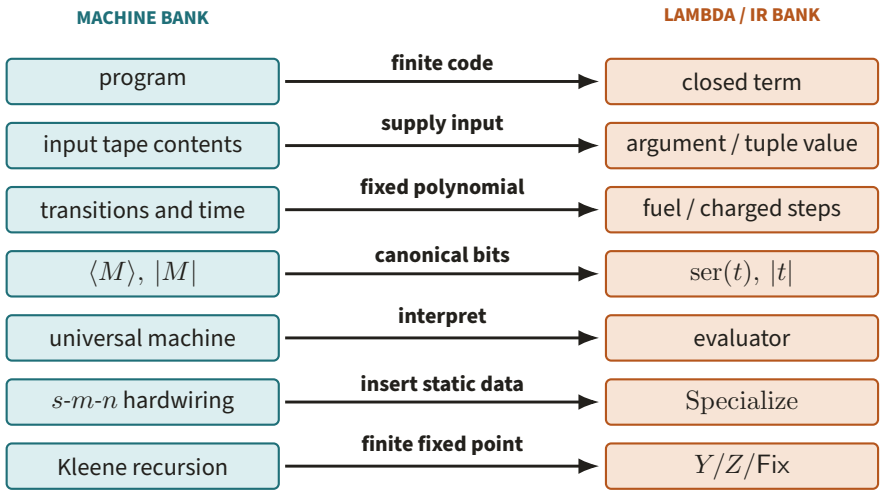


Figure 36. Seven labelled spans pair the machine bank with its lambda/IR counterpart, from finite programs and inputs through interpreters, specialization, and fixed points. The bridge records semantic agreement plus linear-size and fixed-polynomial overhead rather than step-for-step identity.

Turing-machine notion	Lambda/IR notion	Operative correspondence
Program	Closed term	Finite rule table versus finite syntax tree
Input tape contents	Argument term/value	Tuple components keep the inputs separate
Machine transitions and time	Fuel/charged reduction steps	Each simulates the other with fixed polynomial overhead
$\langle M\rangle$, $\lvert M\rvert$	Serialized quoted tree, $\lvert t\rvert$	Canonical finite data and its bit length
Universal machine	Evaluator	One fixed program interprets supplied code
s-m-n hardwiring	Specialize	A computable map from code and static data to new code
Kleene recursion theorem	Y/Z, or the IR's Fix	A finite behavioral fixed point

Figure 36 turns the same seven rows into explicit compiler spans between the two representations.

What the dictionary does not say. It does not assert step-for-step equality, equal constants, or equal description strings. It asserts a pair of explicit compilers, semantic agreement on each input, linear description growth, and polynomial running time overhead. Those are exactly the invariants needed when an exponent λ is enlarged by a universal constant.

6 Why this machinery appears in a paper about entangled provers

A nonlocal game is an analytic object: its value optimizes over strategies, and in the entangled setting those strategies use states and measurements. Nevertheless, the *outer construction* of the game is algorithmic. The compression theorem starts from the finite description of a normal-form verifier $V = (S, D)$, together with a resource exponent λ, and returns another finite verifier description. Its hypotheses constrain both $|V|$ and worst-case running times. Consequently, extensional statements such as "D decides the same predicate" are

not enough: the proof must know which code is embedded, how long it is, and how expensive its universal simulation will be.

At the description level, the main pipeline is

$$\langle V\rangle \xrightarrow{\text{Introspect}} \langle V^{(1)}\rangle \xrightarrow{\text{AnswerReduce}} \langle V^{(2)}\rangle \xrightarrow{\text{Repeat}} \langle V^{(3)}\rangle.$$

Each arrow is a terminating computable map on finite strings. Introspection embeds a description so a verifier can check a smaller representation of its own computation. Answer reduction constructs a bounded-trace circuit and then a PCP-style verifier. Repetition constructs several coordinated copies and an acceptance rule. The composite is Compress. Polynomial bookkeeping proves that its output is still a legal bounded verifier. The three terminating code transformations and their analytic boundary are separated in Figure 37.

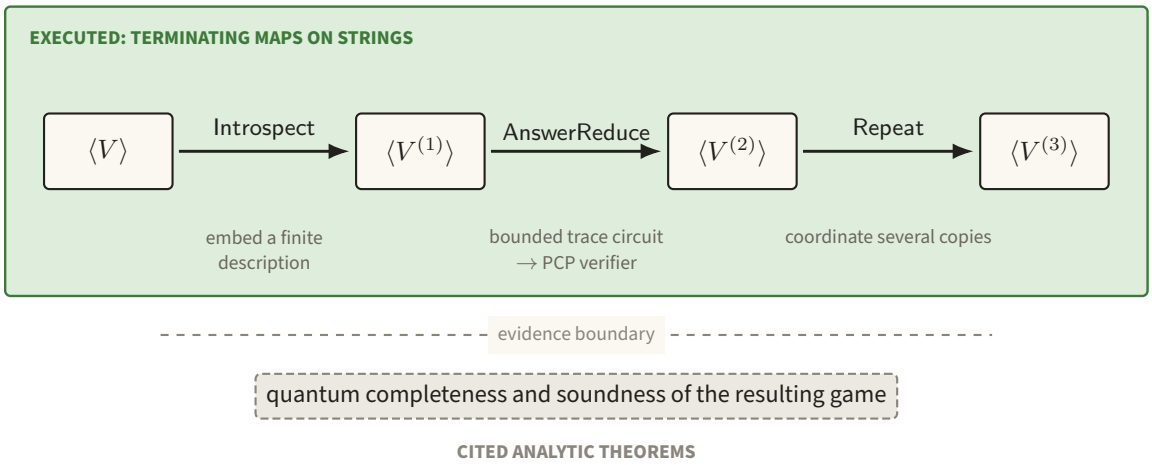


Figure 37. The green description pipeline embeds code, builds the bounded-trace PCP verifier, and coordinates copies before stopping at a dashed evidence boundary. Executing these finite transformations constructs the game but does not establish the cited quantum completeness and soundness theorem.

Why the fixed point enters. For a supplied machine M and resource bound λ, the Halting verifier must arrange that its decider is the one appearing inside the verifier fed to Compress. This is a fixed-point equation on descriptions. Kleene's theorem, the explicit $\mathcal{F}(\mathcal{F}, M, \lambda, \ldots)$ program, and the lambda fixed-point constructor are three presentations of the same outer operation. The point is not mystical self-awareness: it is a finite compiler which prints code containing an appropriate code literal.

Worked pipeline fragment. Suppose the input decider has code d. Answer reduction first constructs a succinct circuit whose hardwired local transition rule is read from d. It then emits a new decider code d' that reconstructs and checks the corresponding local constraints. The transformation $d \mapsto d'$ can be run by a Turing machine, and its time and output length can be bounded before either decider is executed. Compression composes several transformations of exactly this kind.

What is not computability theory. The dictionary stops at the boundary between constructing a game and proving what quantum strategies can do in it. The low-degree test's soundness, rigidity of observables, state-dependent approximation estimates, oracularization against entangled strategies, entanglement-dimension bounds, and the quantitative preservation of completeness and soundness are analytic statements.

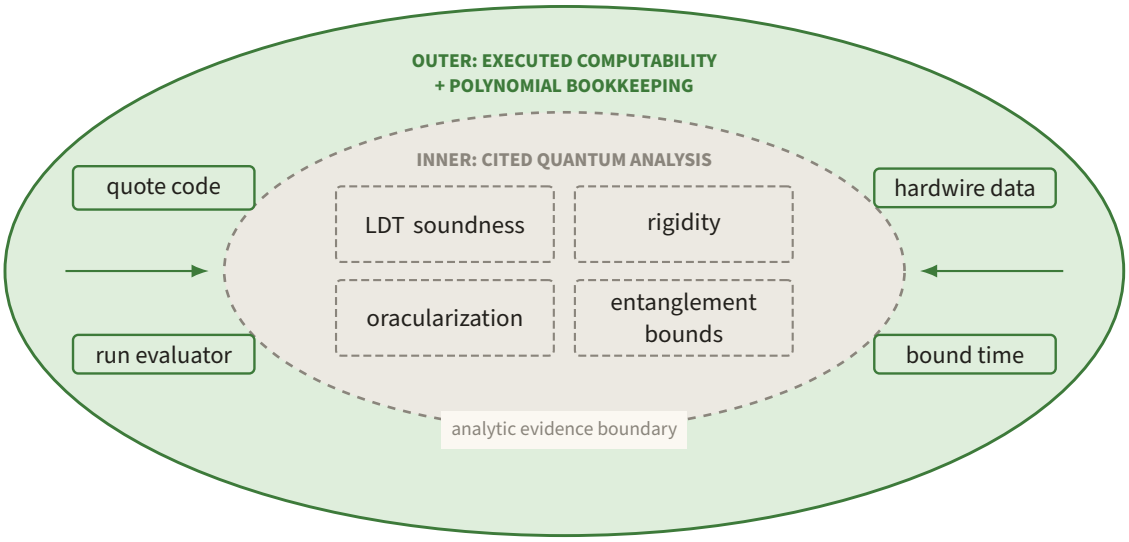


Figure 38. The green outer ring executes quotation, hardwiring, evaluation, and polynomial bookkeeping around a dashed slate core of LDT soundness, rigidity, oracularization, and entanglement bounds. Keeping this evidence boundary visible prevents source-level mechanization from being mistaken for a proof of the quantum analytic theorems.

They do not follow from universality, quotation, or a fixed-point theorem. A lambda implementation can expose the verifier transformation and check finite algebraic identities; it cannot by that fact alone prove the operator-algebraic leaves.

Why this separation matters here. It prevents two opposite mistakes. One is to treat source manipulation as informal and accidentally hide a noncomputable or super-polynomial step. The other is to imagine that a clean self-interpreter settles the quantum soundness argument. The paper needs both layers: computability theory with polynomial bookkeeping on the outside, and the genuinely quantum analytic theorems on the inside. The resource-accounted description language in Part II is designed for the first layer and records exactly where it must call upon the cited second layer, as Figure 38 makes explicit.

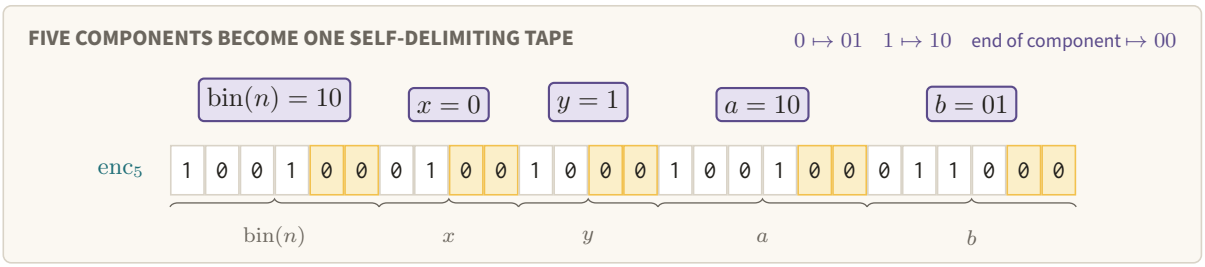


Figure 39. Dual rail turns each data bit into a two-cell codeword and marks every component boundary by the amber 00 delimiter. The same self-delimiting tape is consumed on both sides of the machine–term translation.

Part II

Resource-accounted analytic underpinnings

7 The paper's machine model and its intensional obligations

7.1 Machines, inputs, time, and descriptions

We first repeat the conventions on which the later change of description language depends. This is necessary because the change is intensional: it must preserve programs, bounds, and programs which inspect other programs, not only partial functions.

Reminder. Section 1 distinguished the finite transition table from its run. In this part, $\overline{M}$ is that table's canonical bit string, $|M|$ is its bit length, and $\mathrm{TIME}_{M,x}$ counts transitions of the run.

Definition 7.1 (The paper's Turing-machine convention)**.** A k-input Turing machine has k input tapes, one work tape, and one output tape. Tapes are one-way infinite, indexed by $\mathbb{N}$, and use $\{0, 1, \sqcup\}$. Work and output tapes are initially blank. If the machine halts, its output is the longest nonblank initial string on the output tape, with the all-blank output interpreted as the one-bit string 0. On $x = (x_1, \ldots, x_k)$, write $M(x)$ for this string and write $M(x) = \bot$ if it never halts. Its instance time $\mathrm{TIME}_{M,x} \in \mathbb{N} \cup \{\infty\}$ is the number of transitions to the halt state.

Every machine has canonical binary code $\overline{M}$. The code lists its finite states and transition table using a fixed reasonable encoding; its bit length is $|\overline{M}|$, abbreviated $|M|$. For a bit string α, $[\alpha]_k$ is the k-input machine described by α (with the fixed convention for malformed strings). Tuples are encoded by enc_k: each data bit is replaced by $0 \mapsto 01$, $1 \mapsto 10$, and each component ends in 00. Thus $|\mathrm{enc}_k(x)| = 2\sum_i |x_i| + 2k$, and encoding takes linear time.

Figure 39 makes the separators and the five decider inputs visible on the literal tape consumed by the machine.

This is the model in the Turing-machine subsection of the paper. [1] Its universal-machine convention is quantitative. For each k there is a fixed one-tape U_k, and universal constants $C_U, c_U \geq 1$, such that if $[\alpha]_k(x)$ halts in T steps then $U_k(\mathrm{enc}_{k+1}(\alpha, x))$ produces the same answer within

$$C_U\big(k|\alpha||x|T\big)^{c_U}, \qquad |x| := \sum_i |x_i|.$$

[1] Ground truth: `sec:tms` in `ground-truth/gt-03-prelim.tex`.

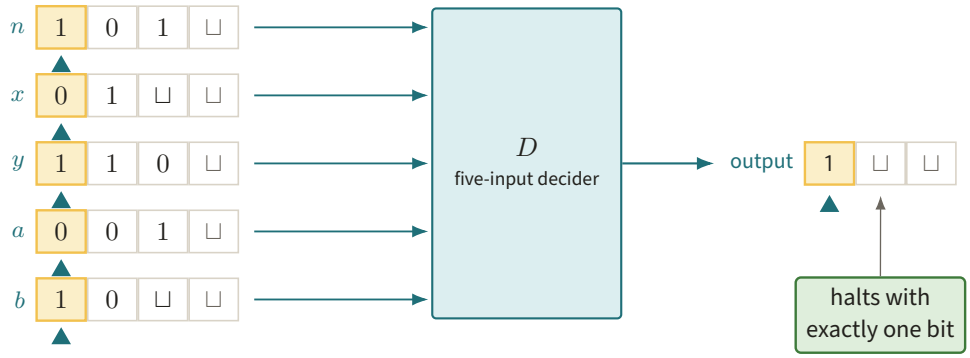


Figure 40. Five independent input tapes feed n, x, y, a, b to the decider, whose output tape must contain exactly one halting bit. Bounding the worst run at fixed n therefore also bounds how far any head can inspect the other four tapes.

The paper also assumes that a high-level machine which invokes a fixed submachine has description length linear in the embedded submachine code. Hardwiring a string z into one input of M is an effective source transformation $s(M, z)$; its output description has length $O(|M| + |z| + 1)$, is produced in time $O(|M| + |z| + 1)$, and incurs polynomial simulation overhead. A timeout wrapper counts down in binary: before the one-tape simulation it uses $O(T \log T)$ time and $O(|M|)$ description length. These are conventions, not exact equalities between particular machine encodings.

Definition 7.2 (Decider)**.** A decider is a five-input machine D which halts with one bit on every (n, x, y, a, b), where n is an integer in binary and $x, y, a, b \in \{0, 1\}^*$. The first input is the *index*, and

$$\mathrm{TIME}_D(n) = \max_{x,y,a,b} \mathrm{TIME}_{D,(n,x,y,a,b)} .$$

The maximum is understood in $\mathbb{N} \cup \{\infty\}$; a finite bound means that the machine cannot spend time reading arbitrarily remote symbols of the four unrestricted input tapes. Output 1 is acceptance and 0 is rejection.

Figure 40 makes the universal quantifier in $\mathrm{TIME}_D(n)$ concrete as five independent read-only tapes.

This is exactly `def:decider`. [2]

Definition 7.3 (Conditionally linear sampler)**.** A sampler S is a six-input Turing machine. On index n its legal modes are

$$(n, \mathsf{dimension}), \qquad (n, w, \mathsf{marginal}, j, z),$$
$$(n, w, \mathsf{linear}, j, u, y), \quad (n, w, \mathsf{factor}, j, u),$$

where $w \in \{\mathrm{A}, \mathrm{B}\}$. Its outputs must describe the ambient dimension, the j-th marginal, the conditional linear map, and the coordinate indicator of its factor space for a pair of level-ℓ CL functions over $\mathbb{F}_{q(n)}^{s(n)}$. The paper writes $\mathrm{TIME}_S(n)$ for the number of steps before S halts on index n, i.e. the worst legal call at that index. Inputs and outputs which denote integers, finite-field elements, vectors, and modes are represented by fixed binary conventions.

This transcribes `def:sampler`; its four interfaces and the fact that no call uses more than six tapes are explicit in the source. [3]

Figure 41 separates the four legal modes by the mathematical type of the answer they request.

[2] Ground truth: `def:decider` in `ground-truth/gt-05-games-normalform.tex`.
[3] Ground truth: `def:sampler` in `ground-truth/gt-04-cl.tex`.

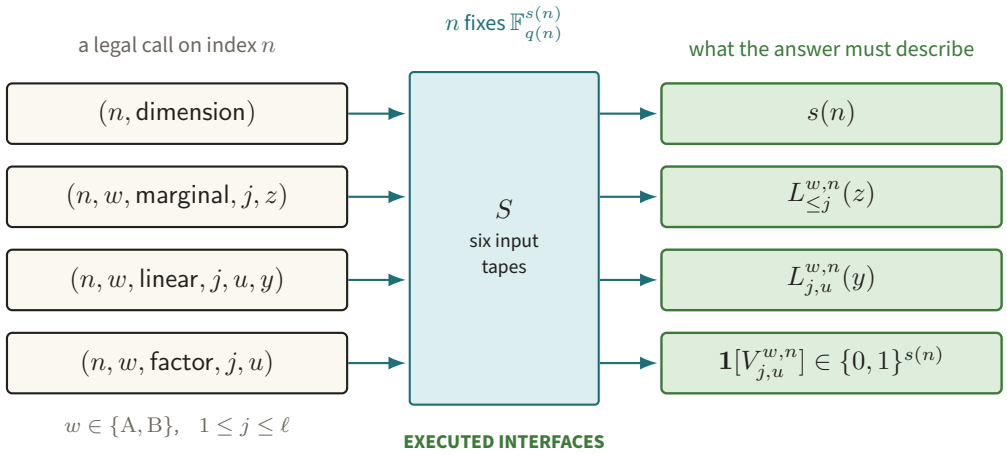


Figure 41. Four calls enter one six-input sampler and return the dimension, a marginal value, a conditional linear value, or a factor-space indicator (after `def:sampler`). Making every interface executable is what lets introspection consume the sampler's description.

Definition 7.4 (Normal form and λ-boundedness)**.** A normal-form verifier is $V = (S, D)$, where S is a CL sampler with field size $q(n) = 2$ and D is a decider. Its description length and level are

$$|V| = \max\{|S|, |D|\}, \qquad \mathrm{level}(V) = \mathrm{level}(S).$$

For an integer $\lambda \ge 1$, V is λ-bounded exactly when

$$|V| \le \lambda, \qquad \mathrm{TIME}_S(n) \le n^\lambda, \qquad \mathrm{TIME}_D(n) \le n^\lambda \quad (n \ge 2).$$

No bound is required at $n = 1$.

Here $\mathrm{level}(S)$ is the nesting depth of the sampler as a conditionally linear function; the inductive definition is Definition 13.1 in Section 13, and until then only the integer matters. These are `def:normal-ver` and `def:lambda`. [4] In the game V_n, question and answer alphabets are padded to lengths $\mathrm{TIME}_S(n)$ and $\mathrm{TIME}_D(n)$, respectively. The maximum-over-all-inputs definition above is therefore stronger than a promise only about strings of those lengths.

Figure 42 displays the three simultaneous inequalities and the exceptional index (n=1).

Four further symbols travel with λ through the whole of Part II and are collected once, in Figure 43, rather than introduced where they are first used. They are the constants Compress is parameterized by; the toy values at which the project instantiates them are in Figure 105.[5]

[4] Ground truth: `def:normal-ver`, `def:lambda` in `ground-truth/gt-05-games-normalform.tex`.

[5] Ground truth: `fig:compress`, `eq:mu-gamma`, `eq:c_rep` in `ground-truth/gt-12-compression.tex`.

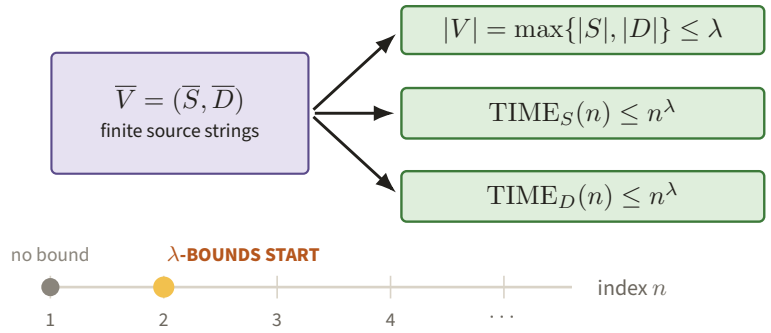


Figure 42. A single bound λ caps both source descriptions and both worst-case running times, with the time ruler beginning at $n = 2$. This common scale is what can later be rescaled uniformly to $A\lambda$.

THE UNIVERSAL CONSTANTS OF Compress

symbol	what it fixes	source
λ	the resource bound: $\lvert V\rvert \le \lambda$ and $\mathrm{TIME}_S(n), \mathrm{TIME}_D(n) \le n^\lambda$ for $n \ge 2$; also Compress's second argument	`def:lambda`
9	the level requested of ComputeIntroVerifier, and the level of the verifier Compress returns	`fig:compress`
$\mu = \lceil C_{\mathrm{intro}} \rceil$	answer-reduction time exponent passed to ComputeAnsVerifier	`eq:mu-gamma`
$\gamma = \lceil 2a_1/(b_1 b_2) \rceil$	answer-reduction soundness exponent passed with μ	`eq:mu-gamma`
τ	least integer with $\tau \ge C_{\mathrm{ar}}$ and $(\lambda n)^\tau \ge c_3^{-1}\varepsilon_2^{-17}\ln(8/\varepsilon_2)$ for all $n \ge \tau$ and all integers $\lambda \ge 1$	`eq:c_rep`
$k(n) = (\lambda n)^{(1+c_3')\tau}$	the number of repeated copies Repeat runs	`gt-12:355`
$\varepsilon_1, \varepsilon_2$	the two soundness parameters substituted into τ	`eq:re-eps-1`, `eq:re-eps-2`
C_0	index threshold: completeness and soundness are asserted only for $n \ge C_0$	`thm:compression`

The other seven are universal constants of the source, fixed before any verifier is supplied; λ alone is supplied per call. None is given a numerical value here. τ always means this repetition constant — the transition-window function of Lemma 11.1 is written ω — and the formula-occurrence bound is written $\deg_F$, as in `claims/CLAIMS.md`.

Figure 43. The eight constants Compress carries, each with the ground-truth label that fixes it, so that μ, γ, τ and $k(n)$ are defined before they are used in Sections 10 and 14. Reading them together also shows that only λ is supplied per call; the rest are decided once and for all.

7.2 What a verifier description means

Reminder. A description is syntax as data, not an oracle for behavior. The distinction was illustrated by the code of the equality machine in Section 1; it is now used in anger because compilers scan, copy, and hardwire verifier code.

A "description of a verifier" is the ordered pair $(\overline{S}, \overline{D})$ of canonical machine strings. It is neither an oracle for the functions computed by S, D nor an equivalence class of programs. An algorithm receiving it may count its bits, copy it, reject it as malformed, put it on the input tape of a universal simulator, and manufacture a new string with parts of it hardwired. Two extensionally equal deciders can have different $|D|$, running times, and behavior under those source transformations.

The compression theorem makes this interface literal. There is a polynomial-time machine Compress whose input is $(\overline{V}, \lambda)$, with $\overline{V} = (\overline{S}, \overline{D})$, and whose output is the description of a nine-level verifier. It calls, in

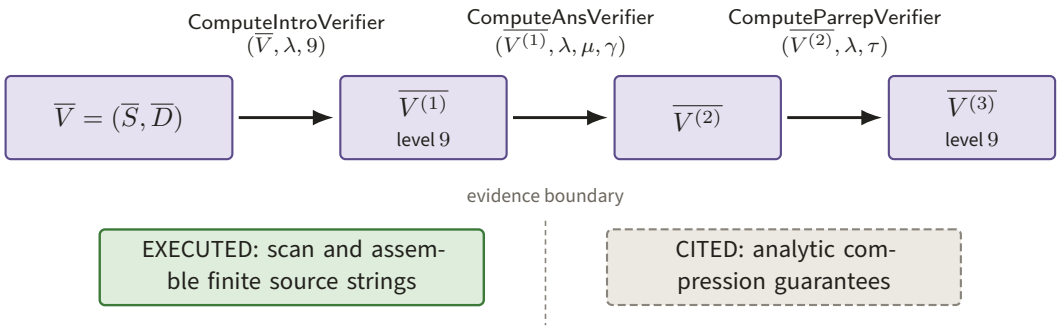


Figure 44. The compression program successively constructs $\overline{V^{(1)}}, \overline{V^{(2)}}, \overline{V^{(3)}}$ by introspection, answer reduction, and repetition, with finite string assembly in green and analytic guarantees behind the dashed boundary. This separation matters because the compiler executes on descriptions even where soundness is supplied by cited theorems.

order,

$$\overline{V^{(1)}} = \mathsf{ComputeIntroVerifier}(\overline{V}, \lambda, 9),$$
$$\overline{V^{(2)}} = \mathsf{ComputeAnsVerifier}(\overline{V^{(1)}}, \lambda, \mu, \gamma),$$
$$\overline{V^{(3)}} = \mathsf{ComputeParrepVerifier}(\overline{V^{(2)}}, \lambda, \tau).$$

Each call returns another pair of machine descriptions. [6]

Figure 44 follows the three literal description-to-description calls and marks their evidence boundary. Likewise, `thm:ar` states that ComputeAnsVerifier "outputs the description of the answer reduced verifier," and its proof separately builds the sampler and decider descriptions. [7]

7.3 Universal and source-level obligations

Reminder. Universality and s-m-n, reviewed in Section 2, justify calling a supplied program and fixing some of its arguments. Every use below must additionally pay for the universal simulation and for printing the specialized description.

The following list is the bookkeeping that a homoiconic description language must discharge. Calling the syntax "lambda calculus" removes none of these obligations.

Figure 45 collects the source-level actions that the description language must support and cost.

1. **Simulation and timeouts.** Every high-level instruction which calls S, D, or a generated verifier uses a universal machine, with the polynomial overhead and timeout counter from `sec:tms`. Hardwiring must produce finite code with the asserted length.

2. **Introspection.** The introspection decider contains the description of the original sampler and invokes that sampler to obtain its dimension, marginals, linear maps, and factor spaces. The source constructs a constant-size raw universal decider and then hardwires $(\overline{V}, \lambda, \ell)$; it first scans $|V|$ and replaces an overlong description by a trivial verifier. Thus introspection consumes the sampler description, not merely its induced distribution. [8]

[6] Ground truth: `fig:compress`, `thm:compression` in `ground-truth/gt-12-compression.tex`.
[7] Ground truth: `thm:ar` in `ground-truth/gt-10-answer-reduction.tex`.
[8] Ground truth: `lem:intro-decider-complexity`, `thm:introspection` in `ground-truth/gt-08-introspection.tex`.

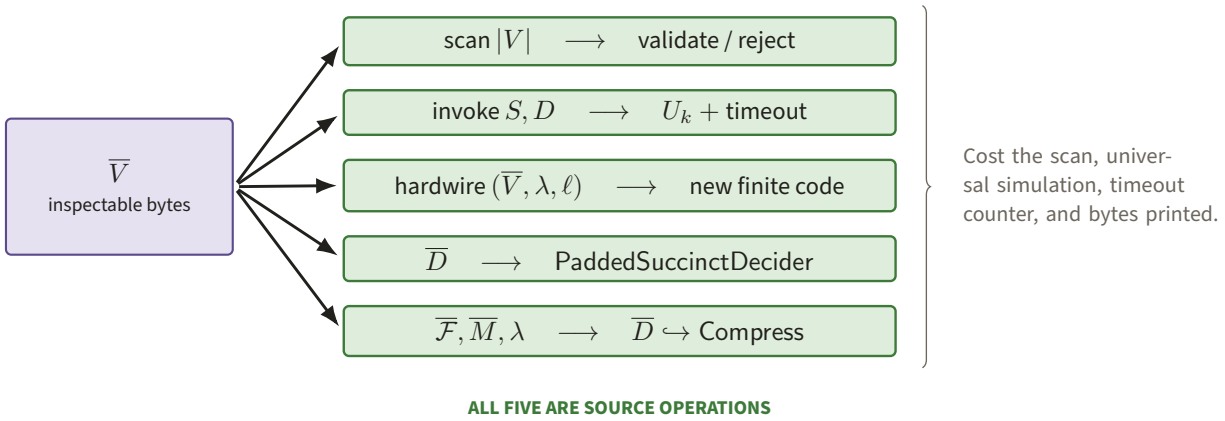


Figure 45. Every green branch starts from the actual bytes of $\overline{V}$: scan, universal invocation, hardwiring, Cook–Levin compilation, and the self-description fed to Compress. The bridge is intensional only if all five source operations remain finite and resource-accounted.

3. **Answer reduction.** In Step 1 of `fig:pcpverifier`, which the answer-reduced decider invokes from its game-check step, the verifier computes

$$\text{Circuit} = \text{PaddedSuccinctDecider}(\overline{D}, n, T, Q, \sigma, x, y)$$

from the *description* of D. The Cook–Levin circuit depends on the transition table even though it is much smaller than the computation tableau. The construction uses $|D| \leq \sigma$ quantitatively.[9] Figure 46 separates the two machines that this one step belongs to.

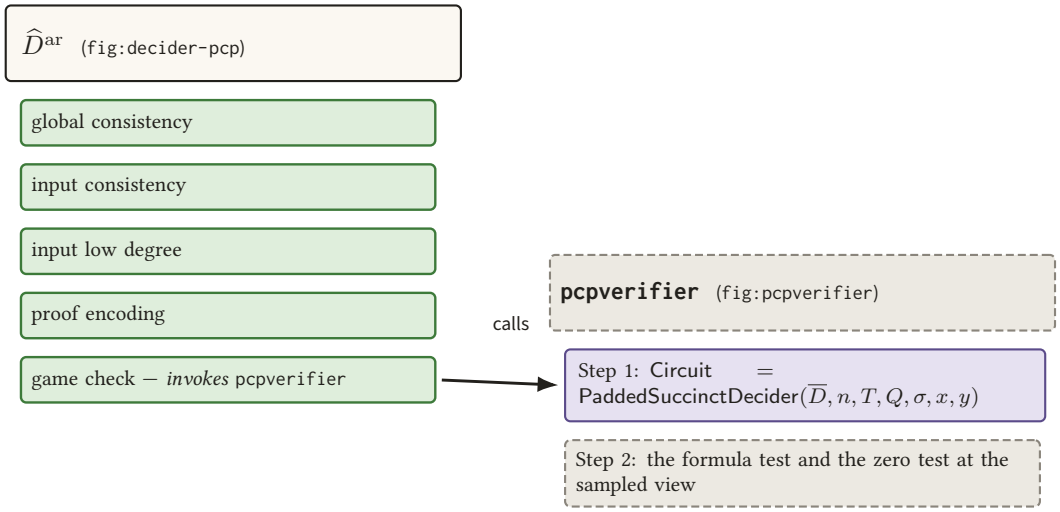


Figure 46. The answer-reduced decider owns the five guards of `fig:decider-pcp`; only its last guard calls pcpverifier, and Step 1 — the succinct-circuit construction from $\overline{D}$ — lives inside that call. Keeping the two apart is what makes the description, rather than the tableau, the object the compiler handles.

4. **Compiler composition.** The three machines displayed in the preceding subsection traverse and assemble source descriptions. The proof that Compress is polynomial time uses their running times.

[9] Ground truth: `fig:pcpverifier`, `prop:standard-succinct-sat` in `ground-truth/gt-10-answer-reduction.tex`.

The independence of the compressed sampler is also a statement about which source strings are embedded in its output.

5. **Self-reference.** In the Halting section, the eight-input machine $\mathcal{F}$ is run as

$$\mathcal{F}(\overline{\mathcal{F}}, \overline{M}, \lambda, n, x, y, a, b).$$

It constructs the five-input program obtained by hardwiring its first three arguments, pairs that decider description with a computed sampler description, and passes the pair to Compress. This is a source-level diagonalization, not recursive invocation alone. [10]

Sections 8–10 construct one description language which meets precisely these obligations. The point is not Church–Turing equivalence; the point is a costed equivalence between finite, inspectable descriptions.

8 A lambda calculus with resources

Figure 47 sorts every named result of Part II into three provenances, and the reminder below states the consequence for the middle column.

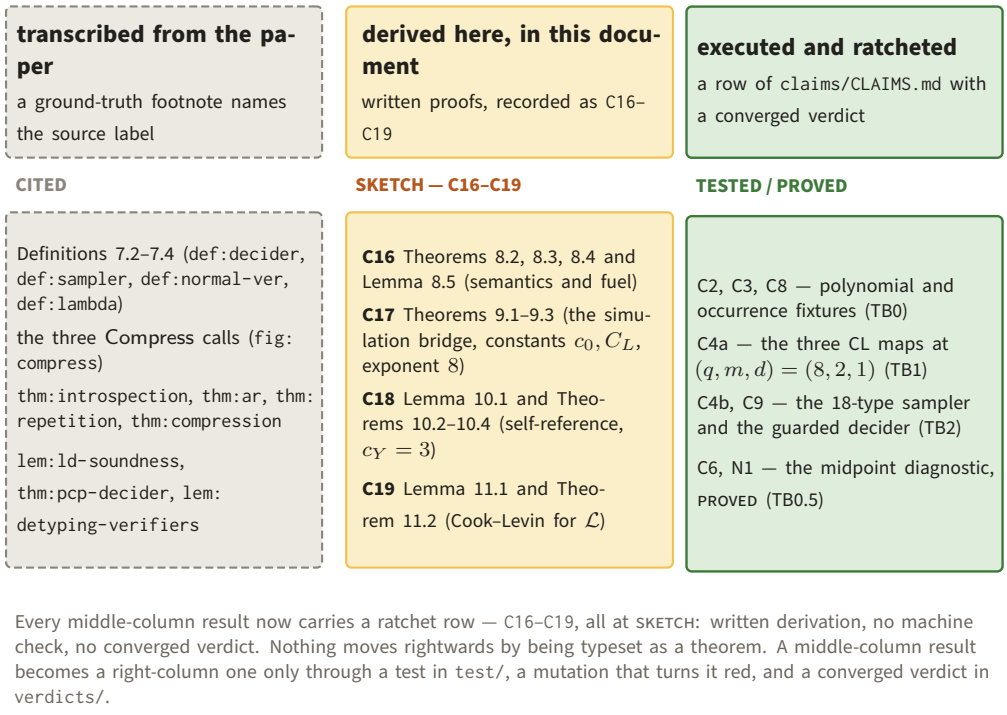


Figure 47. Every named result of Part II belongs to exactly one of three columns: transcribed from the paper, derived here and carried at SKETCH by C16–C19, or executed and ratcheted by a converged verdict. The middle column is the largest, which is the honest shape of a document that costs a construction rather than proving its soundness.

[10] Ground truth: fig:halt_f in ground-truth/gt-12-compression.tex.

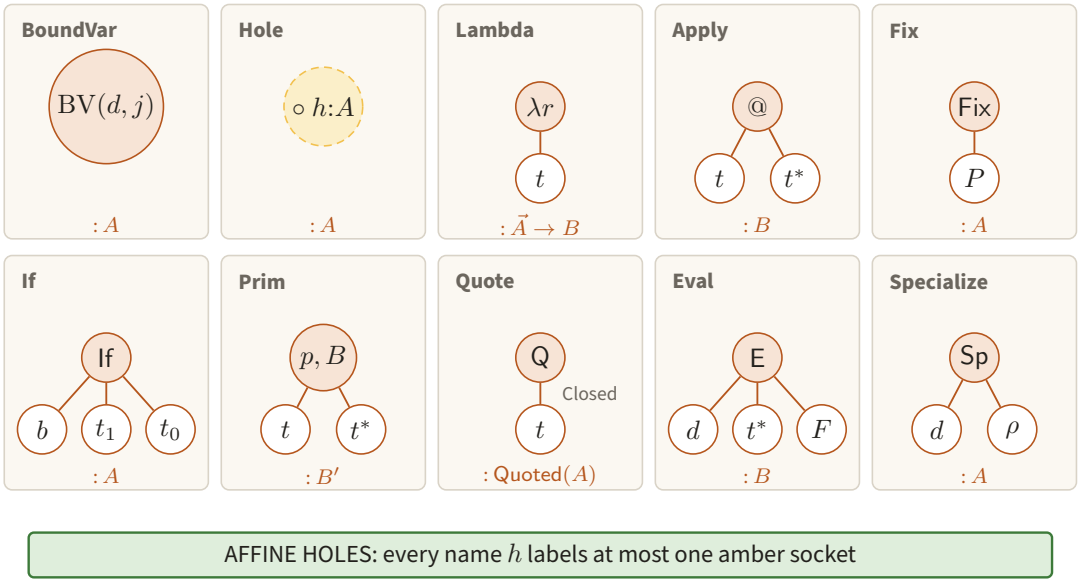


Figure 48. The ten constructors of $\mathcal{L}$ have distinct tree shapes and sorts, while a Hole is an amber typed socket rather than a variable. The affine-hole discipline makes later source substitution a one-site tree operation.

Reminder. Status of everything typeset as a theorem from here to Section 14. Sections 8–11 state results about $\mathcal{L}$ — the language defined here — and not about the paper. They are this project's own derivations: Theorems 8.2, 8.3, 8.4, 9.1, 9.2, 9.3, 10.2, 10.3, 10.4 and 11.2, and Lemmas 8.5, 10.1 and 11.1, are written proofs which have *not* been adversarially verified. They are recorded as `C16`–`C19` in `claims/CLAIMS.md` at SKETCH — NOT YET ADVERSARIALLY VERIFIED: a written derivation, no machine check, no converged verdict. The four rows partition exactly this list — `C16` the semantics and fuel results of Section 8 (Theorems 8.2–8.4, Lemma 8.5), `C17` the costed machine/term bridge of Section 9 (Theorems 9.1–9.3), `C18` the self-reference results of Section 10 (Lemma 10.1, Theorems 10.2–10.4), and `C19` the Cook–Levin construction of Section 11 (Lemma 11.1, Theorem 11.2). The constants they name $(a_0, b_0, c_0, a_U, b_U, C_L, c_Y = 3, a_s, b_s, a_K, b_K)$ and exponents $(8,\ 3 + \lceil \log_2 72c_0 \rceil)$ are fixed by those proofs alone. Results transcribed from the paper are marked instead by a ground-truth footnote and are CITED; the two kinds are never mixed.

8.1 Syntax, phases, and canonical bytes

Reminder. The bare calculus of Section 4 had variables, abstraction, application, beta reduction, and quotation. The language $\mathcal{L}$ below is a typed, call-by-value engineering IR: de Bruijn addresses make binding canonical, and explicit fuel makes evaluation cost a first-class quantity.

Fix a finite set of ground sorts including Bit, Nat, Bytes, Tuple, MachineDesc, and Quoted(A). Function sorts are finite products followed by a result sort. The raw terms of the description language $\mathcal{L}$ are

$$\begin{aligned} t ::= {} & \mathrm{BoundVar}(d, j) \mid \mathrm{Hole}(h, A) \mid \mathrm{Lambda}(r, t) \mid \mathrm{Apply}(t, t^*) \mid \mathrm{Fix}(t) \\ & \mid \mathrm{If}(t, t, t) \mid \mathrm{Prim}(p, B, t^*) \mid \mathrm{Quote}(\mathrm{Closed}(t)) \\ & \mid \mathrm{Eval}(t_{\mathrm{code}}, t^*, F) \mid \mathrm{Specialize}(t_{\mathrm{code}}, \rho). \end{aligned} \tag{8.1}$$

Figure 48 turns the ten grammar alternatives into typed tree shapes and marks the affine static socket. Here t^* is a finite ordered list. $\mathrm{BoundVar}(d, j)$ addresses slot j of the environment frame d binders outward; hence alpha-conversion is absent from the representation. A hole is a named, typed, static substitution site,

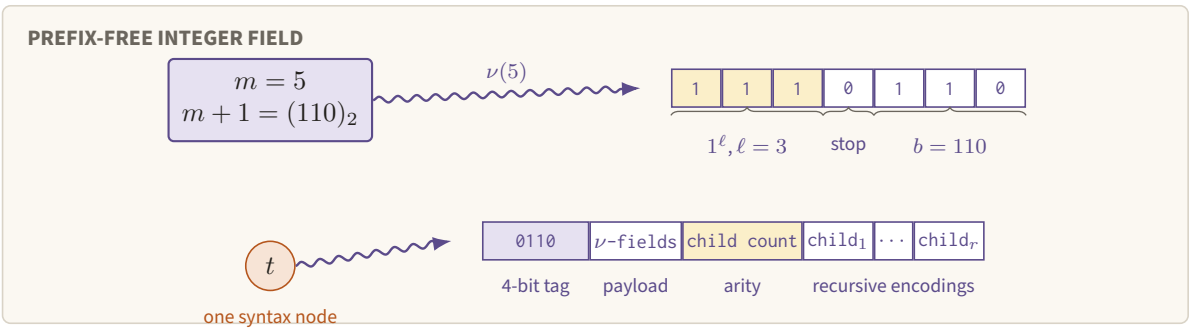


Figure 49. For $m = 5$, the unary length 1110 announces the three payload bits 110, and every term node then lays out its tag, fields, arity, and recursive children. Prefix-free fields plus arities make the canonical byte stream uniquely parseable without subtree lengths.

not a dynamic variable. We impose the *affine-hole discipline*: each hole name occurs at most once. Reuse of static data is expressed by binding it once and using de Bruijn variables. F is either $\mathrm{FuelLiteral}(m)$ or a closed arithmetic term $\mathrm{FuelBound}(t_n, t_\lambda)$.

Each primitive name p has a total type contract, a deterministic reference machine, and a natural-valued charge $\kappa_p(v_1, \ldots, v_r)$. The serialized annotation B proves $\kappa_p(v) \le B(|v_1|, \ldots, |v_r|)$. The registry contains the usual finite byte, bit, tuple, list, comparison, and arithmetic operations, as well as the bounded machine-table operation used in Theorem 9.1. A primitive is not an oracle: its reference machine runs in at most $c_p(1 + \kappa_p + \sum_i |v_i|)^{e_p}$ Turing steps, with c_p, e_p fixed in the language definition.

The sorting judgment $\Gamma; H \vdash t : A$ is the ordinary simply typed call-by-value judgment for variables, abstraction, application, and conditionals, extended as follows. H records affine holes; Quote requires a scoped closed term with empty H; $\mathrm{Specialize}$ requires an environment covering exactly H with terms of the declared sorts; Eval requires quoted code, arguments matching the quoted function sort, and natural fuel. $\mathrm{Fix}(P)$ requires a partial body $P : A$ with the single hole $\mathsf{self\_code} : \mathsf{Quoted}(A)$, and closes that hole. We write $P\{A\}$ for sorted syntax, $\mathrm{Closed}(P)$ for empty variable and hole contexts, and $\mathrm{Quoted}\{A\}$ for its canonical bytes. Raw ill-sorted syntax is retained so that the evaluator can return $\mathsf{TypeError}$ rather than throw a host exception.

Definition 8.1 (Canonical serialization)**.** Let $\nu(m) = 1^\ell 0 b$, where b is the ℓ-bit binary expansion of $m + 1$ and $\ell = \lfloor \log_2(m + 1) \rfloor + 1$. Thus $|\nu(m)| = 2\ell + 1$, and ν is prefix-free. Encode a byte string z as $\nu(|z|)z$. Encode a term by a four-bit constructor tag followed, in the order displayed in (8.1), by ν-encoded integer and string fields, the encoded child count, and the recursive child encodings. Sorts, primitive contracts, fuel expressions, and static environments use the same rule with fixed tags and lexicographically ordered field names. No node contains a redundant subtree length.

The resulting map ser is injective and self-delimiting. For a term or code value t, define $|t| := |\,\mathrm{ser}(t)|$ in bits. This is the description length used below.

The two nested delimiters in Figure 49 make the unique prefix parse visible at both the integer-field and constructor levels.

Prefix-freeness of ν determines the end of every field; arities determine the number of recursive parses. Induction on the first constructor tag therefore gives a unique parse. In particular, description size is a number computed from syntax, not the size of a runtime closure.

The operative representations are:

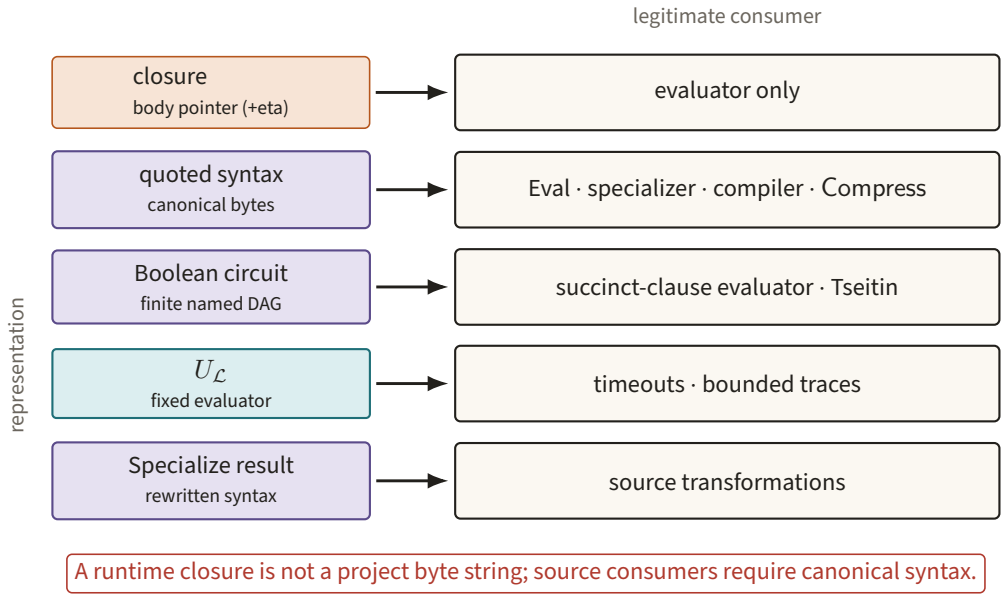


Figure 50. Five representations travel to different legitimate consumers, with canonical quoted syntax occupying the source-code path. Keeping closures off that path prevents runtime sharing from being mistaken for finite description bytes.

Object	Contents	Legitimate consumer
Closure	Body pointer and runtime environment	Evaluator only; it has no canonical project byte string
Quoted syntax	Canonical bytes of closed $\mathcal{L}$ syntax	Evaluator, specializer, compiler, and Compress
Circuit	Finite Boolean DAG with named input blocks	Succinct clause evaluator and Tseitin transformation
Universal evaluator	One fixed program interpreting quoted syntax	Timeout and bounded-trace construction
Specialization	Capture-free code-to-code replacement	Source transformations; its result is syntax, not a closure

Figure 50 routes each representation only to consumers that are allowed to inspect it.

8.2 Certificates and the derivation tree

Every evidence grade printed in Section 14 is a node of a concrete data structure, not a marginal note. The single source is `docs/DESIGN.md` §3; the implementation is `src/certificates.jl`. A transformation does not return a bare term; it returns

$$\texttt{Checked}\{T, C\} \;=\; (\,\texttt{term} :: T,\ \texttt{certificate} :: C\,),$$

where C is a `CertNode`: a grade, a rule name, a finite record of facts, an ordered tuple of children, and — for one grade only — a replay function. The grades are exactly five, and Figure 51 lists them with what each obliges its author to supply.

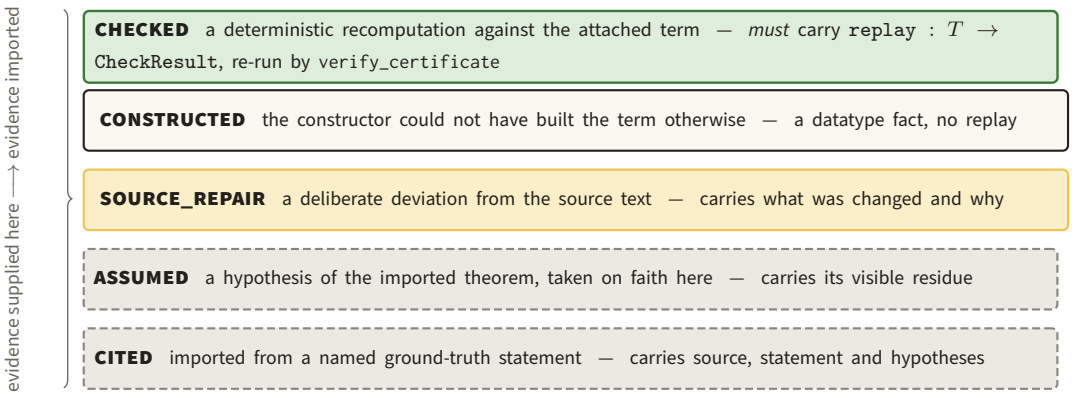


Figure 51. The five grades a `CertNode` may carry, in decreasing order of what they oblige the author to supply; only CHECKED carries executable evidence, and only it is re-run at test time. Reading them as a scale makes an all-CITED subtree visibly worthless as evidence, however green the code around it looks.

`verify_certificate` walks the tree and re-runs every CHECKED replay against the term actually attached; a CHECKED node with no replay function is a verification failure, not a pass. That is what makes stale evidence unrepresentable: there is no detached cache to go out of date, and a mutation that changes the term but not the certificate turns the suite red. Symbolic bounds are either $\mathrm{Concrete}(k)$ or $\mathrm{Opaque}(\text{description}, \text{parameters})$; no $\mathrm{poly}(\cdot)$ is silently given an exponent.

Figure 52 is the tree that `AnswerReduce` actually builds on the TB2 fixture, exactly as `test/tb2_answer_reduce.jl` inspects it. Reading it top down answers the question the grade columns of Section 14 raise: the *root* of the executed answer-reduction certificate is CITED, because detyping is imported; the executed content sits strictly below it.

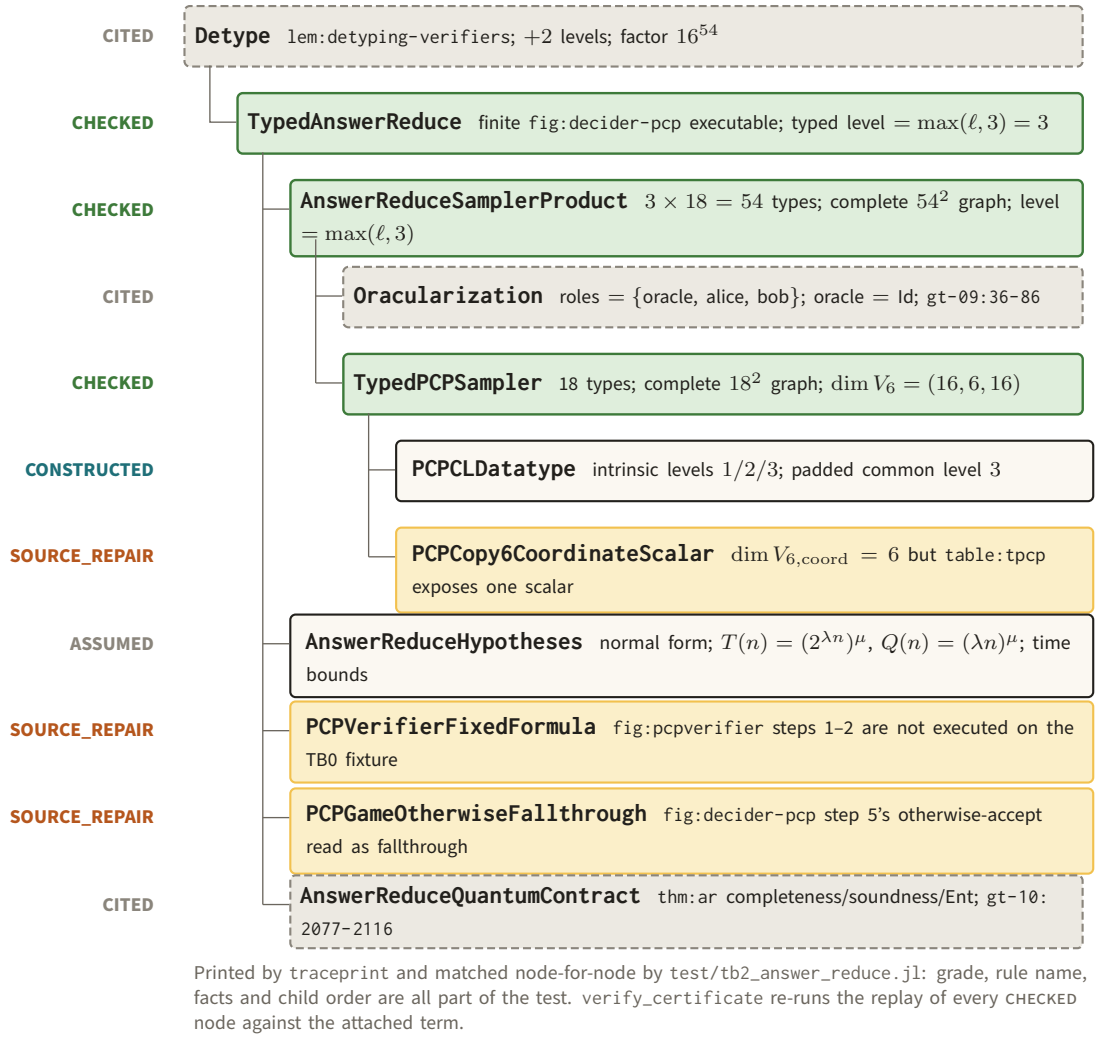


Figure 52. The certificate that AnswerReduce returns on the TB2 fixture, with each node's grade, rule name and printed facts. A CITED root over CHECKED children is the normal shape here, and it is why an executed subtree never promotes the theorem above it.

The TB5–TB7 design adds four *predicate outcomes* on top of these grades, for the parameter guards a toy instantiation cannot satisfy: PASS, FAIL, NOT_EVALUABLE (the cost or fit was never measured) and VACUOUS (the guard set is empty, or a promised margin is $\geq 1/2$). They are outcomes printed beside a predicate, not grades on a node, and none of them may be folded into PASS; the TB7 report of Figure 105 is where they appear.

8.3 Small-step semantics and fuel

Reminder. Normal form alone does not specify the cost of reaching it. Recall the term $KI\Omega$: normal order terminates and call-by-value does not. We now fix one deterministic call-by-value transition system, so a fuel count denotes a particular run rather than an arbitrary beta-reduction path.

We use a deterministic call-by-value CEK machine. A runtime closure is $[\lambda r.t, \eta]$, where $\eta = (\eta_0, \eta_1, \ldots)$ is

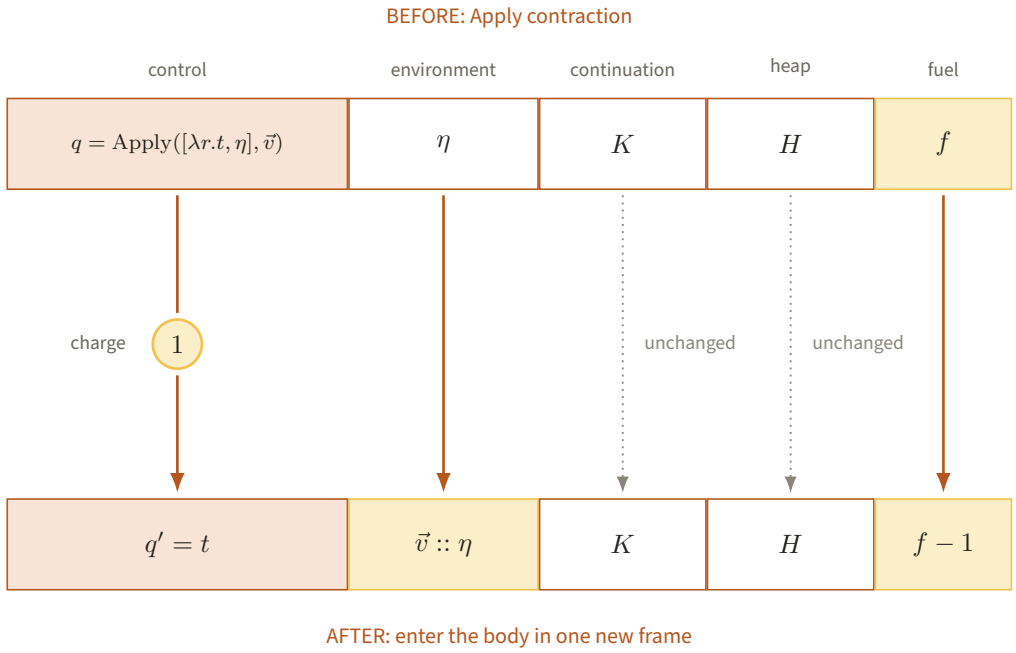


Figure 53. One Apply contraction replaces control by the body, pushes $\vec{v}$ onto the environment, and changes fuel from f to $f - 1$, leaving K and H untouched. Exposing all five registers makes the resource charge part of the operational state rather than an external annotation.

a stack of frames and $\eta_d[j]$ supplies $\mathrm{BoundVar}(d, j)$. Other values are canonical ground data and $\mathrm{Code}(t)$ for closed syntax. A configuration is

$$C = \langle q, \eta, K, H, f \rangle,$$

where q is the current term or value, K is a continuation stack, H is an append-only heap for lists, tuples, closures, and syntax pointers, and $f \in \mathbb{N}$ is remaining fuel. Arguments are evaluated left to right.

Figure 53 expands one charged Apply transition across all five registers of the CEK configuration.

For reference, the contractions after all indicated subterms have become values are the following. The notation $C \longrightarrow_k C'$ means that the transition has charge k. Every row is a full five-register configuration $\langle q, \eta, K, H, f \rangle$; in the application row η' is the ambient environment at the call site, which is discarded in favour of the closure's own η extended by the argument frame.

$$\begin{aligned}
\langle \mathrm{BoundVar}(d, j), \eta, K, H, f \rangle &\longrightarrow_1 \langle \eta_d[j], \eta, K, H, f - 1 \rangle && \text{if the address exists,} \\
\langle \mathrm{Lambda}(r, t), \eta, K, H, f \rangle &\longrightarrow_1 \langle [\lambda r.t, \eta], \eta, K, H, f - 1 \rangle, \\
\langle \mathrm{Apply}([\lambda r.t, \eta], \vec{v}), \eta', K, H, f \rangle &\longrightarrow_1 \langle t, (\vec{v}) :: \eta, K, H, f - 1 \rangle && (|\vec{v}| = r), \\
\langle \mathrm{If}(1, v_1, v_0), \eta, K, H, f \rangle &\longrightarrow_1 \langle v_1, \eta, K, H, f - 1 \rangle, \\
\langle \mathrm{If}(0, v_1, v_0), \eta, K, H, f \rangle &\longrightarrow_1 \langle v_0, \eta, K, H, f - 1 \rangle, \\
\langle \mathrm{Prim}(p, B, \vec{v}), \eta, K, H, f \rangle &\longrightarrow_{\kappa_p(\vec{v})} \langle p(\vec{v}), \eta, K, H, f - \kappa_p(\vec{v}) \rangle, \\
\langle \mathrm{Quote}(\mathrm{Closed}(t)), \eta, K, H, f \rangle &\longrightarrow_1 \langle \mathrm{Code}(t), \eta, K, H, f - 1 \rangle.
\end{aligned} \tag{8.2}$$

An absent variable address, failed primitive contract, nonbit condition, wrong application arity, or noncode argument to Eval/Specialize has the sole successor TypeError at charge one. Evaluation contexts supply the omitted rules: for example, Apply first pushes a frame remembering its unevaluated arguments, evaluates the operator, then evaluates each argument from left to right. Thus the display specifies contractions while the context discipline specifies their unique order.

All administrative CEK transitions cost one unit. They include variable lookup, creating a closure, pushing or popping one continuation frame, selecting a Boolean branch, and returning a value. Beta reduction of $\mathrm{Apply}([\lambda r.t, \eta], v_1, \ldots, v_r)$ costs one and continues with t in environment $((v_1, \ldots, v_r), \eta_0, \eta_1, \ldots)$. A wrong arity, applying a nonclosure, or using a nonbit condition takes one transition to TypeError. A residual Hole does the same. $\mathrm{Quote}(\mathrm{Closed}(t))$ takes one transition to the syntax pointer $\mathrm{Code}(t)$; serialization was performed when the enclosing description was admitted, so this transition does not copy $|t|$ bits.

After its arguments have become values, $\mathrm{Prim}(p, B, v_1, \ldots, v_r)$ costs exactly $\kappa_p(v_1, \ldots, v_r) \geq 1$. If the contract or dynamic sorts fail it uses one unit and returns TypeError. For purposes of the local semantics, a charge k is expanded into k deterministic microsteps of the primitive reference machine; thus every microstep consumes one fuel unit.

$\mathrm{Specialize}(\mathrm{Code}(P), \rho)$ first checks the sort and coverage of ρ, then traverses P in prefix order. It uses $1 + |P| + \sum_h |\rho(h)|$ fuel and returns the code value of the substituted term, or TypeError. Eval evaluates its code, arguments, and fuel expression, installs a delimiter holding the requested inner budget, and runs the same CEK transition relation inside it. Installing and removing that delimiter costs two units. Each inner microstep decrements both the inner budget and the ambient budget. Exhausting either produces OutOfFuel.

The paired counters in Figure 54 expose how an inner Eval budget remains subordinate to the ambient run.

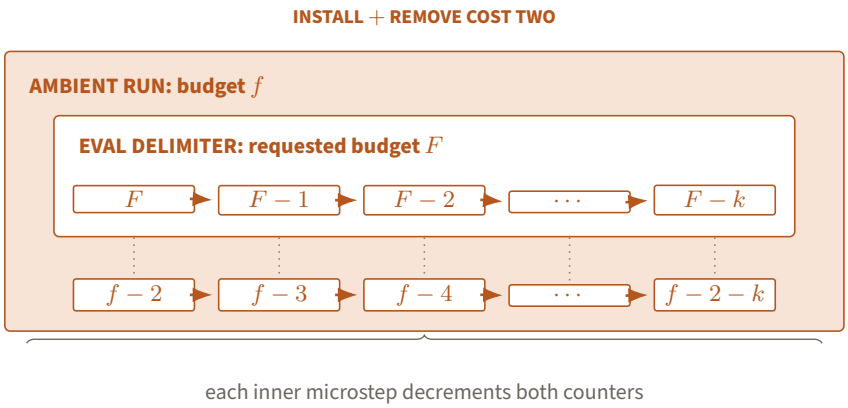


Figure 54. The Eval delimiter nests an F-unit budget inside ambient fuel f, charging two units for installation and removal and one from both counters per inner microstep. Exhausting either ledger therefore ends the same finite outer computation.

Finally, $\mathrm{Fix}(P)$ is a value of the result sort of P. When it is applied to arguments u, one unfolding transition installs the static binding

$$\mathsf{self\_code} \longmapsto \mathrm{Code}(\mathrm{Fix}(P))$$

and evaluates P with the same arguments. This is a constant-size heap link, not a serialization of a recursively expanded tree. Including the application and binding frames, an unfolding costs the fixed constant $c_Y = 3$. The corresponding fully specialized syntax can be materialized by Specialize when a source consumer asks for it.

If a transition has charge k and $f < k$, its unique successor is OutOfFuel; otherwise the successor has counter $f - k$. A final configuration is exactly one of

$$\mathsf{Value}(v), \qquad \mathsf{OutOfFuel}, \qquad \mathsf{TypeError}.$$

We write $\mathrm{run}(t; f)$ for this result. For closed function syntax t and an argument tuple u, $\mathrm{eval}_{\mathcal{L}}(t, u; f)$ abbreviates the run of $\mathrm{Apply}(t, \overline{u})$, where $\overline{u}$ is the literal value syntax. Results do not record unused fuel.

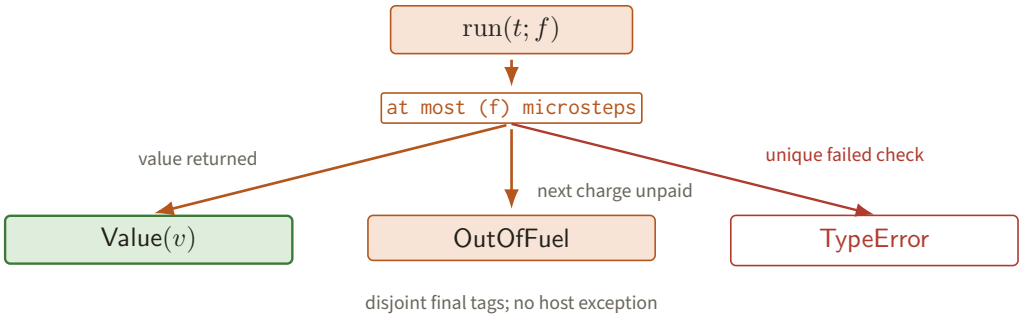


Figure 55. A deterministic run reaches exactly one disjoint leaf: a value, exhausted fuel, or a typed error. Totalizing malformed programs in this tree is what makes every bounded evaluation suitable for a finite trace encoding.

Figure 55 records the three and only three leaves of every finite-fuel execution.

For the paper, the next theorem buys a unique bounded-evaluation trace for each quoted verifier call, which can later be encoded by Cook–Levin.

Theorem 8.2 (Determinism and outcome trichotomy)**.** *Every nonfinal configuration with positive fuel has at most one successor. For every closed raw term and finite initial fuel, evaluation terminates in exactly one of* Value(v), OutOfFuel, *or* TypeError.

Proof. The current control item is either a term, a value, or a primitive microstate. Constructor tags are disjoint. For a term tag, the evaluation-context rule selects the leftmost child not yet represented by a value; if there is one, exactly one continuation frame is pushed. If all children are values, exactly one contraction above applies. Variable addresses select at most one frame and slot. Primitive reference machines and their contracts are deterministic by definition. The Boolean and arity tests have disjoint success and error cases. The same reasoning applies to the unique top Eval delimiter and to the single self-code binding of Fix. Hence two distinct successors cannot exist.

Every nonfinal transition consumes at least one unit. Starting from fuel f, at most f transitions or primitive microsteps can occur before a value or error is returned; if neither has occurred, the next requested step returns out-of-fuel. The three final tags are disjoint, and determinism makes the reached tag unique. □

For the paper, the next theorem buys safe enlargement of a proved timeout: extra fuel cannot change a completed verifier answer.

Theorem 8.3 (Fuel monotonicity)**.** *If* $\mathrm{run}(t; f) = R$ *with* $R \neq$ OutOfFuel, *then* $\mathrm{run}(t; f + g) = R$ *for every* $g \geq 0$. *More precisely, if the first run uses* $c \leq f$ *units, the second uses the same* c *transitions and leaves* $g + (f - c)$ *units.*

Figure 56 plots the invariant fuel gap along the common deterministic trajectory.

Proof. Induct on the c transitions of the terminating run. At step i, the larger-fuel run has exactly g more ambient fuel and, inside each Eval delimiter, exactly the same inner budget as the smaller run. The smaller run can pay the uniquely determined charge k_i, so the larger run can pay it as well. Determinism gives the same nonfuel successor data, and subtraction preserves the difference g. After c steps the larger run therefore reaches the same final tag and value. No operational rule branches on the numeric fuel except the insufficient-fuel rule, which was not selected in the smaller run. □

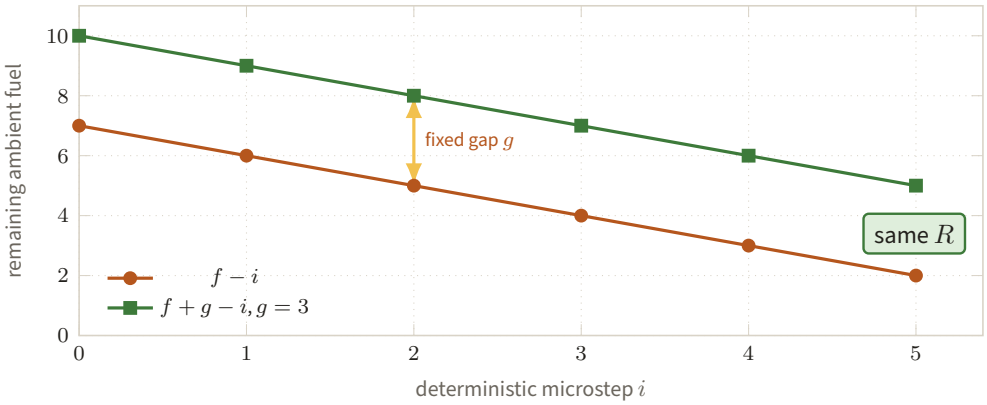


Figure 56. Two runs follow the same charged transitions while their remaining-fuel curves stay exactly g units apart. Once the smaller budget reaches a non-fuel result R, extra fuel cannot change that result.

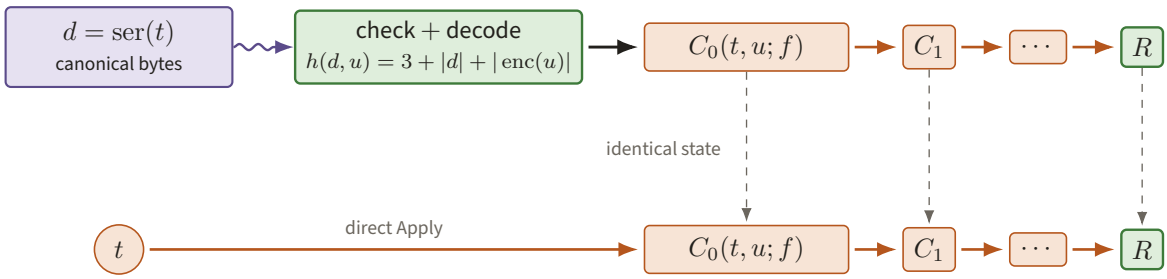


Figure 57. Quoted execution spends $h(d,u) = 3 + |d| + |\,\mathrm{enc}(u)|$ to decode canonical bytes and then lands on the direct run's exact initial CEK state. From that meeting point determinism forces both trajectories to return the same result.

8.4 Quotation, evaluation, and specialization

Reminder. Quotation turns a finite syntax tree into data; specialization is the lambda counterpart of s-m-n. Neither operation decides what the quoted program computes. It only parses or rewrites its canonical tree.

External evaluation of bytes must pay for decoding even though an internal Quote is a syntax pointer. Define

$$h(d,u) = 3 + |d| + |\,\mathrm{enc}(u)|.$$

The quoted evaluator $\mathrm{Eval}_{\mathcal{L}}(d,u;F)$ checks and decodes d, installs the argument tuple, and then runs the CEK machine. By definition the front-end scan uses exactly $h(d,u)$ fuel; malformed code returns TypeError during that scan. The constant 3 is not a spare allowance: it pays for installing the Eval delimiter, removing it, and the one tag check that decides whether d is well-formed code. It therefore *absorbs* the two delimiter units charged above, and the two accounts do not double-count.

Figure 57 aligns the byte-decoding run with direct execution at the first identical CEK configuration.

For the paper, the next theorem buys the right to replace execution of a term by execution of its transmitted description while charging for decoding.

Theorem 8.4 (Quote/Eval equation with explicit overhead). *Let t be a closed function term, let $d = \mathrm{ser}(t)$, and let u have the required argument sorts. For every $f \geq 0$,*

$$\mathrm{Eval}_{\mathcal{L}}(\mathrm{Quote}(t),u;f + h(d,u)) = \mathrm{eval}_{\mathcal{L}}(t,u;f). \tag{8.3}$$

Here the left side takes the canonical bytes denoted by $\mathrm{Quote}(t)$*. If the right side uses* $c \le f$ *units and returns a value or type error, the left side uses exactly* $h(d,u)+c$*. If it requests a* $(f+1)$*-st unit, both sides return out-of-fuel after their respective budgets are exhausted.*

Proof. The prefix decoder is inverse to serialization by Definition 8.1; hence after its fixed scan the left machine holds the same syntax tree t, literal arguments, empty lexical environment, and application continuation as the right machine. The left counter is then exactly f. From this point the two configurations are identical. Theorem 8.2 gives identical successors until a final outcome. The front-end accounts for the stated additional $h(d,u)$ units and no other rule differs. □

Lemma 8.5 (Capture-free specialization and its size)**.** *Let* P *be well-scoped partial syntax satisfying the affine-hole discipline, and let* σ *map every hole of* P *to closed, sort-correct syntax. There is a unique term* $\mathrm{Specialize}(P,\sigma)$*, it has no holes, it preserves the sort of* P*, and*

$$|\,\mathrm{Specialize}(P,\sigma)| \le |P| + \sum_{h\in\mathrm{dom}\,\sigma} |\sigma(h)|. \tag{8.4}$$

The construction takes at most $1+|P|+\sum_h |\sigma(h)|$ *CEK microsteps.*

Figure 58 depicts the unique prefix-order splice at an affine hole and the exact one-hole size identity.

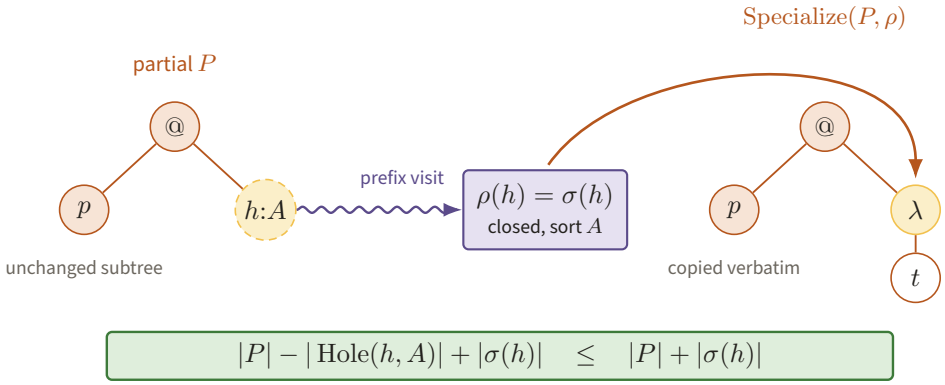


Figure 58. Prefix traversal replaces the unique amber $\mathrm{Hole}(h,A)$ by the closed tree $\sigma(h)$ and copies every other node unchanged. Affineness turns the serialization cost into one deletion plus one insertion, yielding the stated linear size bound.

Proof. Traverse P in prefix order. Copy every nonhole tag and payload unchanged. At $\mathrm{Hole}(h,A)$, check the unique table entry, check its sort, and emit $\sigma(h)$. The inserted term is closed, so its de Bruijn indices refer to no binder in P; indices in the surrounding syntax are also unchanged. Thus no variable can be captured. Induction on P proves sort preservation, because the hole and replacement have the same sort at the only new case. Coverage removes every hole, and deterministic traversal gives uniqueness.

Serialization has no ancestor subtree-length fields. Replacing the one occurrence of h deletes a nonempty encoded Hole node and inserts exactly $|\sigma(h)|$ bits. Since distinct affine holes have disjoint occurrences,

$$|\,\mathrm{Specialize}(P,\sigma)| = |P| - \sum_h |\,\mathrm{Hole}(h,A_h)| + \sum_h |\sigma(h)|,$$

which implies (8.4). One scan step per input or emitted bit, plus the initial check, gives the time bound. Without the affine-hole discipline the correct last term would sum once per hole occurrence; this is why the discipline is part of the formal system. □

9 Simulation theorems: the machine–term bridge

Reminder. Section 5 gave the two compiler ideas and warned that Church–Turing equivalence alone says nothing quantitative. This section supplies the description-length and running-time inequalities needed by the compression theorem.

9.1 From the paper's machines to $\mathcal{L}$

We use explicit functional tapes and one bounded table primitive. A tape is a zipper (L, s, R): L is the reversed list strictly left of the head, $s \in \{0, 1, \sqcup\}$ is the scanned symbol, and R is the list strictly right of it with implicit blank tail. A right move maps (L, s, rR') to (sL, r, R'), taking $r = \sqcup$ when R is empty; a left move is symmetric. These are fixed list primitives of charge at most four. A configuration is a tuple of the finite control state and the $k + 2$ zippers for input, work, and output tapes.

Figure 59 zooms into the highlighted matching row of the bounded table scan the next paragraph charges for.

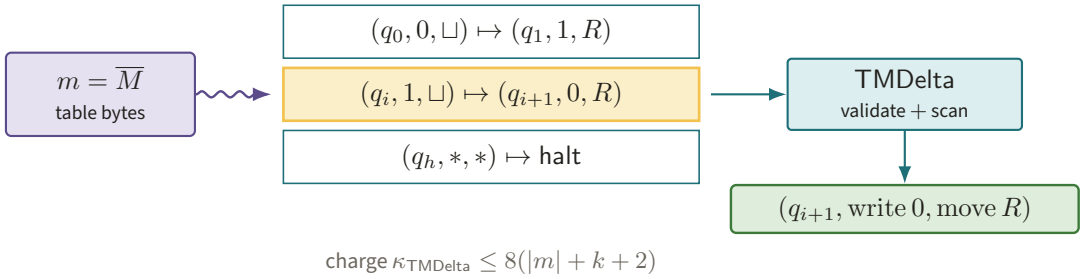


Figure 59. The TMDelta primitive validates and scans the bytes of m until the current state-symbol tuple selects one row, then returns the next state, write symbols, and motions. Keeping the table as data explains both the single-copy description size and charge $8(|m| + k + 2)$.

For a machine description m, the primitive $\mathsf{TMDelta}(m, q, s_1, \ldots, s_{k+2})$ scans the encoded transition table and returns the next state, written symbols, and head motions. Its declared charge is

$$\kappa_{\mathsf{TMDelta}} \le 8(|m| + k + 2).$$

The primitive validates m; malformed descriptions use a fixed rejecting transition. Its implementation is an ordinary finite table scan. Thus the description of the table remains data of length $|m|$, rather than being duplicated into a nest of conditionals.

Let init_k parse the k-tuple encoding into input zippers and blank work/output zippers, and let out read the longest nonblank output prefix. Define

$$\begin{aligned} \mathsf{loop}_m &= \mathrm{Fix}\big(\lambda C.\ \mathrm{If}(\mathsf{halt}(C), \mathsf{out}(C), \\ &\qquad\qquad \mathsf{self}(\mathsf{move}(C, \mathsf{TMDelta}(m, \mathsf{view}(C))))\big)\big), \\ \langle\!\langle M \rangle\!\rangle &= \lambda z.\mathsf{loop}_{\overline{M}}(\mathsf{init}_k(z)). \end{aligned} \tag{9.1}$$

Here self is the closure obtained from the Fix self-code link; equivalently it is Eval of that link with the next configuration. All other displayed operations are fixed primitives with the stated list representation.

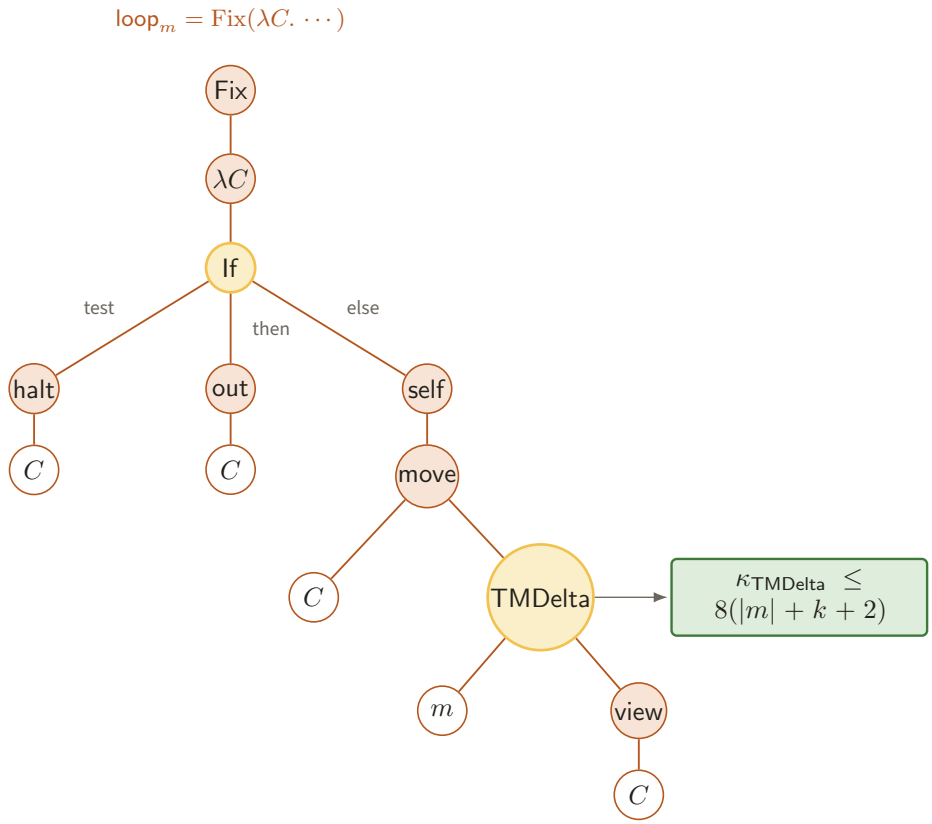


Figure 60. The tree for loop$_m$ tests the current configuration, emits its output on halt, and otherwise feeds move(C, TMDelta(m, view(C))) through the self link. The amber table node isolates the only m-dependent per-iteration charge, bounded by $8(|m| + k + 2)$.

Figure 60 exposes every constructor on the recursive branch of loop$_m$, including the charged table lookup.

For the paper, the next theorem buys a lambda description for every original sampler or decider without changing its answers or hiding a size blow-up.

Theorem 9.1 (TM to $\mathcal{L}$, with resources)**.** *There are universal constants $a_0, b_0, c_0 \ge 1$ such that for every k-input machine M,* (9.1) *is a closed term and*

$$|\langle\langle M \rangle\rangle| \le a_0|M| + b_0.$$

If $M(x_1, \ldots, x_k)$ halts within $T \ge 1$ transitions, then

$$\mathrm{Eval}_{\mathcal{L}}(\mathrm{Quote}(\langle\langle M \rangle\rangle), \langle x_1, \ldots, x_k \rangle; c_M T(T + |x|)) = M(x_1, \ldots, x_k), \tag{9.2}$$

where $|x| = \sum_i |x_i|$ and one may take $c_M = c_0(|M| + k + 1)$. The tuple on the left is the same dual-rail encoding as in Definition 7.1. In particular, a decider receives exactly

$$\langle \mathrm{bin}(n), x, y, a, b \rangle_{\mathcal{L}}, \qquad \mathrm{enc}_{\mathcal{L}}\big(\langle \mathrm{bin}(n), x, y, a, b \rangle_{\mathcal{L}}\big) = \mathrm{enc}_5(\mathrm{bin}(n), x, y, a, b),$$

and the five components initialize five separate input zippers.

Proof. The term contains one copy of $\overline{M}$, the fixed interpreter body, and self-delimiting constructor overhead. Each input bit in $\overline{M}$ contributes a fixed number of serialized bits, so constants a_0, b_0 independent of M give the size bound.

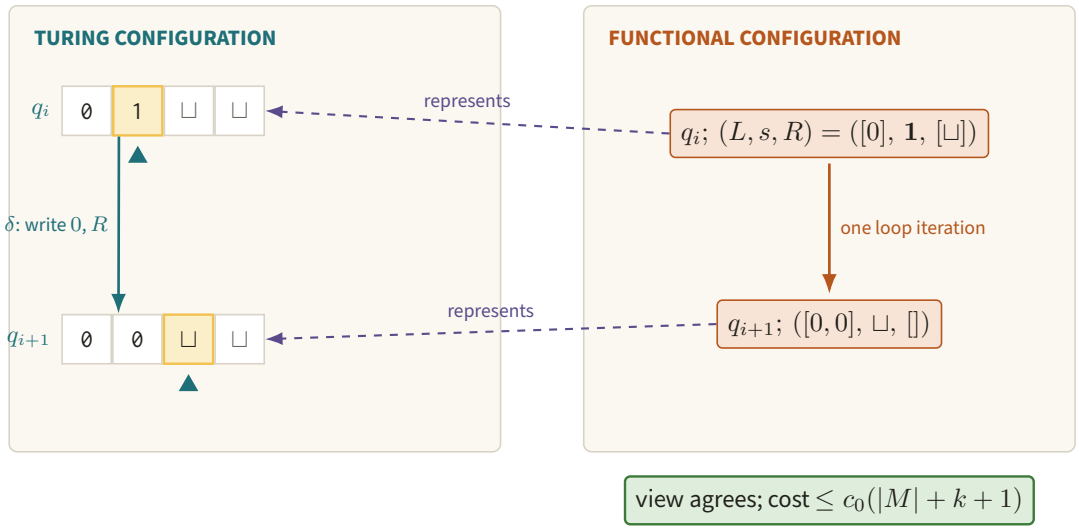


Figure 61. A Turing tape that scans 1 and writes 0 while moving right matches the zipper update from $([0], 1, [\sqcup])$ to $([0, 0], \sqcup, [])$. This commuting rung is the induction step behind semantic preservation and the $c_0(|M| + k + 1)$ iteration charge.

Initialization scans the dual-rail input once. For valid input it consumes at most $c_0(|x| + k + 1)$ units and produces exactly the paper's initial tapes. Assume inductively that the functional configuration before loop iteration i represents the machine configuration after i transitions. The views return the same scanned symbols. The table primitive selects the unique transition of M, and each zipper update writes the selected symbol and moves in the selected direction. Therefore the next functional configuration represents the machine configuration after $i + 1$ transitions. This proves the simulation invariant for $0 \leq i \leq T$.

The commuting rung in Figure 61 draws one concrete write-and-move step on both representations.

One loop iteration, including the fixed-point unfolding, table scan, tuple operations, and $k + 2$ zipper updates, uses at most $c_0(|M| + k + 1)$ units. No visited zipper has more than $|x| + T$ explicit cells. The final output scan therefore uses at most $c_0(|x| + T + 1)$ units. Adding initialization, T iterations, output, and the quote/decode overhead from Theorem 8.4 gives at most

$$c_0(|M| + k + 1)\big(|x| + 1 + T\big) + c_0(|M| + k + 1)T.$$

For $T \geq 1$, both $|x| + T + 1$ and T are at most $2T(T + |x|)$. Increasing the universal choice of c_0 by a factor four makes this no greater than the fuel in (9.2). The invariant and the definition of out then give the same output string. The explicit quadratic form is deliberately conservative; the zipper simulation itself is linear in T apart from the hardwired table scan. □

9.2 From $\mathcal{L}$ to the paper's machines

For the paper, the next theorem buys a route back from the IR to the exact Turing-machine model in which verifier bounds and compression are stated. Figure 62 lays the records the proof builds onto the universal evaluator's literal work tape and follows its lazy handle to a separate input tape.

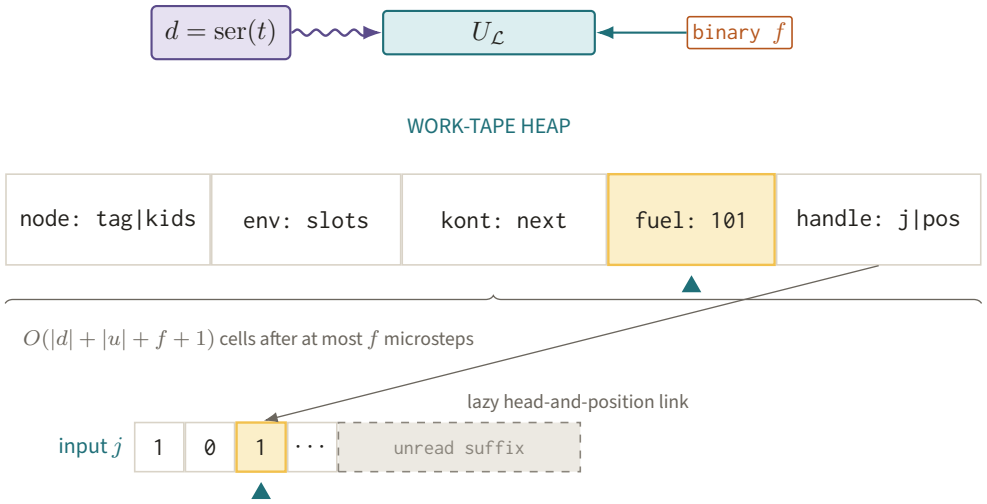


Figure 62. The work tape of $U_{\mathcal{L}}$ stores syntax nodes, environments, continuations, binary fuel, and lazy input handles as finite records, with the active fuel record highlighted. Because a handle reaches only the visited prefix, an f-unit run is independent of every dashed unread suffix.

Theorem 9.2 (A universal evaluator for $\mathcal{L}$). *There is a fixed three-input Turing machine $U_{\mathcal{L}}$, of description length $b_U = O(1)$, and a universal constant $C_L \geq 1$, such that on input (d, u, f) it returns the same value, out-of-fuel, or type-error result as $\mathrm{Eval}_{\mathcal{L}}(d, u; f)$, and*

$$\mathrm{TIME}_{U_{\mathcal{L}}}(d, u, f) \leq C_L\big(|d| + |u| + f + 1\big)^8. \tag{9.3}$$

Given closed t, hardwiring $d = \mathrm{ser}(t)$ produces a machine $(U_{\mathcal{L}})_t$ with

$$|(U_{\mathcal{L}})_t| \leq a_U|t| + b_U.$$

On the paper's separate input tapes it may represent each argument by a lazy head-and-position handle. Consequently, a run with fuel f accesses at most f symbols of those tapes and has the refined bound $C_L(|t| + f + 1)^8$, independent of unread suffixes. Hence a total decider term with fuel schedule $f_D(n)$ is a decider in the paper's sense and

$$\mathrm{TIME}_D(n) \leq C_L(|t| + f_D(n) + 1)^8. \tag{9.4}$$

Proof. The machine first checks the prefix serialization and writes node records on a work-tape heap. It stores a node address, environment-frame addresses, continuation records, and binary fuel counters. During at most f microsteps the heap acquires at most a fixed number of records per step, so its length is at most $c(|d| + |u| + f + 1)$ for a universal c. Write $N = |d| + |u| + f + 1$. A conservative *multitape* implementation finds a record by a full scan of the heap track, compares binary addresses cell by cell, and returns to the simulated head, all in at most $(cN)^2$ steps.

Figure 63 is the whole cost ladder of this proof in one column, with the point where the exponent doubles marked.

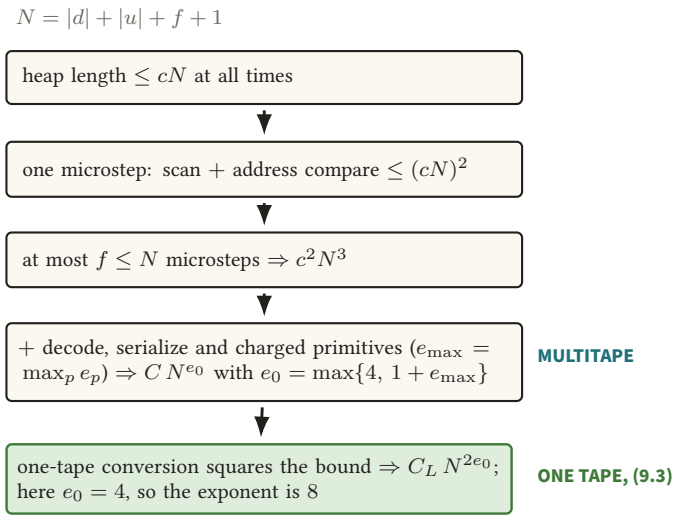


Figure 63. The five steps that produce (9.3), ending at the one-tape conversion that squares a multitape bound. Making the doubling explicit is what keeps the statement and the proof of Theorem 9.2 in agreement.

There are at most $f \leq N$ microsteps, so the simulation itself costs at most c^2N^3; decoding the input serialization and writing the final answer each take a constant number of passes over a track of length at most cN, which is absorbed. Charged primitives run on their fixed reference machines in at most $c_p(1+\kappa_p+\sum_i |v_i|)^{e_p}$ steps, and $\kappa_p \leq f \leq N$ with $\sum_i |v_i| \leq cN$, so each such step costs at most $c_p(c'N)^{e_p}$; since the registry is fixed when $\mathcal{L}$ is fixed, $e_{\max} = \max_p e_p$ is a constant. Taking $e_0 = \max\{4,\ 1 + e_{\max}\}$ and increasing the constant, the whole *multitape* run is bounded by $C\,N^{e_0}$.

Converting that fixed multitape machine to the paper's one-tape convention costs the standard quadratic overhead, so the one-tape running time is at most $C_L\,N^{2e_0}$. With the registry fixed here, $e_0 = 4$ and the stated exponent is 8; this proves (9.3). The exponent is not optimized and nothing downstream depends on its value: Theorem 9.3 and the A-rescaling use only the existence of *some* fixed e and C_L with $\mathrm{TIME}_{U_{\mathcal{L}}}(d, u, f) \leq C_L(|d| + |u| + f + 1)^e$. A different primitive registry replaces 8 by $2\max\{4, 1 + e_{\max}\}$ throughout.

The simulator description contains only the parser, CEK transition table, primitive registry, and serializers, so its length is b_U, independent of d, u, f. The paper's hardwiring convention embeds d once and gives the linear description bound. For separate input tapes, a handle stores a tape number and head position. One microstep can move such a head by at most the work performed during that charged step, so no more than f input symbols can be inspected. Removing the untouched encoded suffixes from the preceding storage account yields the refined bound. Taking a maximum over all (x, y, a, b) now proves (9.4); arbitrarily long unread suffixes do not affect it. □

9.3 The resource dictionary

Reminder. The qualitative dictionary in Section 5 matched machine time with reduction fuel and machine code with serialized syntax. The table below is the quantitative version: it records the polynomials rather than merely asserting that simulations exist.

Figure 64 states the two principal rows as a bidirectional quantitative bridge, including both linear description bounds.

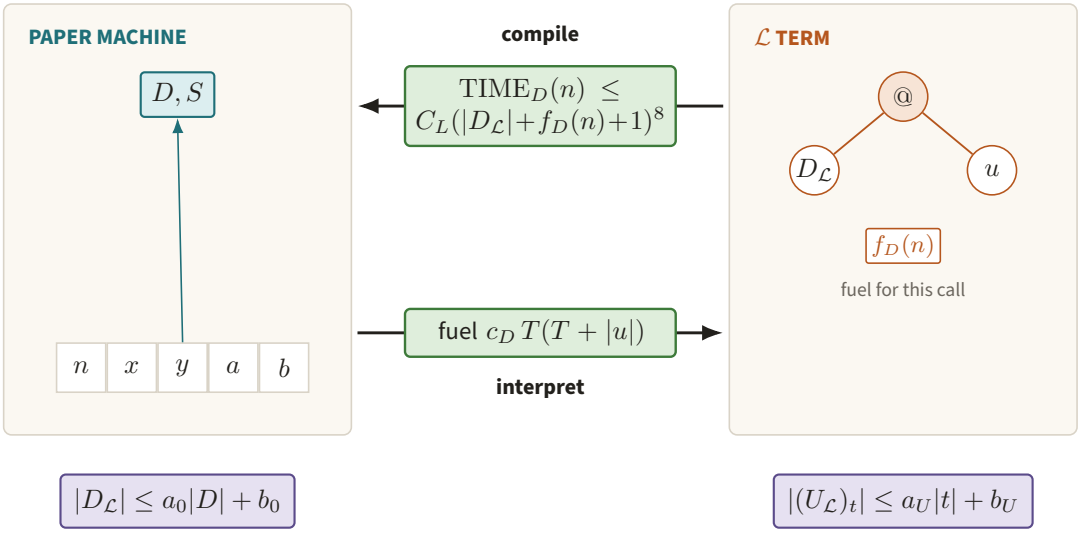


Figure 64. Compiling a term costs a fixed polynomial in fuel and description size, interpreting a machine costs a fixed polynomial in time and input, and the description bounds are linear in both directions. Naming the polynomials, rather than asserting that simulations exist, is what lets λ-boundedness survive the change of language.

For a term program, let $s_t = |t|$ and let $f_t(n)$ be a proved fuel schedule which suffices for every legal call at index n. The translations above give the following dictionary.

Paper quantity	$\mathcal{L}$ counterpart	Quantitative relation
$\mathrm{TIME}_D(n)$	decider fuel $f_D(n)$	Term to TM: $\mathrm{TIME}_D(n) \le C_L(\|D_{\mathcal{L}}\| + f_D(n) + 1)^8$. TM to term: fuel $c_D T(T + \|u\|)$.
$\mathrm{TIME}_S(n)$	maximum sampler-mode fuel $f_S(n)$	Same two bounds, taking the maximum over the dimension, marginal, linear, and factor calls.
$\|D\|$	$\|\mathrm{ser}(D_{\mathcal{L}})\|$	Each translation is linear: at most $a_0\|D\| + b_0$ or $a_U\|D_{\mathcal{L}}\| + b_U$.
$\|S\|$	$\|\mathrm{ser}(S_{\mathcal{L}})\|$	The same linear bounds; static CL tables and source descriptions are counted once.
λ-bounded	size at most λ, fuel at most n^λ	Preserved in both directions as $A\lambda$-bounded for a universal integer A, as proved below.
Threshold $n \ge C_0$	same semantic index condition	Translation does not change n; resource estimates may replace it by $n \ge \max\{C_0, 2\}$, never hide it in poly.

For the paper, the next theorem buys invariance of the verifier class under a change of programming language, after one universal rescaling of λ. Figure 65 records the two exponent calculations of its proof, whose maximum determines the universal rescaling A.

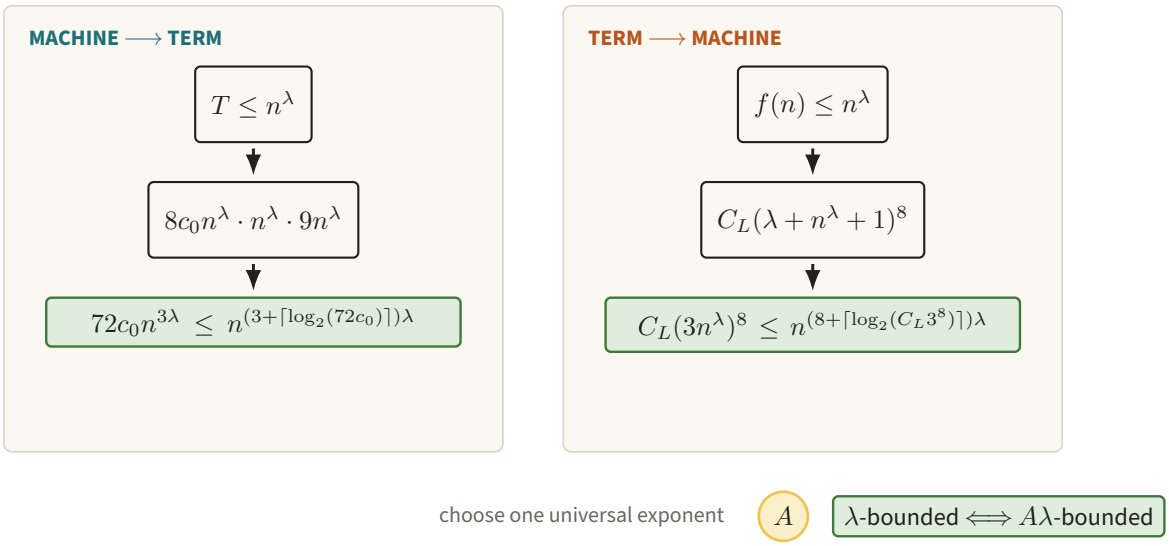


Figure 65. The machine-to-term direction contributes exponent $3 + \lceil \log_2(72c_0) \rceil$, while the term-to-machine direction contributes $8 + \lceil \log_2(C_L 3^8) \rceil$. Choosing one universal A above both calculations preserves the bounded verifier class in either language.

Theorem 9.3 (Preservation of λ-boundedness)**.** *Fix the encodings and primitive registry above. There is a universal integer $A \ge 1$ such that:*

1. *translating any λ-bounded paper verifier term-by-term with Theorem 9.1 gives an $A\lambda$-bounded $\mathcal{L}$-verifier;*
2. *compiling any λ-bounded $\mathcal{L}$-verifier with Theorem 9.2 gives an $A\lambda$-bounded paper verifier.*

The translation preserves the verifier's index and its input/output function.

Proof. For the first direction, $|M| \le \lambda$ and the size theorem gives $|\langle\langle M \rangle\rangle| \le (a_0 + b_0)\lambda$, since $\lambda \ge 1$. At index $n \ge 2$, the paper machine runs for $T \le n^\lambda$. It can inspect or emit at most T tape symbols. Across at most six input tapes the portion relevant to the computation therefore has encoded length at most $8n^\lambda$, after increasing the constant to cover $\log n$ and mode tags. Also $c_M \le c_0(\lambda + 7) \le 8c_0 n^\lambda$. Theorem 9.1 then requires at most

$$8c_0 n^\lambda \cdot n^\lambda \cdot 9n^\lambda = 72c_0 n^{3\lambda} \le n^{(3+\lceil \log_2(72c_0)\rceil)\lambda}.$$

The last inequality uses $n \ge 2$ and $\lambda \ge 1$. It applies to both the sampler and decider.

For the second direction, hardwiring gives description size at most $(a_U + b_U)\lambda$. With fuel at most n^λ, the lazy-input form of (9.4) is at most

$$C_L(\lambda + n^\lambda + 1)^8 \le C_L(3n^\lambda)^8 \le n^{(8+\lceil \log_2(C_L 3^8)\rceil)\lambda}.$$

We used $\lambda \le 2^\lambda \le n^\lambda$. The same calculation holds for every sampler mode. Taking A to be the maximum of the four displayed description and exponent coefficients proves both assertions. Semantics is preserved by Theorems 9.1 and 9.2; neither translation changes the binary index. □

The constants $a_0, b_0, c_0, a_U, b_U, C_L$, fixed arities, serialization tags, and the universal simulation exponent are absorbed when the paper writes $O(\cdot)$ or $\mathrm{poly}(\cdot)$: they are universal after the two languages are fixed. The particular $|D|, |S|$, a supplied λ, a fuel or time parameter, and hardwired $|M|$ are *arguments* of those

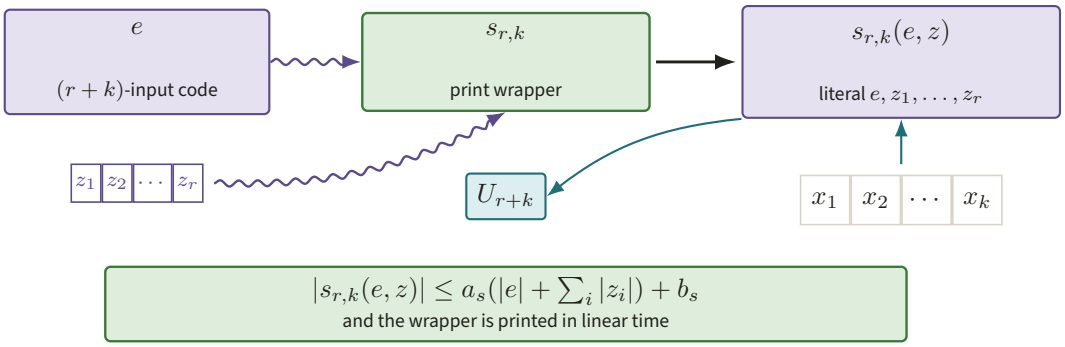


Figure 66. The transformer $s_{r,k}$ copies $e, z_1, \ldots, z_r$ into a wrapper that accepts only the live inputs $x_1, \ldots, x_k$ before invoking U_{r+k}. One copy of every static bit gives both the linear output-size bound and linear construction time.

polynomials and cannot be hidden as universal constants. Nor can the semantic threshold C_0, the change from λ to $A\lambda$, or constants appearing inside completeness and soundness inequalities be erased by poly notation.

10 Self-reference as a construction on descriptions

Reminder. Kleene's theorem in Section 3 produced a behavioral fixed point by applying a parameter-fixing compiler to its own code. We now repeat that construction with explicit description-size bounds and identify it with the paper's diagonal verifier.

10.1 The recursion theorem with size accounting

Let $\varphi_e^{(k)}$ be the partial function of the k-input paper machine with description e. We use equality of partial functions only in the conclusion; every operation in the construction acts on strings.

Lemma 10.1 (The s-m-n operation). *For every $r, k \ge 1$ there is a total source transformer $s_{r,k}$ such that*

$$\varphi^{(k)}_{s_{r,k}(e,z_1,\ldots,z_r)}(x) = \varphi_e^{(r+k)}(z_1, \ldots, z_r, x).$$

Under the paper's hardwiring convention there are constants a_s, b_s, depending only on r, k, for which

$$|s_{r,k}(e,z)| \le a_s\big(|e| + \textstyle\sum_i |z_i|\big) + b_s,$$

and the output code is computed in time linear in the right-hand side.

Figure 66 follows the static strings into the printed wrapper while keeping the k live inputs outside the hardwiring pass.

Proof. Output the fixed wrapper which writes $\mathrm{enc}_{r+k+1}(e, z_1, \ldots, z_r, x)$ on its work tape and invokes the universal machine U_{r+k}. The wrapper contains one copy of each static bit and constant code for tuple encoding and universal simulation. It therefore has the displayed linear length and can be printed in linear time. The universal-machine theorem gives the stated partial-function equation on halting inputs; on a nonhalting input both sides simulate the same machine forever. □

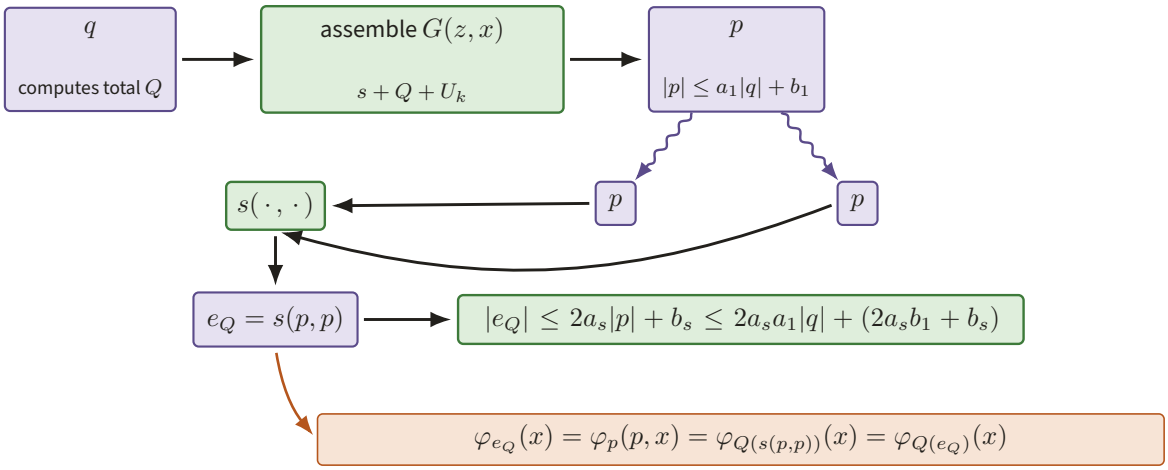


Figure 67. A program p containing q and fixed code for s, Q, U_k is duplicated only at the hardwiring node $e_Q = s(p,p)$, while the lower chain verifies the fixed-point equation. Substituting the linear bound on $|p|$ makes $|e_Q| \le a_K|q| + b_K$ explicit.

For the paper, the next theorem buys self-reference while keeping the fixed point's source length linear in the source transformer.

Theorem 10.2 (Kleene's second recursion theorem, paper model)**.** *Let Q be a total computable transformation from descriptions of k-input machines to descriptions of k-input machines. There is an effectively constructible description e_Q such that*

$$\varphi^{(k)}_{e_Q} = \varphi^{(k)}_{Q(e_Q)}. \tag{10.1}$$

If q is a description of the machine computing Q, then, for constants a_K, b_K fixed by k and the encodings,

$$|e_Q| \le a_K|q| + b_K,$$

and e_Q is constructed in time polynomial in $|q|$.

Figure 67 keeps both copies of p visible from their construction through the behavioral equality.

Proof. By Lemma 10.1 with $r = 1$, let $s(z,z)$ denote the description obtained by hardwiring the string z into the first input of the program z itself — the same operation as in Part I — for a suitable $(k+1)$-input machine. Define the partial computable function

$$G(z,x) = \varphi^{(k)}_{Q(s(z,z))}(x).$$

A machine for G first computes the finite string $s(z,z)$, runs the total code transformer Q on it, and universally evaluates the resulting k-input code on x. Let p be its description and set $e_Q = s(p,p)$. Then for every x, including the case in which the final computation diverges,

$$\varphi^{(k)}_{e_Q}(x) = \varphi^{(k+1)}_{p}(p,x) = \varphi^{(k)}_{Q(s(p,p))}(x) = \varphi^{(k)}_{Q(e_Q)}(x),$$

which proves (10.1).

The program p contains one copy of q and fixed code for s and the universal evaluator, so $|p| \le a_1|q| + b_1$. Applying the linear size bound of Lemma 10.1 to $s(p,p)$ gives $|e_Q| \le 2a_s a_1|q| + (2a_s b_1 + b_s)$. These are the asserted a_K, b_K. Printing the two wrappers and executing the total hardwiring transformations takes polynomial time under the paper's conventions. □

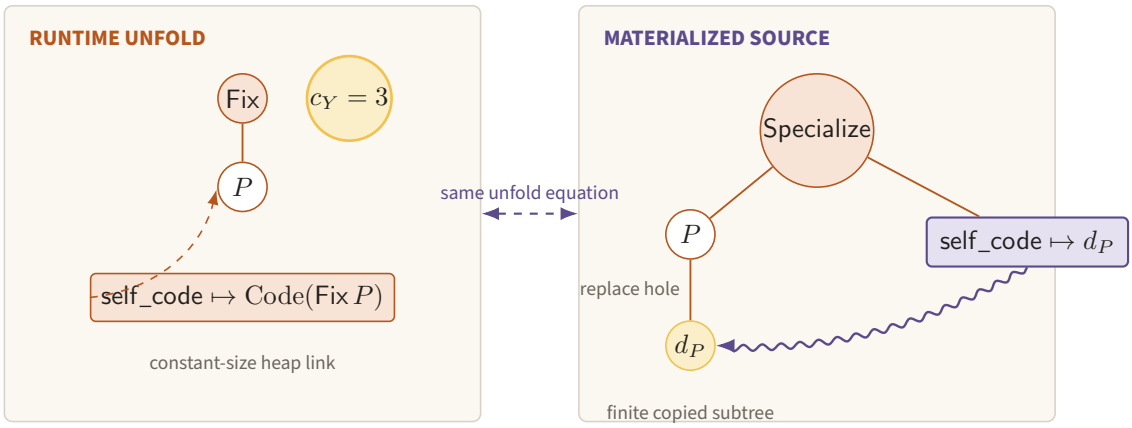


Figure 68. At runtime, $\mathrm{Fix}(P)$ installs the constant-size link self_code $\mapsto \mathrm{Code}(\mathrm{Fix}\, P)$ for exactly $c_Y = 3$ units, whereas a source consumer may materialize the specialized tree containing d_P. Separating sharing from copying prevents recursive unfolding from inflating the serialized fixed point.

10.2 The call-by-value Z combinator and YCode

Reminder. Raw Y unfolds too early under call-by-value; Section 4 derived the eta-delayed Z combinator. The constructor below realizes that delay as a typed, constant-size self-code link.

For orientation, call-by-value lambda calculus uses

$$Z = \lambda f.(\lambda x.f(\lambda u.xxu))(\lambda x.f(\lambda u.xxu)),$$

rather than the call-by-name Y, because eager evaluation of $Y\,f$ unfolds before supplying an argument. This untyped term has constant size, but it does not itself provide a canonical code value of the fixed point. Our typed description constructor does.

For a partial body P with its one distinguished quoted hole, define

$$\mathrm{YCode}(P) := \mathrm{Fix}(P), \qquad d_P := \mathrm{Quote}(\mathrm{YCode}(P)).$$

The source equation is

$$\mathrm{unfold}(\mathrm{YCode}(P)) = \mathrm{Specialize}(P, \{\mathsf{self\_code} \mapsto d_P\}). \tag{10.2}$$

It is an equation between finite descriptions. Its right side is produced on demand by Lemma 8.5; the runtime evaluator instead uses a constant-size environment link.

Figure 68 contrasts the three-unit runtime link with the larger source tree materialized only on demand.

For the paper, the next theorem buys an operational self-code equation whose unfolding cost and serialized size are both explicit.

Theorem 10.3 (Description-level fixed-point equation)**.** *For every well-sorted P, argument tuple u, and $f \geq c_Y = 3$,*

$$\mathrm{eval}_{\mathcal{L}}(\mathrm{YCode}(P), u; f) = \mathrm{eval}_{\mathcal{L}}(\mathrm{Specialize}(P, \{\mathsf{self\_code} \mapsto d_P\}), u; f - c_Y)\,. \tag{10.3}$$

Equivalently, if f_P is the code transformer represented by P, the right side is conventionally written $\mathrm{eval}_{\mathcal{L}}(f_P(\mathrm{YCode}(f_P)), u; f - c_Y)$. *Moreover*

$$|\,\mathrm{YCode}(P)| = |P| + c_{\mathrm{fix}}$$

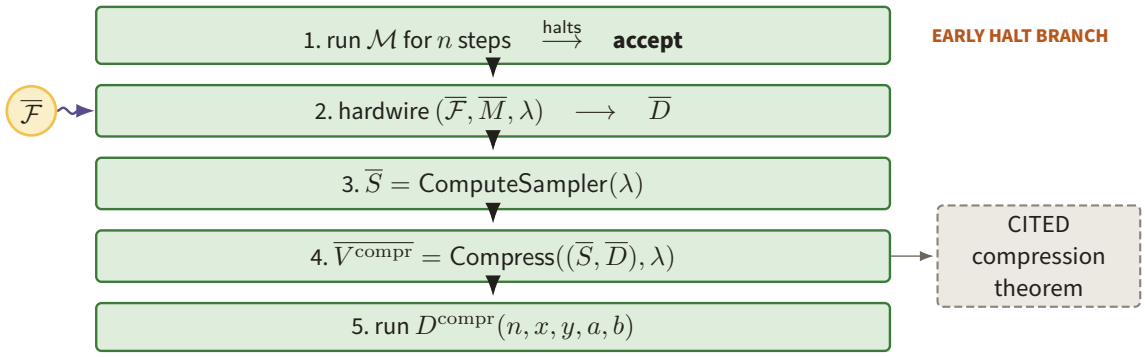


Figure 69. The machine $\mathcal{F}$ first takes the amber early-halt branch or else hardwires its own description, computes S_λ, calls Compress, and runs the returned decider (after `fig:halt_f`). The dashed box isolates the cited compression guarantee from the green source operations that are actually performed.

for the fixed four-bit Fix tag and its fixed sort annotation $c_{\mathrm{fix}} = O(1)$.

Proof. The unique Fix application rule performs three administrative actions: push the application frame, install the self-code heap link, and enter P. The resulting control, ordinary environment, and continuation are the same as for the specialized body; a lookup of the sole hole follows the installed link and therefore returns d_P. Subsequent transitions coincide by Theorem 8.2. Exactly three units have been consumed, which proves (10.3), including matching out-of-fuel and type-error outcomes. Canonical serialization of Fix writes its fixed tag and annotation followed by one copy of the serialization of P; it never expands the self-code link. This proves the size equality. □

10.3 The paper's explicit diagonal program

Reminder. The expression $\mathcal{F}(\mathcal{F}, M, \lambda, \ldots)$ is the recursion theorem unrolled: hardwiring the first copy of $\mathcal{F}$ manufactures the five-input decider whose code is then supplied to Compress.

Figure 69 transcribes the published machine's five operations before expressing the same construction as $\Psi_{M,\lambda}$.

Let $M : P\{\mathsf{MachineDesc}\}$ and $\lambda : P\{\mathsf{Nat}\}$ be closed literal terms, let S_λ be the sampler description returned by $\mathsf{ComputeSampler}(\lambda)$, and let n, x, y, a, b be the five bound inputs. The partial body corresponding to `fig:halt_f` is

$$\begin{aligned}\Psi_{M,\lambda} = \lambda(n,x,y,a,b).\ \mathrm{If}(\,&\mathrm{Prim}(\mathsf{halts\_within}, M, n),\ \mathsf{true},\\ &\mathrm{Eval}(\mathrm{ans}(\mathrm{Eval}(\mathrm{Quote}(\mathsf{Compress}),\\ &\quad(\mathrm{quoted\_pair}(\mathrm{Quote}(S_\lambda), \mathrm{Hole}(\mathsf{self\_code}, \mathrm{Quoted}(\mathsf{Decider}))), \lambda), F_C)),\\ &\quad(n,x,y,a,b), \mathrm{FuelBound}(n,\lambda))).\end{aligned} \tag{10.4}$$

The finite-fuel symbol F_C is any proved bound for the terminating Compress call; ans selects the returned decider description. Define

$$D_{M,\lambda} = \mathrm{YCode}(\Psi_{M,\lambda}), \qquad V_{M,\lambda} = (S_\lambda, \mathrm{Quote}(D_{M,\lambda})). \tag{10.5}$$

Three points at which (10.4) and the surrounding sections deviate from the single source `docs/DESIGN.md` §1.1 are declared here as SOURCE_REPAIR against that file rather than left implicit. First, §1.1's display carries

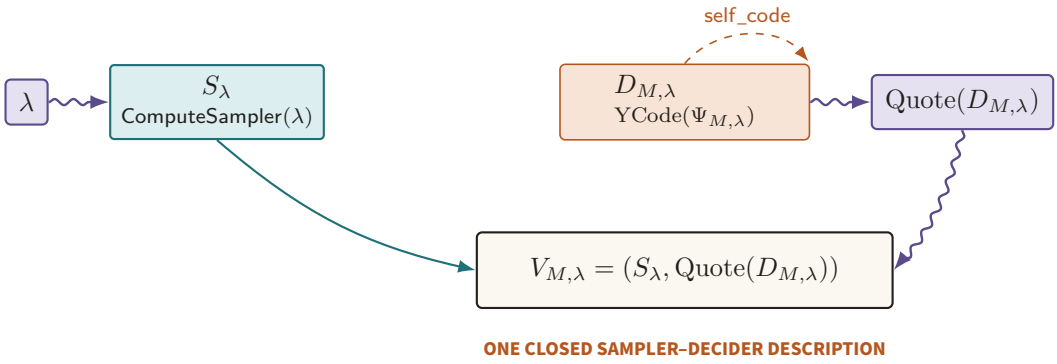


Figure 70. The literal bound λ produces S_λ, while $\mathrm{YCode}(\Psi_{M,\lambda})$ closes its sole self-code socket before quotation places $D_{M,\lambda}$ beside that sampler. The resulting $V_{M,\lambda}$ is one finite normal-form verifier description, not an oracle for either component.

one Eval and neither an ans selector nor a fuel symbol for the compression call; (10.4) uses two nested Evals, ans, and F_C, because the outer evaluation must run the *returned* decider and the inner one must be given a terminating budget. Second, §1.1 writes the fixed-point equation with the *same* fuel on both sides, while Theorem 10.3 charges $f - c_Y$ with $c_Y = 3$ for the self-code link. Third, §1.1 formerly fixed the hole sort to Quoted{Decider}, whereas Section 8 admits Quoted(A) for any result sort A; that repair has since been applied, and `DESIGN.md` §1.1 now declares self_code : Quoted{A}, so on this point the two agree. All three are strengthenings, not simplifications; the first two remain open proposed edits to `DESIGN.md` §1.1, so that the two documents move in lockstep rather than silently diverging.

Figure 70 closes the construction into the exact sampler–decider pair passed to the later game machinery.

The complete constructor tree in Figure 71 locates the single self-code hole inside the nested bounded evaluations.

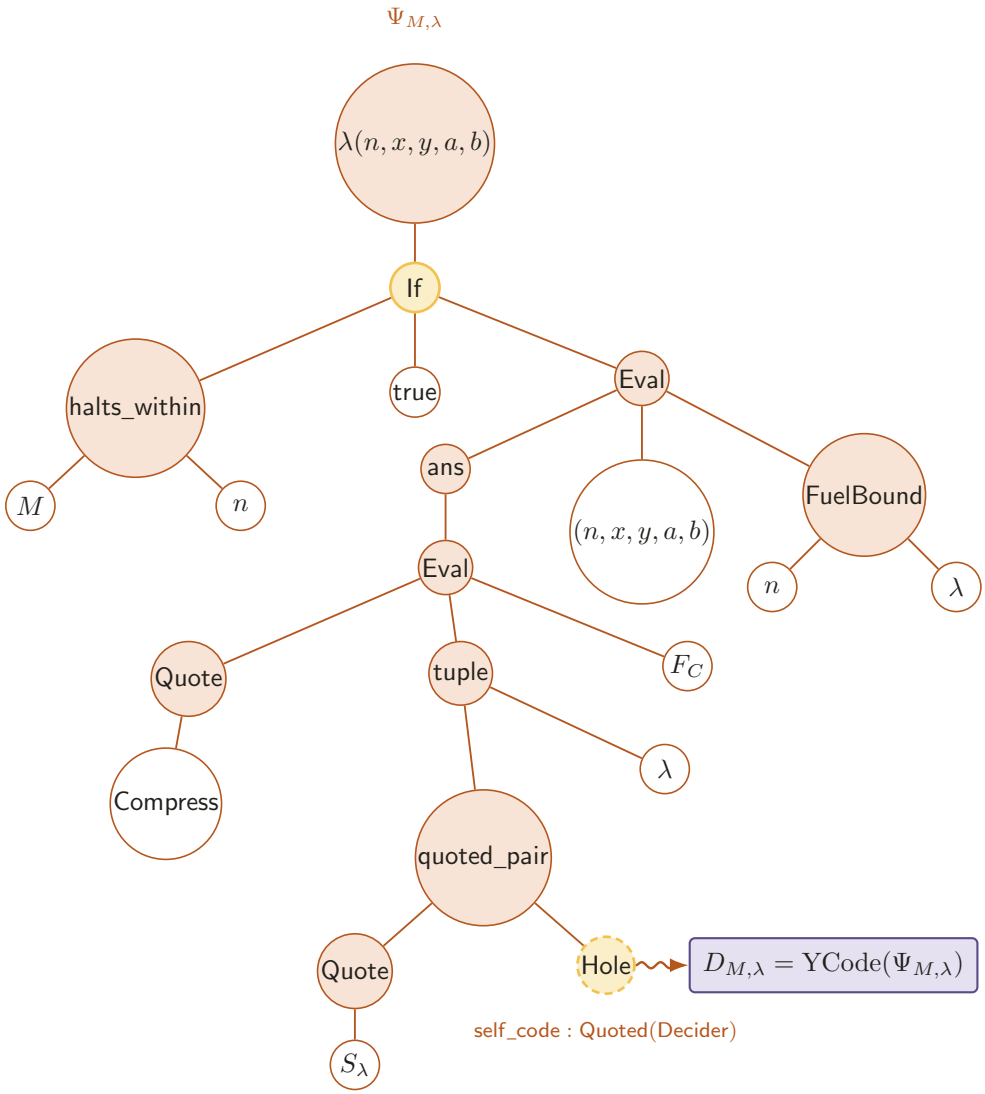


Figure 71. The real term $\Psi_{M,\lambda}$ first tests halts_within(M, n); on failure it evaluates quoted Compress on (quoted_pair$(\mathrm{Quote}(S_\lambda), \mathrm{Hole}(\text{self_code})), \lambda)$, selects the answer decider, and evaluates it on (n, x, y, a, b). The lone amber hole is exactly the source position closed by $D_{M,\lambda} = \mathrm{YCode}(\Psi_{M,\lambda})$.

For the paper, the next theorem buys the identification of the published $\mathcal{F}$ trick, Kleene's theorem, and the IR fixed-point constructor.

Theorem 10.4 (Equivalence of the three fixed-point presentations)**.** *The following are translations of the same description-level construction:*

1. *the paper's explicit* $\mathcal{F}(\overline{\mathcal{F}}, \overline{M}, \lambda, n, x, y, a, b)$*;*
2. *the Kleene fixed point of the transformer which hardwires* $(\overline{M}, \lambda)$ *and inserts the resulting decider code into* Compress*;*
3. *the closed description* $D_{M,\lambda}$ *in* (10.5)*.*

Under the simulations of Section 9, their deciders compute the same partial function. Their description lengths

are all $O(|M| + \log \lambda + 1)$, and their running times differ by a polynomial in $|M|, \log \lambda, n$, the invoked fuel bounds, and the invoked machine times.

Figure 72 identifies the translation along each edge of the paper–Kleene–YCode triangle.

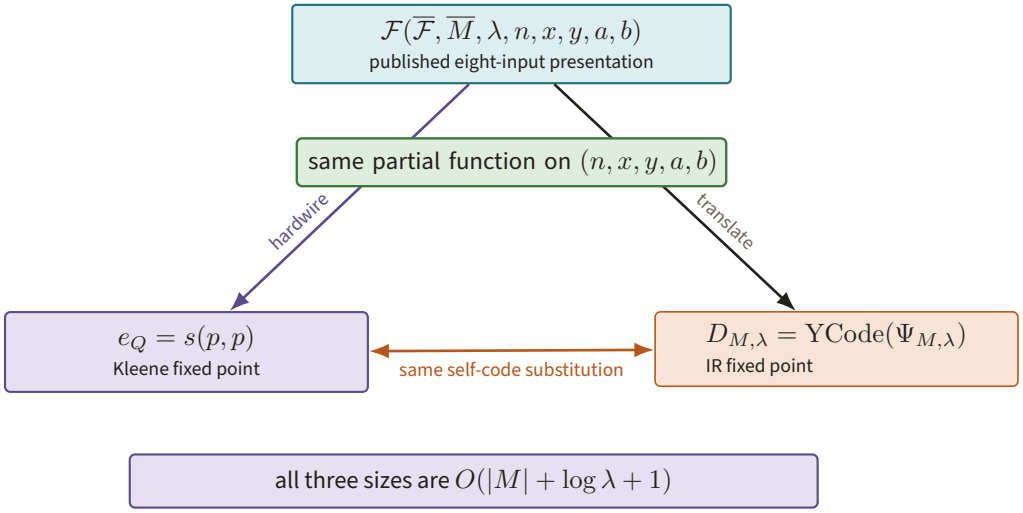


Figure 72. The published $\mathcal{F}(\overline{\mathcal{F}}, \overline{M}, \lambda, \ldots)$, Kleene's $e_Q = s(p, p)$, and $D_{M,\lambda} = \mathrm{YCode}(\Psi_{M,\lambda})$ are three finite descriptions of the same partial function (after `fig:halt_f`). Every edge preserves the self-code substitution and keeps size $O(|M| + \log \lambda + 1)$.

Proof. In the paper, Step 2 of $\mathcal{F}$ applies the hardwiring operation to the eight-input code supplied in its first argument. When that argument is $\overline{\mathcal{F}}$, the resulting five-input code is precisely the code of the outer decider $D(n, x, y, a, b) = \mathcal{F}(\overline{\mathcal{F}}, \overline{M}, \lambda, n, x, y, a, b)$. Thus the string given to Compress is its own decider description. This is the concrete s-m-n operation of Lemma 10.1 with the diagonal argument already chosen.

Alternatively, let $Q(e)$ output the code which performs the halting test and, on failure, applies Compress to (S_λ, e) and runs the returned decider. Theorem 10.2 gives e_Q with $\varphi_{e_Q} = \varphi_{Q(e_Q)}$, which is exactly the behavior just described. Equation (10.2) performs the same substitution with e_Q represented by d_P; Theorem 10.3 proves the operational equation for $D_{M,\lambda}$. Translating it to a machine with Theorem 9.2, or translating $\mathcal{F}$ with Theorem 9.1, preserves that behavior.

The displayed term contains one copy of M, one binary literal λ, and fixed code for the remaining operations. Theorem 10.3 therefore gives $|D_{M,\lambda}| = O(|M| + \log \lambda + 1)$. The paper's hardwiring and the size calculation in Theorem 10.2 give the same bound for the first two forms. Finally, the two simulation theorems replace each bounded call by a fixed polynomial overhead. Composition of finitely many such polynomials is a polynomial in the listed quantities, proving the time statement. □

The paper notes that an earlier version used Kleene's recursion theorem and that its published presentation uses $\mathcal{F}$ explicitly. [11] Theorem 10.4 explains why those presentations differ in notation rather than in the description passed to compression.

What must Compress do with the quoted body? It first applies Specialize to the static verifier and parameter holes, obtaining closed syntax with the size bound of Lemma 8.5. It then traverses that syntax to build the

[11] Ground truth: `fig:halt_f` in `ground-truth/gt-12-compression.tex`.

descriptions for introspection, answer reduction, and repetition, and it records the resulting byte lengths and fuel bounds. It never asks whether two terms compute the same function. Such extensional equality is neither available nor needed. This self-reference is not the hard part: it supplies no low-degree soundness, rigidity, repetition, or gap preservation. Those remain the substantive cited theorems.

Figure 73 makes that final proof boundary explicit: the description construction is costed here, while the analytic consequences remain cited inputs.

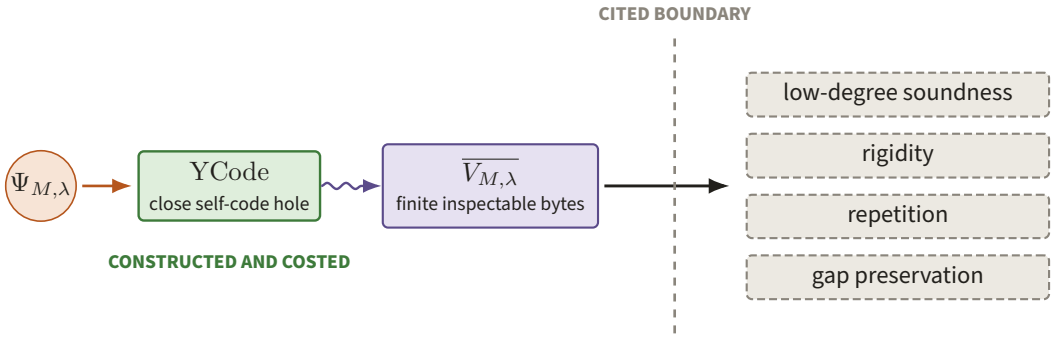


Figure 73. The green path closes $\Psi_{M,\lambda}$ with YCode and emits the finite bytes of $\overline{V_{M,\lambda}}$, after which the arrow crosses a dashed boundary to low-degree soundness, rigidity, repetition, and gap preservation. Self-reference supplies the program consumed by those theorems; it supplies none of their analytic conclusions.

11 Cook–Levin for $\mathcal{L}$-descriptions

Reminder. A machine run is a path of configurations under one local transition rule; Section 1 introduced this viewpoint. Here the same idea is applied to the fixed universal evaluator, turning a fuel-bounded lambda run into locally checkable tableau data.

11.1 A local encoding of bounded evaluation

For closed t, input u, and fuel F, the *bounded trace* is

$$\mathcal{T}(t, u, F) = (C_0, C_1, \ldots, C_r), \qquad r \le F,$$

where C_0 is the initial CEK configuration and each C_{i+1} is its unique small-step successor. A row records the current term pointer, environment-frame pointers, continuation, heap, primitive microstate, outcome flag, and binary fuel. If evaluation halts early, a fixed self-looping final state pads the trace to $F + 1$ rows. This is the promised sequence of (term, environment, fuel) configurations, with the continuation and heap made explicit so that one-step validity is local. Figure 74 isolates the radius-one dependency inside that full padded history.

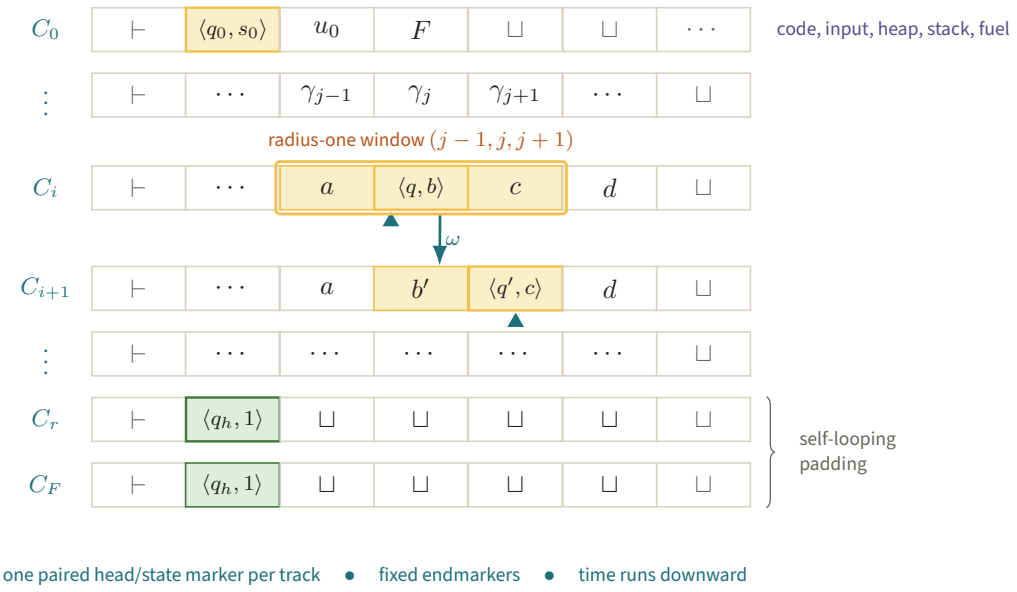


Figure 74. The amber three-cell window in row C_i fixes the marked successor cell and moved head in C_{i+1}. Locality turns a fuel-bounded evaluation into finite constraints, while a halting self-loop supplies exactly $F + 1$ rows.

To obtain a finite-alphabet tableau, compile the CEK evaluator and each charged primitive to a fixed multitape Turing machine $\widehat{U}_{\mathcal{L}}$. The read-only code track contains $\mathrm{ser}(t)$; input tracks hold lazy argument tapes; work tracks hold the environment, heap, stack, and fuel. Pointer arithmetic, traversal, and a primitive of charge k are expanded into their elementary head moves. One microstep changes the finite control, the cells under the heads, and the head positions by at most one.

Lemma 11.1 (Transition-window lemma). *There is a finite alphabet $\Gamma_{\mathcal{L}}$, a fixed number r_0 of tracks, and a Boolean function*

$$\omega : \left(\Gamma_{\mathcal{L}}^{3}\right)^{r_0} \times Q_{\mathcal{L}} \longrightarrow \Gamma_{\mathcal{L}}^{r_0} \times Q_{\mathcal{L}}$$

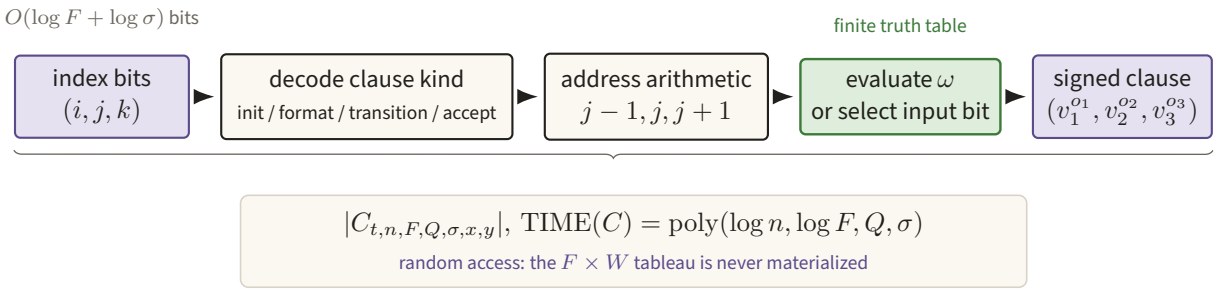


Figure 75. An $O(\log F + \log \sigma)$-bit index is decoded into one real initialization, format, transition, or acceptance clause. Random access keeps the description polynomial in $\log F$ even though the underlying tableau has F time rows.

with the following property. For every nonboundary cell j, the symbols in cell j of row $i+1$, together with the next finite-control state, are determined by the radius-one windows $(j-1, j, j+1)$ of row i and the current control state. Boundary cells use a fixed endmarker version of the same rule. Conversely, if two consecutive rows have one head marker on each track, agree with ω at every window, and have valid endmarkers, then the second row is exactly the successor of the first under the small-step semantics.

Proof. Encode a head by pairing the tape symbol with one of the finitely many machine states when that head is the active head, and use a separate finite-control track for the other heads. In one transition, a cell farther than one position from every head is copied. At a head cell, the transition table determines the written symbol and next state from the old state and scanned symbols. At its left and right neighbors, the table determines whether the head marker enters; it cannot enter any other cell because heads move by at most one. These cases depend only on the three displayed cells on each track. They are mutually exclusive once the old row has one head per track, and therefore define the finite function ω. Endmarkers replace missing neighbors by a fixed boundary symbol.

For the converse, consider each track. Away from the unique old head, the local copy case forces equality. In the three-cell neighborhood of that head, the transition case forces the unique symbol write, direction, and new head position specified by the machine table. The local rules also force all other cells not to acquire a head. Repeating this argument for every track and using the shared finite-control component gives precisely one transition of $\widehat{U}_{\mathcal{L}}$. Primitive execution and CEK operations introduce no exceptional case because they were compiled into the same elementary machine steps. The correspondence between those microstates and the weighted semantics was part of the construction in Section 8. □

11.2 The succinct 3SAT circuit

Let the row width be W. In F microsteps no head visits beyond the initial code and input plus F cells, so $W = O(|t| + |u| + F)$. Introduce Boolean variables for the constant-size binary encoding of every tableau cell and control-state bit. The formula asserts: the initial row contains t, u, F; every row and track has the prescribed format and head marker; each adjacent pair obeys every transition window; and the last row is accepting. Each assertion involves a constant window except binary addressing and the initial read-only strings. Fixed Tseitin gadgets turn it into 3CNF with $O(FW)$ variables and clauses.

For the paper, the next theorem buys a small circuit describing an enormous bounded verifier tableau, which is the input expected by answer reduction. Figure 75 follows one clause index through that circuit.

Theorem 11.2 (Succinct Cook–Levin for $\mathcal{L}$). *There is an algorithm which, on $(t, n, F, Q, \sigma, x, y)$, where $|t| \le \sigma$ and $|x|, |y| \le Q$, outputs a Boolean circuit $C_{t,n,F,Q,\sigma,x,y}$ deciding membership of an indexed signed*

clause in a 3CNF formula Φ. Its index width is $O(\log F + \log \sigma)$, its size and construction time are

$$\mathrm{poly}(\log n, \log F, Q, \sigma), \tag{11.1}$$

and for every pair of answer strings a, b the formula is satisfiable with the prescribed input bits if and only if $\mathrm{Eval}_{\mathcal{L}}(\mathrm{Quote}(t), (n, x, y, a, b); F)$ accepts. Holding the public input fixed, the transition component of the circuit has size $\mathrm{poly}(\log F, |t|)$; the Q term in (11.1) *is solely the hardwired initialization data x, y.*

Proof. Given a clause index, the circuit first decodes whether the requested clause is an initialization, uniqueness/format, transition, or acceptance clause. Row and column numbers use $O(\log F + \log W)$ bits. For a transition clause it computes the addresses of a radius-one window and evaluates the finite truth table ω from Lemma 11.1; binary addition, comparison, and multiplexing have size polynomial in those address lengths. For an initialization clause it selects a bit of the canonical code or public input. A balanced selector for a hardwired string has size linear in $|t| + Q + \log n$. Format and final-state clauses use fixed truth tables. Finally it emits the three variable addresses and signs of the selected 3CNF gadget. This proves the circuit and construction bounds. Since $W = O(|t| + Q + F)$, $\log W = O(\log F + \log \sigma + \log(Q + 1))$, which is absorbed by (11.1) under $Q \le F$, as in the paper.

If evaluation accepts, assign every tableau variable from its unique bounded trace and every Tseitin variable from its gadget value. Initialization, transition, and acceptance clauses are then satisfied. Conversely, any satisfying assignment contains the prescribed initial row. Induction on the row number and the converse part of Lemma 11.1 forces each later row to be the unique evaluator successor. The final clauses force that trace to accept. The prescribed answer bits occupy the initial answer tapes, so the equivalence holds for each a, b. □

This is the $\mathcal{L}$-description restatement of `prop:standard-succinct-sat`. The paper's version takes D, n, T, Q, σ, x, y, reserves the first $2T$ encoded positions for each answer, and has size $\mathrm{poly}(\log n, \log T, Q, \sigma)$. [12] Theorem 9.2 gives the same proposition by compilation; the proof above shows directly why a lambda evaluation trace has the required locality. Figure 76 draws the reserved answer block behind that placement property.

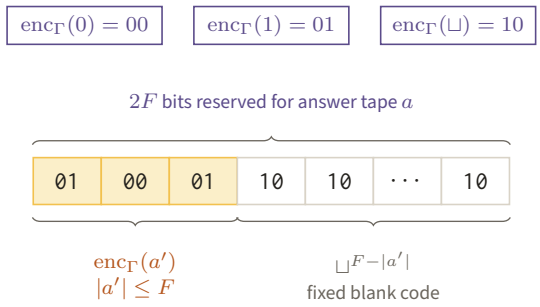


Figure 76. Three dual-rail codes turn a prefix a' of at most F tape symbols into the amber head of one $2F$-bit block, and every remaining cell carries the fixed blank code. Reserving the block this way is what lets the decoupled formula read a as an independent assignment.

More explicitly, use the paper's tape-alphabet encoding $\mathrm{enc}_\Gamma(0) = 00$, $\mathrm{enc}_\Gamma(1) = 01$, $\mathrm{enc}_\Gamma(\sqcup) = 10$, and reserve $2F$ bits in the tableau assignment for each answer tape. For every $a, b \in \{0,1\}^{2F}$, the resulting

[12] Ground truth: `prop:standard-succinct-sat` in `ground-truth/gt-10-answer-reduction.tex`.

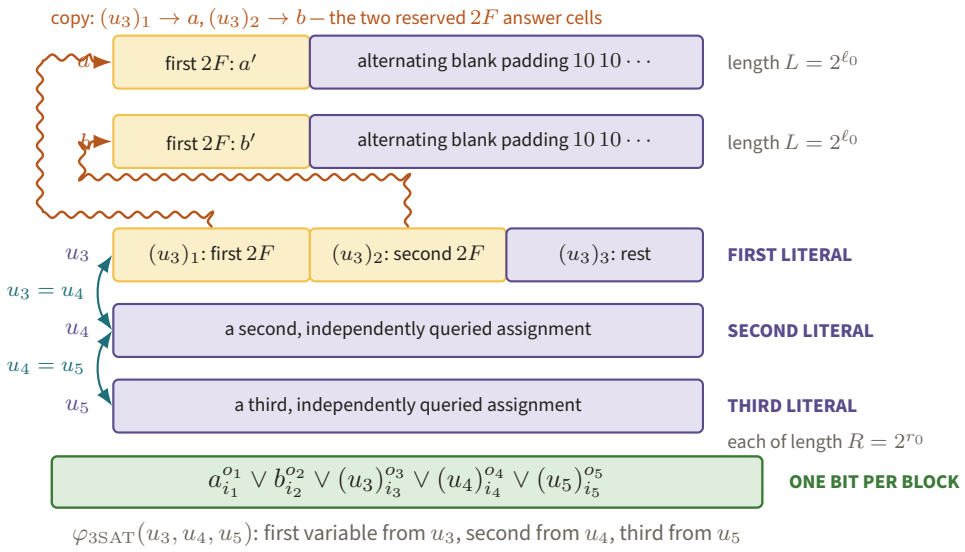


Figure 77. Each 3SAT clause places its first, second and third literal in u_3, u_4, u_5 respectively, and the equality gadgets restore $u_3 = u_4 = u_5$; the two answer blocks are copied out of the first and second $2F$ cells of u_3. One addressed bit per block is all (11.2) permits, which is why decoupling is what makes three independent low-degree answers usable.

formula has an extension c if and only if there are prefixes a', b', of lengths at most F, such that

$$a = \mathrm{enc}_\Gamma(a', \sqcup^{F-|a'|}), \qquad b = \mathrm{enc}_\Gamma(b', \sqcup^{F-|b'|}),$$

and the term accepts (n, x, y, a', b') within fuel F. The forward direction follows because the initialization clauses force the two reserved blocks to be valid padded tape rows; the backward direction fills those blocks from the accepting trace. This is the exact placement property needed when the two blocks are separated in (11.2). Figure 77 makes the three kinds of constraints visible on the five power-of-two blocks.

The one-read-per-block geometry of Figure 78 is the point of the five-way split.

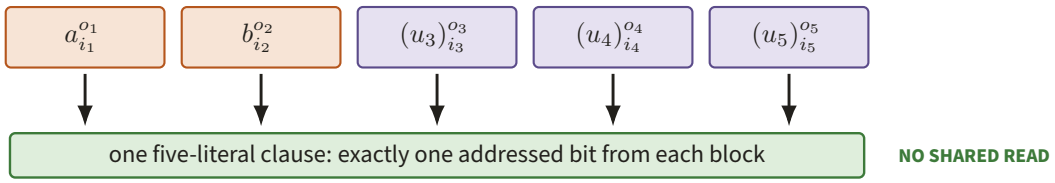


Figure 78. A decoupled clause reads exactly one addressed literal from each of a, b, u_3, u_4, u_5, with the two answer blocks distinguished in rust. Independent low-degree answers can now supply the five locations without pretending to be three reads of one shared assignment.

11.3 From shared 3SAT data to decoupled 5SAT

The paper removes two undesirable features before low-degree encoding. A 3SAT clause reads three locations of one tableau assignment, while three low-degree answers need not be restrictions of one assignment. Also, the strings a, b are embedded inside that assignment, whereas the later sampler must query them as independent blocks. Make five assignments a, b, u_3, u_4, u_5 and impose clauses of the form

$$a_{i_1}^{o_1} \vee b_{i_2}^{o_2} \vee (u_3)_{i_3}^{o_3} \vee (u_4)_{i_4}^{o_4} \vee (u_5)_{i_5}^{o_5}. \tag{11.2}$$

Figure 79 carries one concrete object through all three steps of this section before the general statement, so that "window", "clause index" and "block" can be seen to name the same thing three times.

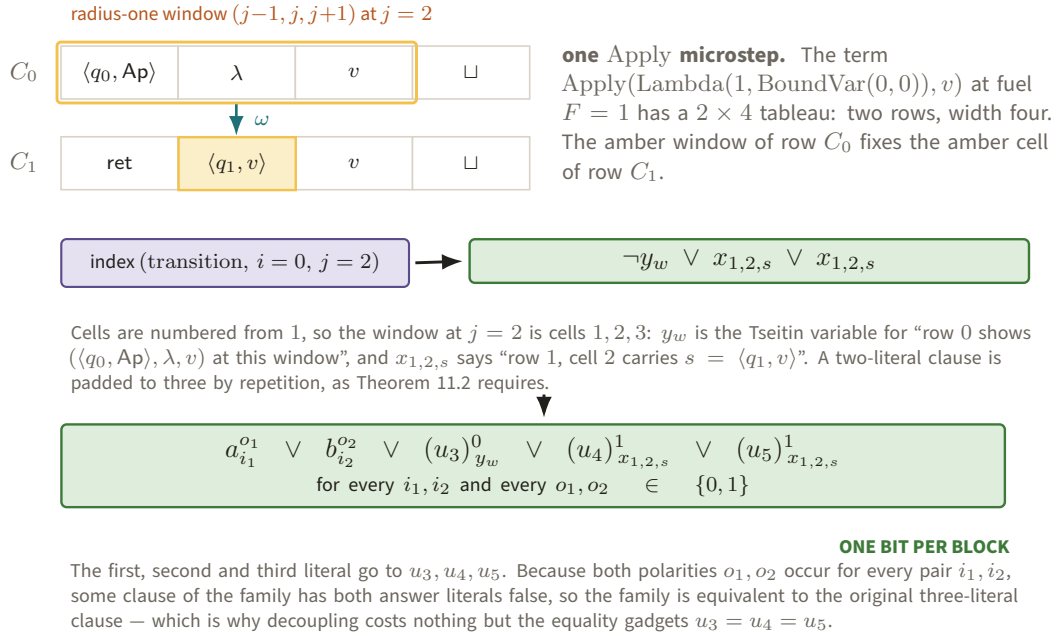


Figure 79. One Apply microstep at fuel $F = 1$: its 2×4 tableau and the window of Figure 74, the single clause index that window produces, and the five-block family that clause becomes. Following one object rather than a schema shows where each literal of a 3SAT clause actually lands.

Each original 3SAT clause contributes its *first* literal to u_3, its *second* to u_4 and its *third* to u_5; this is forced by the shape of (11.2), which addresses exactly one bit per block, and is exactly the paper's convention that in $\varphi_{3\mathrm{SAT}}(u_3, u_4, u_5)$ the first variable of each constraint is taken from u_3, the second from u_4 and the third from u_5. Constant-size five-literal gadgets then enforce $u_3 = u_4 = u_5$, so the decoupled instance is satisfiable exactly when the shared-assignment one is; further gadgets copy the first $2F$ encoded answer cells of u_3 to a and its second $2F$ cells to b, and pad unused positions by the alternating blank encoding. Each gadget is indexed by comparisons and additions on $O(\log F + \log \sigma)$-bit addresses, so it adds only that many gates. Therefore the succinct decoupled circuit retains the bound (11.1). Its padded form gives each block i a power-of-two length $N_i = 2^{n_i}$: the two answer blocks share the length $L = 2^{\ell_0}$ with $\ell_0 = \lceil \log 2F \rceil$, and the three tableau blocks share $R = 2^{r_0}$ with r_0 the succinct index width; the lengths need not agree across the two groups. A satisfying five-tuple restricts to a satisfying trace assignment, and a satisfying trace extends by copying and padding; this proves the required if-and-only-if. [13]

The intended analytic chain is

$$\texttt{Quoted\{Decider\}} \xrightarrow{\texttt{bounded\_trace}} \texttt{BoundedTrace}$$
$$\texttt{BoundedTrace} \xrightarrow{\texttt{cook\_levin}} \texttt{Succinct3SAT}$$
$$\texttt{Succinct3SAT} \xrightarrow{\texttt{decouple5}} \texttt{SuccinctDecoupled5SAT}$$
$$\texttt{SuccinctDecoupled5SAT} \xrightarrow{\texttt{padding}} \texttt{PaddedSuccinctDecoupled5SAT}.$$

[13] Ground truth: `sec:succinct-deciders` in `ground-truth/gt-10-answer-reduction.tex`.

These are the transformations built in TB3 (`briefs/23-tb3.md`, repaired in brief 63): `bounded_trace`, `cook_levin`, `decouple5` and `pad5` exist in `src/frontend/` and run the generated instance through the PCP builder and the answer-reduced decider on two one-bit-answer fixtures. That is fixture evidence for the decoupled five-block placement only (claim C19, SKETCH); the alphabet, window function and answer-block reservation are not implemented and the general contracts remain CITED. The proposed row C10 awaits its critic round.

11.4 Tseitin and arithmetization

Figure 80 previews the three gate replacements and their Boolean-cube contract as one local rewrite diagram.

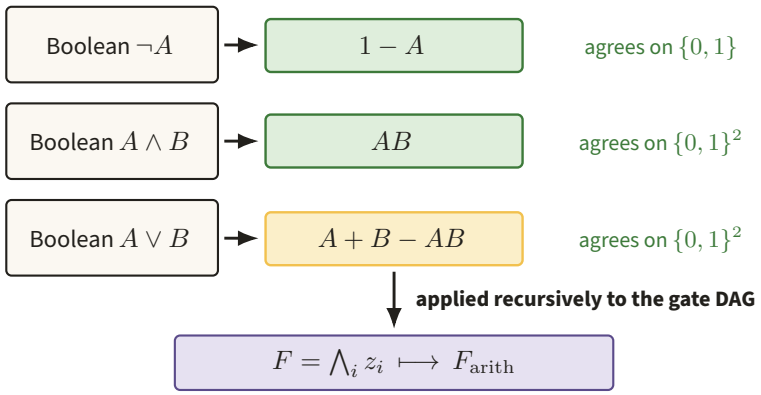


Figure 80. Negation, conjunction, and disjunction become $1 - A$, AB, and the amber polynomial $A + B - AB$, respectively. These replacements preserve values on the Boolean cube while producing the formal polynomial consumed by the PCP layer.

Reminder. At this point the quoted verifier has already been reduced to a finite circuit describing local computation checks. The transformations below manipulate that circuit description; they are computable source maps, while their low-degree and quantum soundness consequences are separate analytic claims.

Given a Boolean circuit C with gates g_i, Tseitin introduces a wire variable w_i and an equivalence formula

$$z_i = (g_i(x, w) \wedge w_i) \vee (\neg g_i(x, w) \wedge \neg w_i), \qquad F = \bigwedge_i z_i.$$

The required contract is $C(x) = 1$ iff there exists w with $F(x, w) = 1$, with size and construction time $O(|C|)$.[14] The project follows the explicit gate-equivalence form in `ground-truth/nw19/nw19-tseitin-arith.tex` and also conjoins the output literal w_{out}, so the stored formula explicitly asserts acceptance. That convention is marked SOURCE_REPAIR; it is not silently attributed to the source.

The Julia constructors `Circuit`, `Input`, `Gate`, `NotGate`, `AndGate`, and `OrGate` make the finite DAG and its topological order explicit. The function `tseitin` returns `Checked{TseitinFormula,CertNode}`. Its replay recomputes the formula's occurrence vector; it does not, by that replay alone, establish the general Cook–Levin or Tseitin semantic equivalence. Those implications are exhausted only on the small TB0 fixture and remain cited in general.

[14] Ground truth: `def:tseitin` in `ground-truth/gt-10-answer-reduction.tex`.

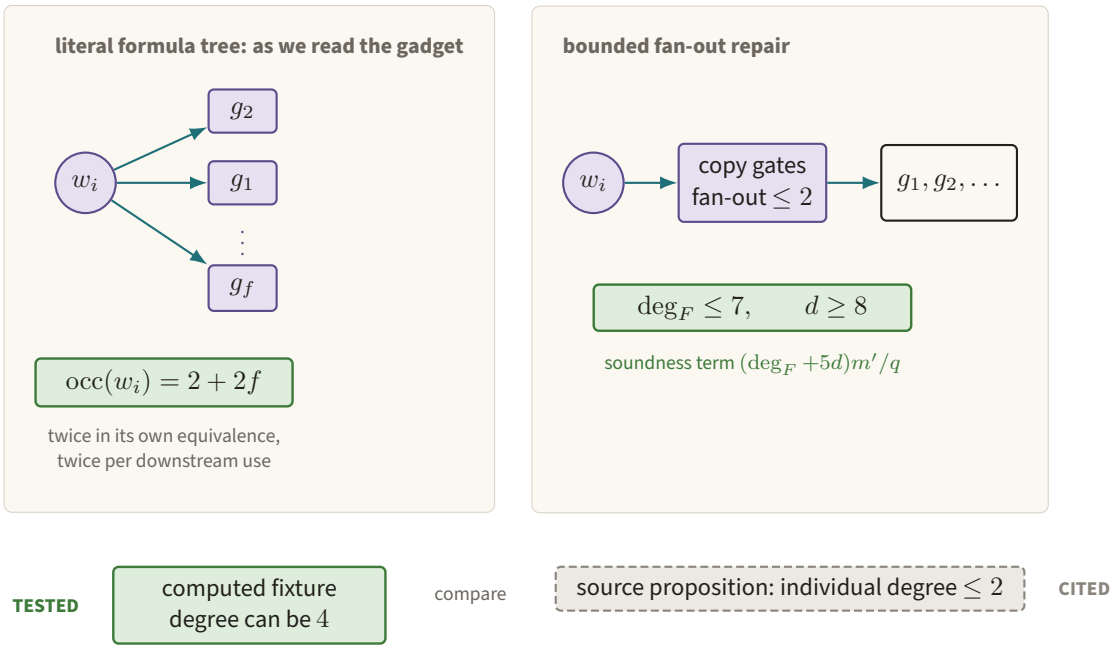


Figure 81. As we read the gate-equivalence formula, a wire used by f later gates contributes $2 + 2f$ literal occurrences, and the two-gate fixture reaches individual degree four. Bounded fan-out restores the conservative $\deg_F \leq 7, d \geq 8$ route without conflating a tested account with the cited source proposition.

Arithmetization recursively applies

$$\neg A \mapsto 1 - A, \qquad A \wedge B \mapsto AB, \qquad A \vee B \mapsto A + B - AB.$$

It agrees with the Boolean formula on the Boolean cube. [15] The related NW19 rule is `def:arithmetization` in `ground-truth/nw19/nw19-tseitin-arith.tex`. Julia's `arith_q` returns a sparse formal `Poly`; its `ArithTseitin` certificate checks support degrees against a structural bound.

11.5 The occurrence-vector discrepancy

There is a discrepancy in the individual-degree bookkeeping as we read NW19; our reading may be wrong. The source proposition states individual degree at most two for the Tseitin arithmetization. [16] Figure 81 contrasts that cited statement with the literal-occurrence account and the bounded-fan-out repair. Reading the NW19 formula literally as a formula tree, arithmetization is multilinear in leaf occurrences, not necessarily in circuit wires. The same wire can occur in several gate gadgets. With the project's explicit output literal, the count we compute is

$$\mathrm{occ}(x_j) = 2\,\mathrm{fanout}(x_j), \qquad \mathrm{occ}(w_i) = 2 + 2\,\mathrm{fanout}(w_i) + \mathbf{1}_{i=\mathrm{out}},$$

and structural induction gives

$$\deg_v(F_{\mathrm{arith}}) \leq \mathrm{occ}_v(F).$$

Figure 82 carries both accounts out on the two-gate fixture, for the one wire whose count exceeds two.

[15] Ground truth: `def:formula-arithmetization` in `ground-truth/gt-10-answer-reduction.tex`.
[16] Ground truth: `prop:tseitin-arith-degree` in `ground-truth/gt-10-answer-reduction.tex`.

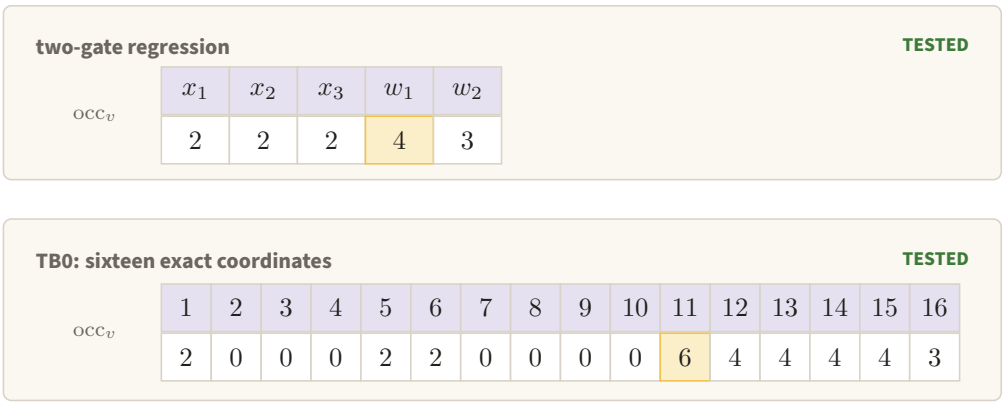


Figure 83. The amber entries record individual degrees four and six in the two named, exactly traversed formula trees. These fixture vectors support the occurrence diagnosis without promoting the general fan-out formula to a machine theorem.

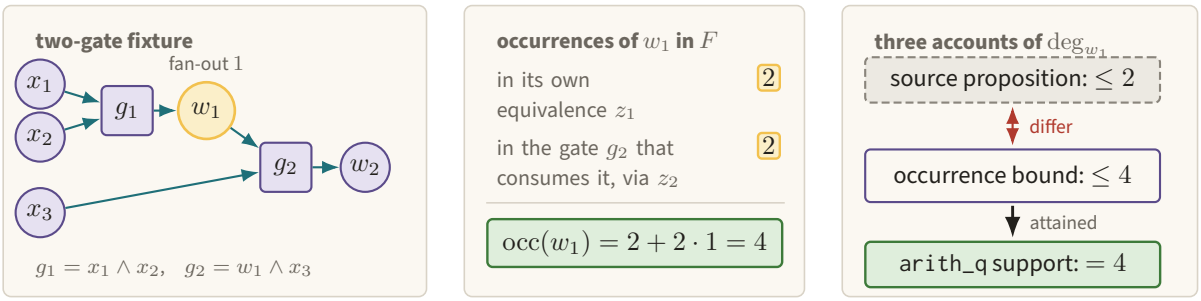


Figure 82. The amber wire w_1 occurs four times in the literal formula tree and the measured sparse-support degree meets that bound, while the cited proposition allows two; our reading of the gadget may be the thing at fault (finding F1 of `docs/findings.md`).

The function `tseitin_occurrence_account` computes this vector; `occurrences` independently traverses the formula; and `arith_q` compares the structural account with actual sparse support.

Claim C8 is only `C8`: TESTED: the two-gate regression gives $(2, 2, 2, 4, 3)$, and the 16-variable TB0 formula attains

$$(2, 0, 0, 0, 2, 2, 0, 0, 0, 0, 6, 4, 4, 4, 4, 3).$$

Figure 83 keeps the two measured vectors side by side and singles out the coordinates that exceed two. The general occurrence formula is a written derivation, not a machine theorem. The downstream soundness repair is only `C5`: SKETCH. With a symbolic bound $\deg_F = \max_v \mathrm{occ}_v(F)$, the conservative formula-test term becomes $(\deg_F + 5d)m'/q$, and `P_formula_structural` is an additional obligation. Under an explicit fanout-reduction hypothesis, $\mathrm{fanout}_{\max} \leq 2$ gives $\deg_F \leq 7$ with the output literal, and $d \geq 8$ suffices for the individual-degree construction bound. Absorption into the asymptotic parameter choice still needs the stated growth hypotheses; it is not promoted by the two finite computations.

12 The low-degree PCP as algebra

This section is one argument in four layers, and Figure 84 is its roadmap: what the prototype evaluates, what the evaluation is allowed to conclude, and the two imported theorems on either side. Sections 12.1–12.3 build the algebra the top row needs; §12.4 walks the boxes in order.

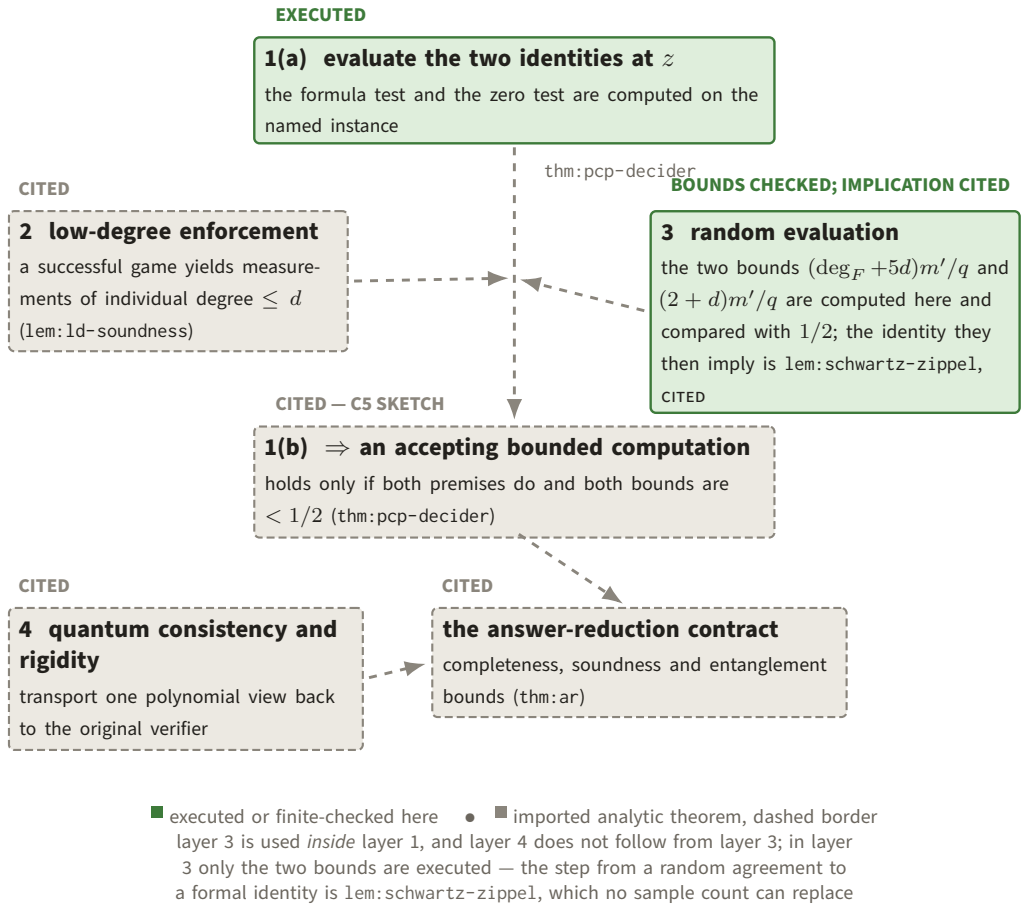


Figure 84. Layer 1 is split: the green box is the identity evaluation the prototype performs, the dashed box below it is thm:pcp-decider, and only the second carries C5. Layer 3 is green for the two bounds it computes, and its box says that the identity they imply is lem:schwartz-zippel, imported. The arrows are the real dependency — layers 2 and 3 feed layer 1(b), and layers 1 and 4 feed thm:ar.

12.1 Interpolation and the vanishing polynomial

For $M = 2^m$ and $a = (a_y)_{y \in \{0,1\}^m}$, define

$$\mathrm{ind}_{m,y}(x) = \prod_{j : y_j = 1} x_j \prod_{j : y_j = 0} (1 - x_j), \qquad g_a(x) = \sum_y a_y \, \mathrm{ind}_{m,y}(x).$$

Then $g_a(y) = a_y$ on the Boolean cube and $a \mapsto g_a$ is linear. This is the low-degree code: the assignment is represented by the evaluation table of its unique multilinear extension. Figure 85 locates the Boolean data inside the ambient field and pairs it with the basic vanishing factor. [17] The Julia function g_a constructs the sparse polynomial; block coordinates are carried by VarLayout and VarBlock; dependency_blocks recomputes the actual support dependency.

For five satisfying blocks, let g_i be their multilinear extensions and write $z = (x_1, \ldots, x_5, o, w) \in \mathbb{F}_q^{m'}$. The central analytic object is

$$c_0(z) = F_{\mathrm{arith}}(z) \prod_{i=1}^{5} \big(g_i(x_i) - o_i \big).$$

[17] Ground truth: sec:ld-encoding in ground-truth/gt-03-prelim.tex.

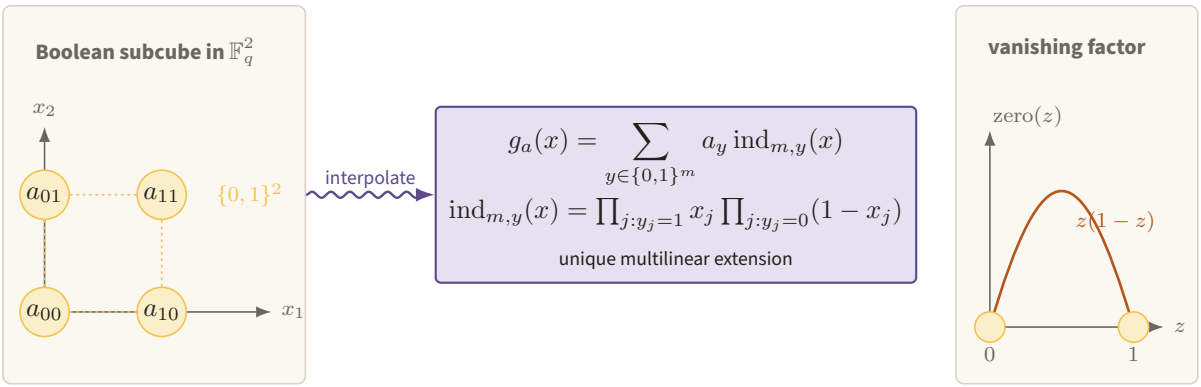


Figure 85. Amber vertices carry the Boolean assignment values inside $\mathbb{F}_q^m$, and the violet arrow produces their unique multilinear extension g_a. The factor $z(1-z)$ vanishes exactly at the two Boolean axis points that generate the subcube ideal.

On a Boolean point, either $F_{\text{arith}} = 0$, or the indicated clause is present and at least one factor $g_i(x_i) - o_i$ vanishes. Hence c_0 vanishes on $\{0,1\}^{m'}$. The code executes the product in `build_c0`; the `BuildC0` certificate replays the structural and actual degree accounts. The implication from a satisfying decoupled instance to Boolean-cube vanishing is checked only on the TB0 fixture and cited in general.[18]

12.2 Division by the Boolean ideal

Let $z_i(1-z_i) = z_i - z_i^2$. The ground-truth basis proposition says that a degree-bounded polynomial vanishing on the Boolean cube lies in the ideal generated by these m' polynomials. [19] The implementation uses the following deterministic monomial rewrite.

Lemma 12.1 (Executable zero-basis rewrite)**.** *For a coefficient a, a monomial u independent of the displayed power, and $e \geq 2$,*

$$auz_i^e \longmapsto auz_i^{e-1} - auz_i^{e-2}\, z_i(1-z_i).$$

Iterating in a fixed variable and monomial order yields polynomials c_i and a remainder r, multilinear in every variable, such that

$$c_0 = \sum_{i=1}^{m'} c_i z_i(1-z_i) + r.$$

If the coordinate degree vector of c_0 is D, then $\deg_j(c_i) \leq D_j$, $\deg_i(c_i) \leq \max(D_i - 2, 0)$, and after reducing coordinates $1,\dots,i-1$, the running remainder has degree at most one in those coordinates.

The concrete two-variable run in Figure 86 shows where the quotient and zero remainder are recorded.

Proof. Expanding the right-hand side of one rewrite gives

$$auz_i^{e-1} - auz_i^{e-2}(z_i - z_i^2) = auz_i^e.$$

Thus every step preserves the coefficient polynomial after adding the recorded quotient contribution. It lowers the current exponent to one and places power $e-2$ in the quotient, proving the degree bounds. Induction over the ordered coordinates gives the displayed decomposition and a final multilinear remainder.

[18] Ground truth: `thm:pcp-decider` in `ground-truth/gt-10-answer-reduction.tex`.
[19] Ground truth: `prop:zero-basis` in `ground-truth/gt-10-answer-reduction.tex`.

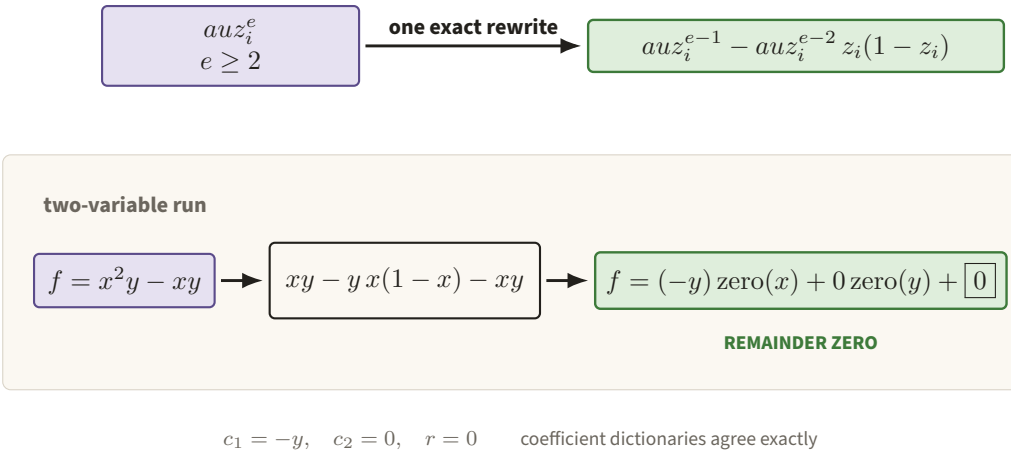


Figure 86. One rewrite peels a power of z_i into a lower power and a multiple of $z_i(1 - z_i)$, with no evaluation sampling. For $x^2y - xy$, the ordered run returns $c_1 = -y$, $c_2 = 0$, and an exactly empty remainder.

If c_0 vanishes on the Boolean cube, then so does r; uniqueness of multilinear interpolation on $\{0,1\}^{m'}$ makes $r = 0$. This is the constructive division used in the proof of `prop:zero-basis` in `ground-truth/gt-10-answer-reduction.tex`. □

`zero_basis_decompose` stores every step as `RewriteStep` inside a `ZeroDecomposition`. `verify_rewrite_step` replays the scalar identity, and `verify_zero_decomposition` reconstructs the entire right side as a normalized coefficient dictionary. Promotion to a zero certificate also requires empty remainder. This is stronger than checking the equality at sampled points, but it remains an instance certificate, not a proof of the PCP soundness theorem.

12.3 PCP view and local verifier

A *PCP view* at z is

$$\mathrm{ev}_z(\Pi) = (g_1(x_1), \ldots, g_5(x_5), c_0(z), c_1(z), \ldots, c_{m'}(z)).$$

The proof object is the tuple of evaluation tables for those polynomials. The labels are `def:pcp-proof` and `def:pcp-eval` in `ground-truth/gt-10-answer-reduction.tex`. Julia represents them by `PCPProof` and `PCPView`; `ev_z` refuses a g_i whose actual support escapes block X_i.

The verifier first reconstructs the succinct circuit, Tseitin formula, and arithmetization. It then checks exactly

$$\beta_0 = F_{\mathrm{arith}}(z) \prod_{i=1}^{5} (\alpha_i - o_i), \qquad \beta_0 = \sum_{j=1}^{m'} \beta_j z_j (1 - z_j).$$

These are the formula test and zero-on-subcube test. Figure 87 draws the queried proof tables and keeps the two executed equalities above the cited low-degree boundary. [20] The Julia function `pcpverifier` executes these two equations on an already supplied `TseitinFormula` and view. In TB2 it does not implement the figure's preceding reconstruction steps; the formula is construction-time data. `build_pcp` combines upstream certificates, coefficient division, degree checking, and stored certified views.

[20] Ground truth: `fig:pcpverifier` in `ground-truth/gt-10-answer-reduction.tex`.

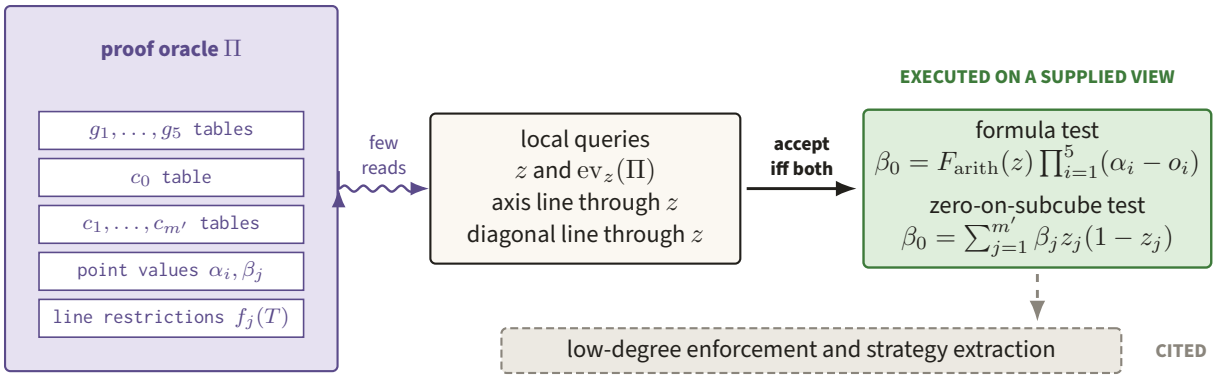


Figure 87. Point and line queries land in evaluation tables for $g_1, \ldots, g_5, c_0, \ldots, c_{m'}$, while the green decider checks the two displayed identities at z. Those local checks are executable, but low-degree enforcement and quantum strategy extraction remain cited analytic input.

12.4 The four soundness layers

The four layers must not be collapsed. The four layers are Figure 84, printed at the head of this section; each paragraph below is one of its boxes.

1. Algebraic soundness under a low-degree premise. Assume every submitted g_i, c_j has individual degree at most d. If the formula-test polynomials are unequal, their difference has total degree at most $(\deg_F + 5d)m'$ under the occurrence-vector account. If the zero-test polynomials are unequal, their difference has total degree at most $(2 + d)m'$. Schwartz–Zippel states that unequal polynomials of total degree at most Δ agree at a uniform point of $\mathbb{F}_q^{m'}$ with probability at most Δ/q. [21] Consequently

$$\Pr[\text{false formula identity}] \le \frac{(\deg_F + 5d)m'}{q}, \qquad \Pr[\text{false zero identity}] \le \frac{(2+d)m'}{q}.$$

The paper's displayed first numerator uses $2 + 5d$; the structural project obligation substitutes the measured $\deg_F$ under the discrepancy stated above. The field size enters nowhere mysteriously: $q = 2^k$ is chosen so both fractions are below the acceptance threshold $1/2$, together with the other PCP parameter obligations. The first structural inequality is extra to the literal `def:pcpparams` predicate in the present project. Figure 88 places both bounds against that threshold.

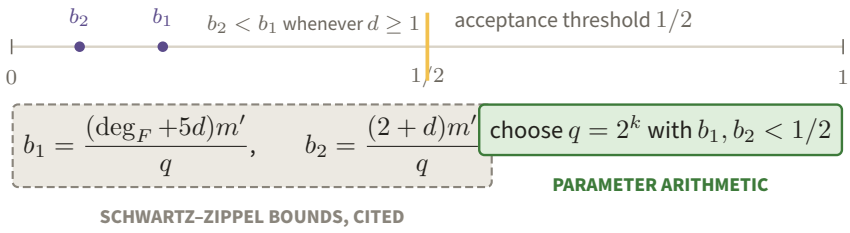


Figure 88. Both Schwartz–Zippel bounds must fall left of the amber acceptance threshold, which is exactly what the choice $q = 2^k$ buys; the formula bound b_1 is the larger of the two for every $d \ge 1$, so it is the binding one. The bounds themselves are cited analytic input; only the parameter arithmetic that compares them with $1/2$ is executed here.

If acceptance is greater than $1/2$ and both upper bounds are below $1/2$, each sampled equality is therefore

[21] Ground truth: `lem:schwartz-zippel` in `ground-truth/gt-03-prelim.tex`.

a polynomial identity. The zero identity makes c_0 vanish on the Boolean cube; the formula identity then decodes five Boolean assignments satisfying the decoupled formula, and hence an accepting bounded computation. This conditional theorem is `thm:pcp-decider` in `ground-truth/gt-10-answer-reduction.tex`. Its current project status is `C5`: SKETCH, not CHECKED.

2. Enforcement of the low-degree premise. The simultaneous low-degree game queries points, axis-parallel lines, and diagonal lines. Its classical decider checks consistency of a point value with a claimed line restriction.[22] The theorem extracting approximately consistent low-individual-degree polynomial measurements from a successful quantum strategy is `lem:ld-soundness` in `ground-truth/gt-07-ldt.tex`. It is CITED. Running `ld_decider` on honest finite inputs does not prove this extraction theorem.

3. Polynomial identity from random evaluation. This is exactly the Schwartz–Zippel step above, used twice with two different degree bounds. The local program checks evaluations. The analytic theorem turns an acceptance probability into formal identity, provided the degree and field-size premises hold. No sample count can replace that implication.

4. Quantum consistency and rigidity. Answer reduction must connect different players' point and line answers to the same polynomial measurements, control disturbance, and transport the result back to the original verifier. Those operator estimates use low-degree soundness, typed consistency transformations, and later rigidity arguments. They are imported in the soundness contract of `thm:ar` in `ground-truth/gt-10-answer-reduction.tex`; none is a sparse-polynomial identity.

The computer adds exactly this: on a named instance it constructs the polynomials, executes the division, compares normalized coefficients, computes degree and dependency vectors, and evaluates the two local equations. It does not test the theorem. Current statuses reflect that separation: C2, C3 and C8 are TESTED on their named fixtures only, C5 is SKETCH, and C1 — the acceptance of the constructed proof at every z — remains CONJECTURE.

13 The typed layer: conditionally linear functions

13.1 The inductive object and its laws

Figure 89 previews the single inductive CL step before the full level diagram.

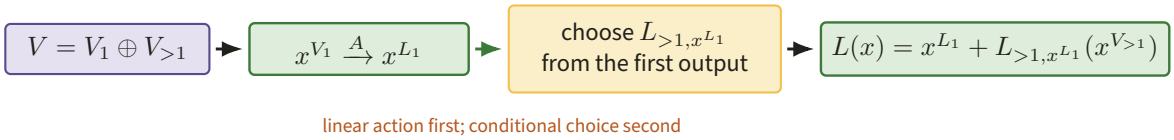


Figure 89. A linear map acts on V_1, and its actual output selects the continuation on the complementary register $V_{>1}$. This one-step constructor is the recursive source of every CL level and marginal law below.

Definition 13.1 (Conditionally linear function)**.** Let $V = \mathbb{F}^n$ with fixed coordinate-register subspaces. The unique level-zero conditionally linear (CL) function is zero. A level-ℓ CL function chooses complementary

[22] Ground truth: `fig:ld-decider` in `ground-truth/gt-07-ldt.tex`.

registers $V_1 \oplus V_{>1} = V$, applies a linear map $A : V_1 \to V_1$, and, conditional only on $A(x^{V_1})$, selects a level-$(\ell - 1)$ CL function on $V_{>1}$. Its output is the sum of the first-stage and continuation outputs.

Figure 90 unfolds this definition into its sequential factor registers, conditional choices, and two level laws.

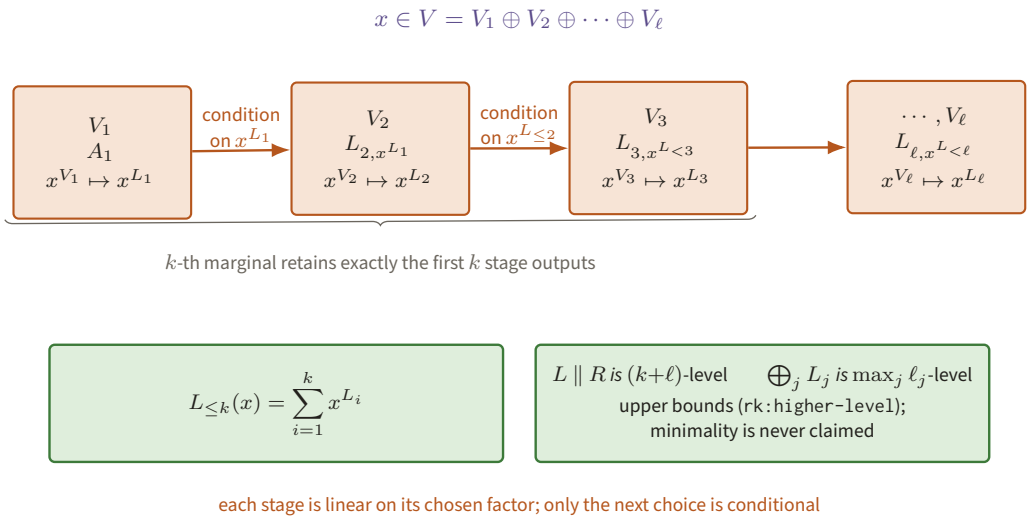


Figure 90. Each rust stage applies a linear map on one complementary register, while the labelled dependency lets earlier outputs select the next stage. Marginals sum prefixes of stage outputs; concatenation exhibits $k + \ell$ stages and direct sum $\max_j \ell_j$; neither is claimed minimal.

This is `def:cl-func` in `ground-truth/gt-04-cl.tex`. Julia mirrors it with `AbstractCL`, `CLZero`, and `CLStep`. The constructor verifies disjoint factor/rest coordinate sets, a matrix acting inside the factor, a fixed child level, and exact coverage of the rest register. Thus an inhabitant of `CLStep` has a constructed level; linearity is carried by the matrix, not inferred from samples.

The marginal $L_{\le k}$ is the sum of the first k stage outputs, together with their factor spaces and linear maps. This is the decomposition of `lem:cl-kth` in `ground-truth/gt-04-cl.tex`. Julia's `marginal_k` returns a `CLMarginal`; `sum_stage_outputs` recomputes its value.

A level is a property a CL function *has*, never one it has uniquely. The source says so directly: the marginal functions, factor spaces and linear maps of a CL function need not be unique, and the identity on $\mathbb{F}^n$ is a 1-level CL function which may equally be presented as a k-level one for any $k \in \{2, \dots, n\}$. Correspondingly, every $(\ell - 1)$-level CL function is also ℓ-level.[23] Both facts are used below, and neither can be strengthened to an equality.

Lemma 13.2 (Level laws). *Let L be a k-level CL function on U, let every continuation R_u be an ℓ-level CL function on V, and let L_j be ℓ_j-level CL functions on pairwise disjoint registers $V^{(j)}$ with $V = \bigoplus_j V^{(j)}$. Then*

$$\begin{aligned} \mathrm{concatenate}(L, R) &\text{ is a } (k+\ell)\text{-level CL function on } U \oplus V, \\ \textstyle\bigoplus_j L_j &\text{ is a } \big(\max_j \ell_j\big)\text{-level CL function on } V. \end{aligned}$$

For CL distributions S_1, S_2, their product, formed by direct-summing the left maps and the right maps on independent seed registers, is CL of the maximum of the two levels.

[23] Ground truth: `rk:higher-level` in `ground-truth/gt-04-cl.tex`.

Proof. For concatenation, induct on the outer constructor of L. At level zero, grafting R_0 supplies ℓ stages. At a `CLStep`, retain its first linear stage and apply the induction hypothesis to each child; this exhibits $1 + (k - 1 + \ell) = k + \ell$ stages. This is the construction in `lem:cl-concat` of `ground-truth/gt-04-cl.tex`.

For direct sum, pad each component by zero stages to $r = \max_j \ell_j$, which is legitimate by `rk:higher-level`. At stage i, use the direct sum of the components' stage matrices on the direct sum of their factor registers. The continuation depends only on the tuple of earlier outputs, hence remains CL. Induction on r exhibits an r-stage presentation, matching `lem:cl-func-prod` in `ground-truth/gt-04-cl.tex`. A distribution product applies this construction separately to its left and right maps, so it is CL of the same maximum. □

Both statements are memberships, not invariants, and both proofs establish only the $\leq$ direction: they exhibit a presentation with that many stages and never argue that fewer are impossible. The converse fails. Take $U = V = \mathbb{F}^1$, $L = \mathrm{id}_U$ and $R_u = \mathrm{id}_V$ for every u, each of level one; their concatenation is $T(x) = x^U + x^V = x$, the identity on $U \oplus V$, which is linear and hence level one, not $k + \ell = 2$. Likewise, regarding the zero map on $V^{(1)}$ as 5-level (legitimate, again by `rk:higher-level`) and the zero map on $V^{(2)}$ as 0-level makes $\max_j \ell_j = 5$ while the direct sum is the zero map, which is level zero.

The code functions are `concatenate`, `direct_sum`, `distribution`, and `product`. Their type constructions carry these constructed presentations, which is why C4a and C4b record nesting depths as *upper bounds in the sense of* `rk:higher-level` and disclaim minimality. The stronger structural hypothesis that all compiler operations of interest close inside one useful algebra is not implied by these individual lemmas.

13.2 Point and line maps

One seed coordinate does all the choosing, so Figure 91 fixes first how a field element names an axis and what that costs in levels; Figure 92 then tracks the three register maps and the prefix projection $\pi_{\chi(s)-1}$ explicitly.

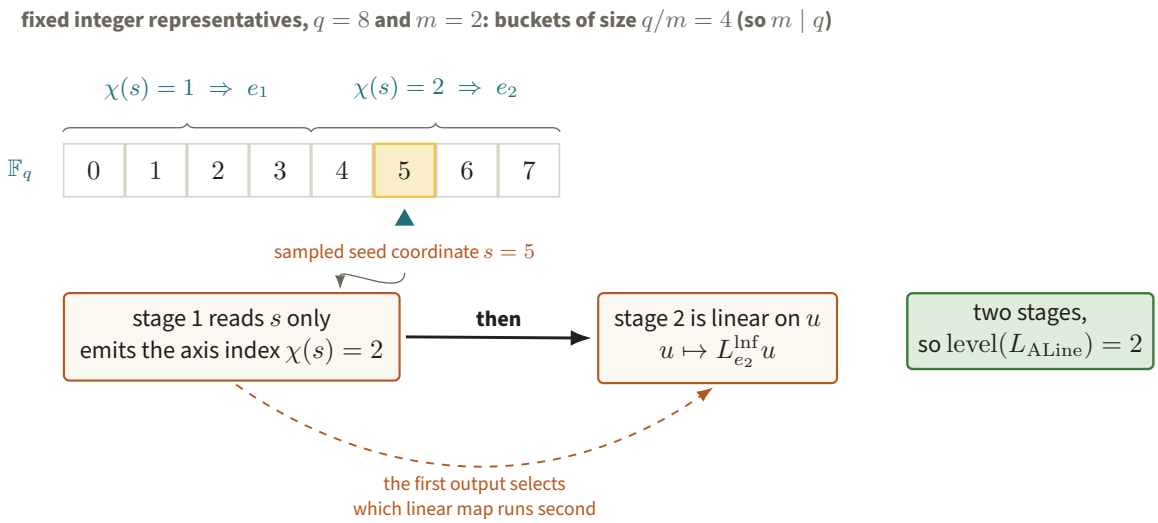


Figure 91. The amber seed coordinate $s = 5$ falls in the second of the two $q/m = 4$-element buckets, so $\chi(s) = 2$ and the axis-line map takes its direction e_2 from a stage that reads s alone. The second stage is then linear on u with the choice already fixed, which is why the constructed presentation of L_{ALine} has two stages — an upper bound in the sense of `rk:higher-level`, with no minimality claimed.

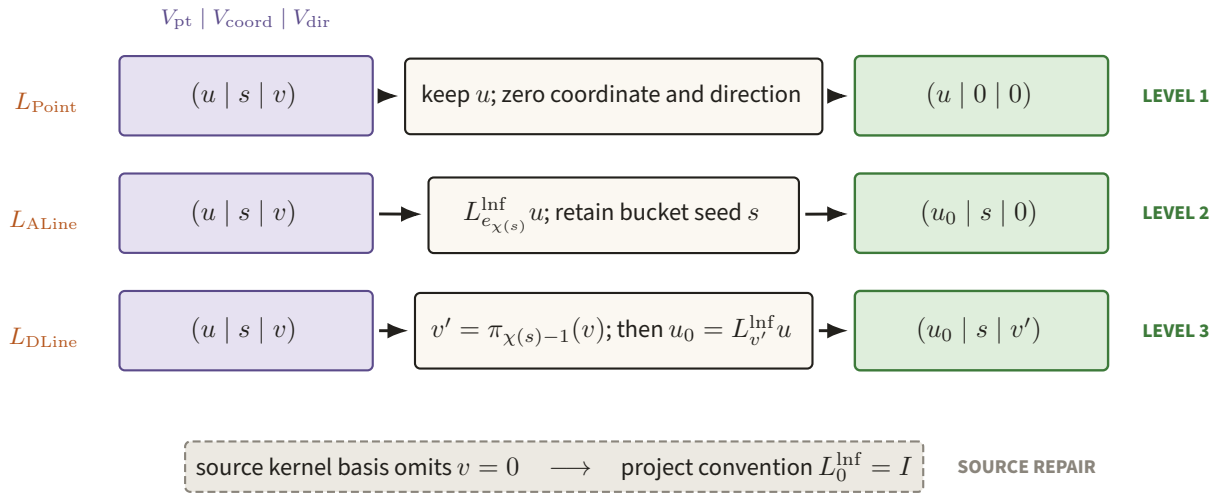


Figure 92. The point, axis-line, and diagonal-line maps act on the same $V_{\mathrm{pt}} \oplus V_{\mathrm{coord}} \oplus V_{\mathrm{dir}}$ seed but retain different register data. Their one-, two-, and three-level factorizations make the sampled geometry CL, with $L^{\mathrm{lnf}}_0 = I$ visibly marked as a source repair.

For $V = V_{\mathrm{pt}} \oplus V_{\mathrm{coord}} \oplus V_{\mathrm{dir}}$, write a seed as $(u, s, v) \in \mathbb{F}_q^m \times \mathbb{F}_q \times \mathbb{F}_q^m$. Let L^{lnf}_v be the canonical projection whose kernel is $\mathrm{span}(v)$, so $L^{\mathrm{lnf}}_v u$ is a canonical representative of the affine line $u + \mathbb{F}_q v$. [24] The line-representative use is `def:line-representative` in `ground-truth/gt-07-ldt.tex`. Define $\chi(s)$ by splitting the fixed integer representatives of $\mathbb{F}_q$ into m equal buckets; this requires $m \mid q$. With π_{i-1} zeroing the first $i-1$ coordinates,

$$
\begin{aligned}
L_{\mathrm{Point}}(u, s, v) &= (u, 0, 0), \\
L_{\mathrm{ALine}}(u, s, v) &= (L^{\mathrm{lnf}}_{e_{\chi(s)}} u, s, 0), \\
L_{\mathrm{DLine}}(u, s, v) &= (L^{\mathrm{lnf}}_{\pi_{\chi(s)-1}(v)} u, s, \pi_{\chi(s)-1}(v)).
\end{aligned}
$$

Their stage factorizations exhibit them as CL functions of level one, two and three respectively; by `rk: higher-level` these are upper bounds, and no minimality is claimed.[25] The implementation names are `L_Point`, `L_ALine`, `L_DLine`, `L_lnf`, `chi`, and `pi_prefix`. Since the source's kernel-basis definition does not cover $v = 0$, the code uses $L^{\mathrm{lnf}}_0 = I$ and marks the convention SOURCE_REPAIR.

The distribution $\mu_{L,R}$ pushes one uniform shared seed through the two maps.[26] For $(q, m) = (8, 2)$, TB1 enumerates all $8^5 = 32768$ seeds. The axis-line/point histogram has support 512, and the diagonal-line/point histogram has support 18432, including the zero directions. Both agree entry-for-entry with separately transcribed formulas for `lem:alnf` and `lem:dlnf` in `ground-truth/gt-07-ldt.tex`; every marginal is replayed. This finite statement is `C4a`: TESTED. It is not a statistical argument and does not establish quantum low-degree soundness.

13.3 Types and the six-copy sampler

Figure 93 separates the five block-local tests from the sixth simultaneous proof test.

[24] Ground truth: `def:cl-canonical` in `ground-truth/gt-03-prelim.tex`.
[25] Ground truth: `sec:ld-game` in `ground-truth/gt-07-ldt.tex`.
[26] Ground truth: `def:cl-dist` in `ground-truth/gt-04-cl.tex`.

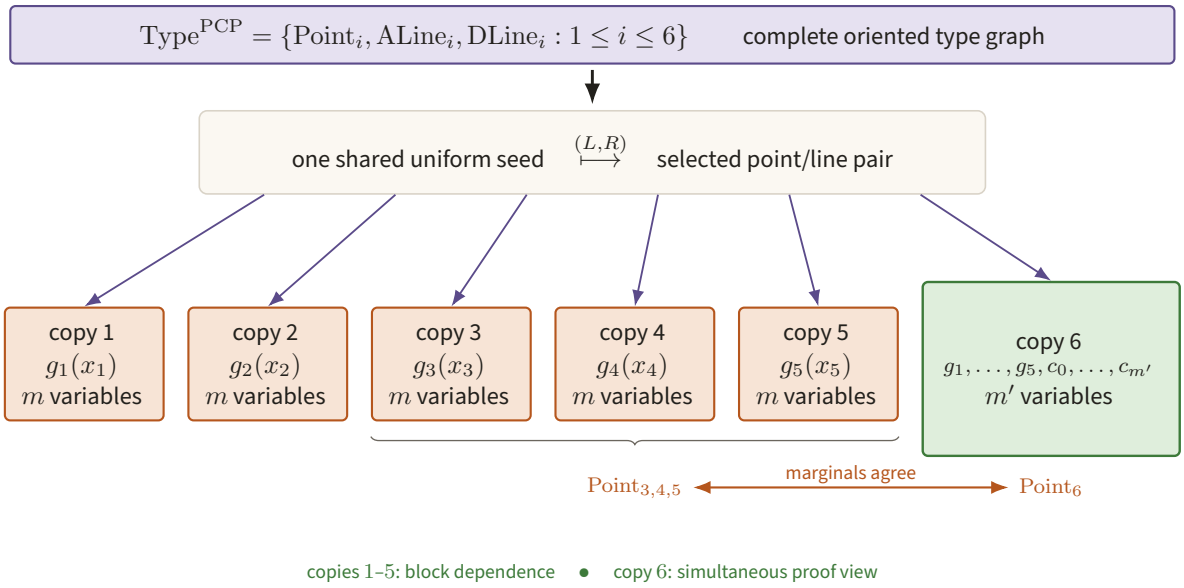


Figure 93. Copies one through five test the five block polynomials individually, while copy six tests the complete $(g_1, \ldots, g_5, c_0, \ldots, c_{m'})$ bundle in dimension m'. The 18 point/axis-line/diagonal-line types share one ambient seed and a complete oriented type graph.

A *typed sampler* is a finite type-graph-indexed family of CL maps sharing one ambient seed space. It first samples an oriented type edge and then pushes one uniform seed through the selected left/right maps. The seven-input machine interface itself is `def:typed-sampler`; the two-step sampling behaviour just described is its induced distribution `def:typed-sampler-sample`.[27] Julia's `TypedSampler` checks exact type coverage, a common field and seed dimension, graph endpoints, and a padded common level.

Detyping encodes the external graph choice into an untyped CL distribution by a rejection-sampling construction. It adds two levels, preserves value-one completeness, and changes soundness error by $16^{|\mathrm{Type}|}$.[28] The Julia function `detype` returns `CitedDetypedVerifier`; the certificate is deliberately CITED. For the 54 answer-reduction types, the displayed factor is 16^{54}, not an executed estimate.

The answer-reduction PCP sampler has the 18 types

$$\{\mathrm{Point}_i, \mathrm{ALine}_i, \mathrm{DLine}_i : 1 \le i \le 6\}$$

and a complete type graph. Copies 1–5 query one m-variate g_i each; copy 6 queries the bundle $(g_1, \ldots, g_5, c_0, \ldots, c_{m'})$ in dimension m'. The six-copy construction and its ambient direct sums are in `sec:ld-compiler` of `ground-truth/gt-10-answer-reduction.tex`. The current code uses `PCPType` and `PCPRegisterLayout` for the type and register bookkeeping, the private constructor `_pcp_cl_map` (`src/samplers/pcp_sampler.jl`) for the 18 maps, `pad_level` (`src/samplers/typed.jl`) to bring them to a common level, and `pcp_sampler` for the family. It carries a visible SOURCE_REPAIR for the copy-6 coordinate-register conflict between the displayed ambient space and the parser table.

Oracularization is the typed transformation in which the isolated roles receive the original left or right CL image while an oracle role receives the common seed and can reconstruct both; the code name is `oracularize_sampler`. Its theorem contract is CITED. [29] The typed answer-reduced sampler is the product of

[27] Ground truth: `def:typed-sampler`, `def:typed-sampler-sample` in `ground-truth/gt-06-types.tex`.
[28] Ground truth: `lem:detyping-verifiers` in `ground-truth/gt-06-types.tex`.
[29] Ground truth: `sec:orac-def` in `ground-truth/gt-09-oracularization.tex`.

this three-role family with the 18-type PCP family. `typed_sampler_product` produces 54 types and combines levels by a maximum. The lazy object here is `CLStep` (`src/samplers/cl.jl`), whose `branch` continuation is evaluated only at the seeds actually reached and memoised after a field/dimension/register/level check. `C4b: TESTED` is scoped to exactly what that buys: type and edge counts, the common level, and self-consistent marginals. The decider's questions are generated by an explicit seed split rather than by applying the product maps, so the product is not evidenced as the source of the questions. Local certificate labels must not be confused with a wider promoted claim.

The typed decider implements the five guarded checks of `fig:decider-pcp`: global consistency, input consistency, input low degree, proof encoding (individual and simultaneous low degree), and the PCP game check.[30] Figure 94 runs the five in order and marks the fall-through that a type pair matching no trigger takes.

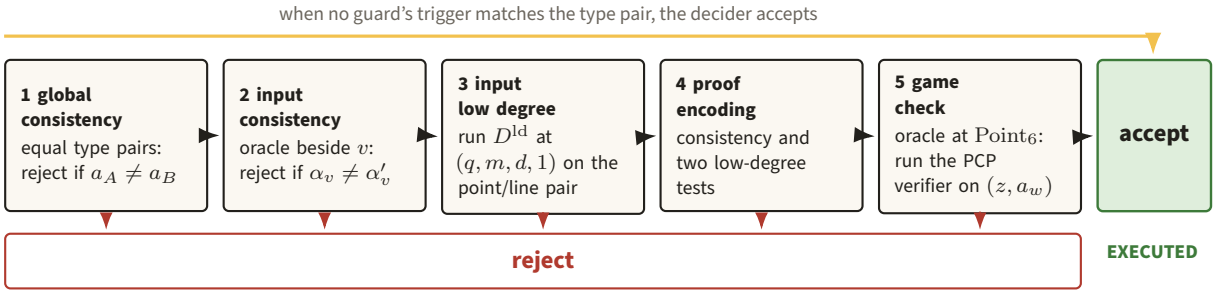


Figure 94. Acceptance flows left to right through five guards and every guard drops out downwards to the same rejection, so a type pair whose trigger never matches follows the amber path straight to acceptance. All five guards are finite executed code in the prototype, with proof encoding running its consistency test and low-degree tests at $(q, m, d, 1)$ and $(q, m', d, m' + 6)$; only the soundness theorem quantifying over them stays cited (after `fig:decider-pcp`).

The Julia names are `TypedAnswerReducedDecider`, `typed_answer_reduced_decider`, `LDParams`, `ld_decider`, and `PCPGameCall`. `answer_reduce_pcp` constructs the typed verifier of level $\max(\ell, 3)$ on its fixture and attaches explicit assumptions.

The full theorem contract is conditional:

$$\begin{aligned}
&\textsc{Assume:}\quad V \text{ is an } \ell\text{-level normal-form verifier,}\\
&\qquad\qquad T(n) = (2^{\lambda n})^{\mu}, \quad Q(n) = (\lambda n)^{\mu},\\
&\qquad\qquad \mathrm{TIME}_D(n) \le T(n), \quad \mathrm{TIME}_S(n) \le Q(n);\\
&\textsc{Prove:}\quad \mathsf{AnswerReduce}(V) \text{ has level } \max\{\ell + 2, 5\},\\
&\qquad\qquad \text{the stated polynomial runtimes and directed completeness,}\\
&\qquad\qquad \text{soundness, and entanglement implications.}
\end{aligned}$$

This is `thm:ar` in `ground-truth/gt-10-answer-reduction.tex`. The finite guarded decider is executable; the quantum implication, detyping, and asymptotic theorem are CITED.

13.4 The structural hypothesis

Figure 95 marks the exact inference that remains unavailable before the longer closure diagram.

[30] Ground truth: `fig:decider-pcp` in `ground-truth/gt-10-answer-reduction.tex`.

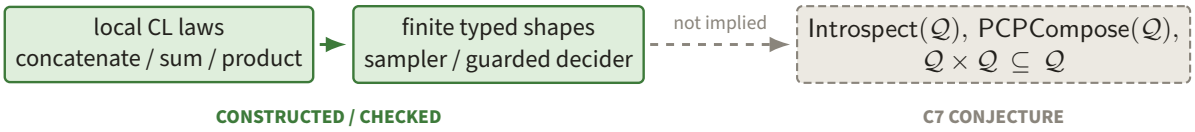


Figure 95. Constructed local CL operations and finite typed shapes stop at the dashed arrow before compiler-wide closure of $\mathcal{Q}$. That missing implication is precisely the C7 structural conjecture rather than a consequence of the datatype alone.

Introspection is the cited verifier transformation that makes encoded prover-side measurement information available to a smaller verifier while preserving a directed soundness contract. [31] *Anchoring* inserts special anchor questions with positive probability; *parallel repetition* runs several copies with an AND acceptance condition against possibly correlated strategies. Their combined repetition contract is cited from `thm: repetition` in `ground-truth/gt-11-parallel-repetition.tex`.

The relevant closure equations are

$$\mathsf{Introspect}(\mathcal{Q}) \subseteq \mathcal{Q}, \qquad \mathsf{PCPCompose}(\mathcal{Q}) \subseteq \mathcal{Q}, \qquad \mathcal{Q} \times \mathcal{Q} \subseteq \mathcal{Q}.$$

Concatenation, direct sum, the shared-seed distribution, and product are visibly CL-preserving in the base inductive datatype. Affine restriction is linear after earlier stages choose a direction, and random-point identity tests are pushforwards of a shared uniform seed. Typed graph choice is external, and the cited detyping transformation moves it back into two further CL levels.

This does not characterize every admissible query distribution, prove that a generic PCPP can be expressed in the algebra, or prove closure under the full introspection compiler. The structural statement is exactly C7: CONJECTURE. Introspection soundness and normal-form closure are not implemented. The present datatype makes several local closure laws legible, not the global compression theorem.

Figure 96 separates those derived local laws from the global closure hypothesis they do not establish.

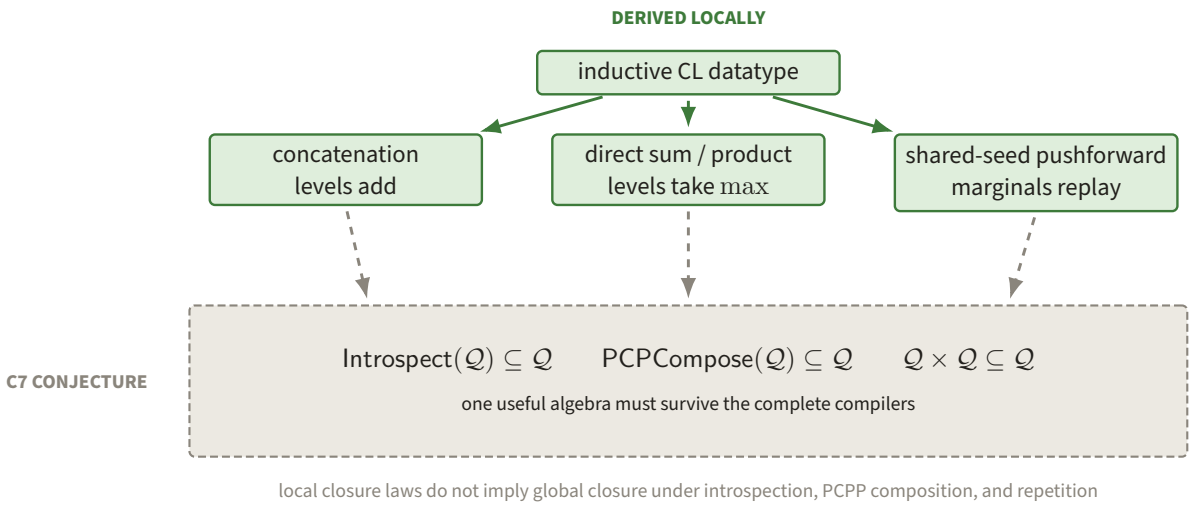
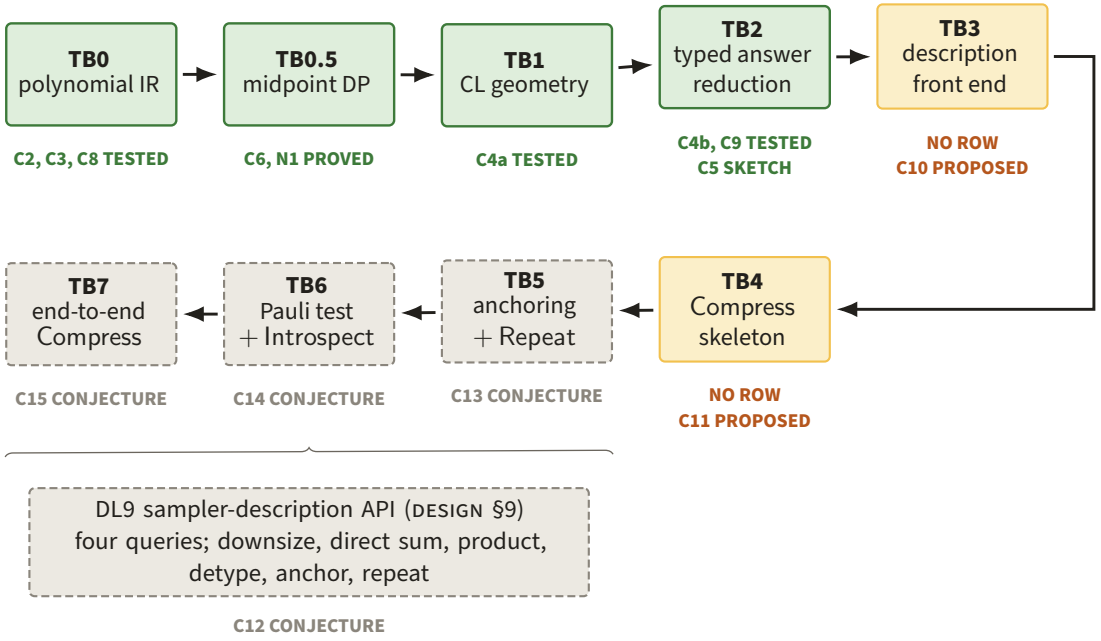


Figure 96. Green branches are closure operations carried by the inductive CL datatype, whereas the dashed slate box is the stronger compiler-wide closure hypothesis C7. The distinction blocks local concatenation and product laws from silently becoming an introspection or compression theorem.

[31] Ground truth: `thm:introspection` in `ground-truth/gt-08-introspection.tex`.

14 Correspondence tables

The evidence grade describes the local certificate leaf. The claim column is the current status in `claims/CLAIMS.md`; it can remain weaker than a certificate label when an adversarial verdict has not promoted the claim. CONSTRUCTED means enforced by a constructor, CHECKED means deterministic replay against an attached object, and CITED means imported from the named ground-truth statement. A claim column reading "no current row" means exactly that: no row of `claims/CLAIMS.md` governs the object. Every theorem and lemma of Part II is instead a written derivation of this project recorded as `C16`–`C19` at SKETCH — NOT YET ADVERSARIALLY VERIFIED: a written derivation, no machine check, no converged verdict. `C16` covers Section 8, `C17` Section 9, `C18` Section 10, and `C19` Section 11, as stated at the head of Section 8. Figure 97 gives the status-bearing order of the tracer-bullet rungs, and Figure 98 compresses the six tables into one evidence map.



Chips transcribe `claims/CLAIMS.md`; `C10` and `C11` are brief-level proposals (`briefs/23-tb3.md`, `briefs/24-tb4.md`) and have no row there. TB3–TB7 are designed and briefed, not executed here.

Figure 97. The ladder runs TB0–TB3 along the top and TB4–TB7 back along the bottom: TB5 is anchoring and Repeat (C13), TB6 the Pauli test and Introspect (C14), TB7 the end-to-end Compress (C15), and all three rest on the DL9 description API (C12). Every chip is transcribed from `claims/CLAIMS.md`, so the ladder grades claims rather than implementation completeness.

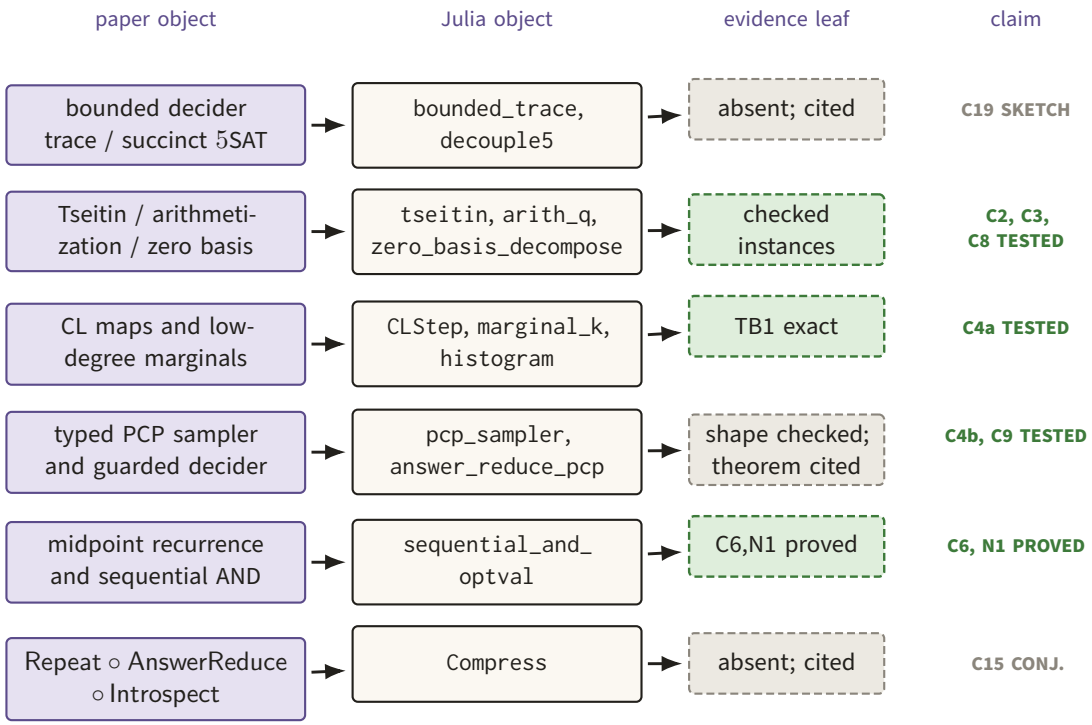


Figure 98. Each row aligns a paper object with its current Julia representation and the narrowest evidence leaf actually available. Green instance checks occupy the middle rungs, while missing front-end and compression implementations remain visibly cited.

14.1 Description front end (planned TB3)

Figure 99 records the currently absent description-to-SAT implementation without weakening the analytic interface.

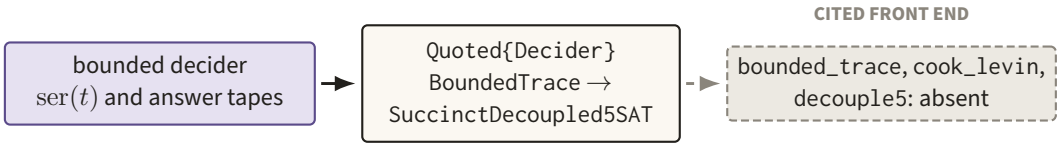


Figure 99. The paper's bounded decider passes through quoted and succinct IR sorts toward three named front-end operations that are still absent. The dashed final arrow marks this row as a cited contract rather than an executed construction; the Part II theorems it rests on are C16–C19 at SKETCH — NOT YET ADVERSARIALLY VERIFIED: written derivations, no machine check, no converged verdict.

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
`sec:tms` `gt-03-prelim.tex`	Machine conventions in Definition 7.1; costed bridge in Theorems 9.1–9.2	`Quoted{A}`	`Quote`, `Eval`, `Specialize`	CITED; absent	C17 SKETCH
`def:decider` `gt-05-games-normalform.tex`	Five-input format in Definition 7.2; bounds preserved by Theorem 9.3	`Quoted{Decider}`	`description_size`	CITED; absent	C17 SKETCH

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
DESIGN §1.1	Determinism, fuel, quotation, and specialization: Theorems 8.2–8.4 and Lemma 8.5	ClosedProgram, Quoted	Eval, specialize	analytic only; absent	C16 SKETCH
prop: standard-succinct-sat gt-10-answer-reduction.tex	Transition locality (Lemma 11.1) and succinct 3SAT (Theorem 11.2)	BoundedTrace, Succinct3SAT	bounded_trace, cook_levin	CITED; absent	C19 SKETCH
sec:succinct-deciders gt-10-answer-reduction.tex	Three shared reads become five independent blocks	SuccinctDecoupled5SAT	decouple5	CITED; absent	C19 SKETCH

14.2 Polynomial rung (TB0)

Figure 100 compresses the polynomial table into the certificate path used on named fixtures.

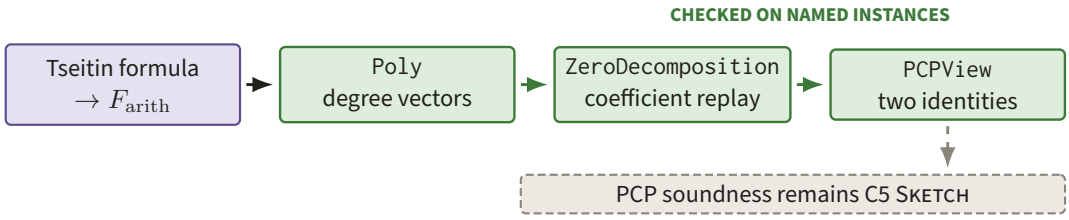


Figure 100. Tseitin syntax flows into sparse polynomials, exact zero-basis division, and the two local PCP identities. Green boxes are checked on named instances, while the separate dashed soundness leaf remains C5 SKETCH.

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
def:tseitin gt-10-answer-reduction.tex	Circuit becomes a linear-size formula with auxiliary wires	Circuit, TseitinFormula	tseitin, occurrences	CHECKED occurrence	C2 TESTED
def: formula-arithmetization gt-10-answer-reduction.tex	Boolean connectives become a formal polynomial	Poly	arith_q, actual_degrees	CHECKED	C8 TESTED
sec:ld-encoding gt-03-prelim.tex	Assignment becomes its multilinear extension	Poly	g_a, dependency_blocks	CONSTRUCTED/ CHECKED	C3 TESTED
prop:zero-basis gt-10-answer-reduction.tex	Divide by $z_i(1-z_i)$ and require zero remainder; the executable rewrite is Lemma 12.1	ZeroDecomposition	zero_basis_decompose, verify_zero_ decomposition	CHECKED	C2 TESTED
fig:pcpverifier gt-10-answer-reduction.tex	Check formula and zero identities at one view	PCPView	pcpverifier	CHECKED finite views	C1 CONJ.
thm:pcp-decider gt-10-answer-reduction.tex	Acceptance $> 1/2$ decodes an accepting trace, assuming low degree	PCPProof	build_pcp, parameter_policy	CITED/ CHECKED parts	C5 SKETCH

14.3 CL and low-degree-test rung (TB1)

Figure 101 places exact CL construction and histogram evidence before the cited low-degree theorem.

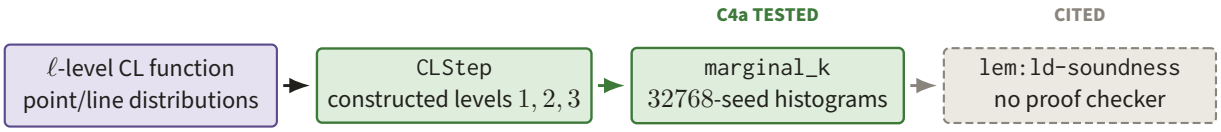


Figure 101. CL constructors realize the point and line maps at levels one, two, and three, and TB1 replays all 32768 shared seeds. The final dashed transition to quantum low-degree soundness has no proof checker.

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
def:cl-func gt-04-cl.tex	CL functions are inductive conditional linear stages (Definition 13.1)	AbstractCL	CLZero, CLStep, level	CONSTRUCTED	C4a TESTED
lem:cl-kth gt-04-cl.tex	A marginal is the sum of the first k stages	CLMarginal	marginal_k, sum_stage_outputs	CHECKED	C4a TESTED
lem:cl-concat, lem:cl-func-prod gt-04-cl.tex	Concatenation *is* $(k+\ell)$-level; direct sum *is* $\max_j \ell_j$-level (Lemma 13.2)	AbstractCL	concatenate, direct_sum, product	CONSTRUCTED	no current row
lem:alnf, lem:dlnf gt-07-ldt.tex	CL pushforwards equal the line/point distributions	CLDistribution	L_Point, L_ALine, L_DLine, histogram	CHECKED TB1	C4a TESTED
fig:ld-decider gt-07-ldt.tex	Point/line answer consistency and degree format	LDParams	ld_decider, D_ld, restrict	CHECKED fixture	no current row
lem:ld-soundness gt-07-ldt.tex	Successful quantum strategy yields polynomial measurements	theorem leaf	no proof checker	CITED	no current row

14.4 Typed answer-reduction rung (TB2)

Figure 102 follows typed sampler shape and guarded decider execution to their cited theorem boundary.

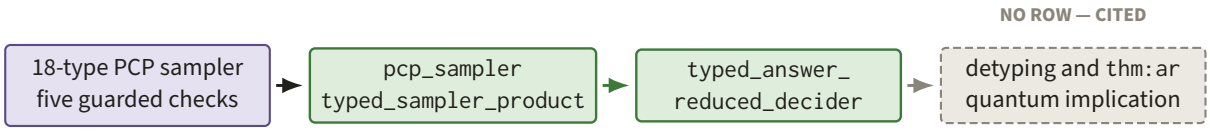


Figure 102. The 18-type sampler and guarded answer-reduced decider have constructed or finite-checked shapes, carrying C4b and C9 at TESTED. Detyping and thm:ar stay dashed: both have no ratchet row and stay CITED. Only the executable classical part of the answer-reduced decider is ratcheted, by C9 at TESTED; C5 governs thm:pcp-decider's parameter arithmetic, not thm:ar's completeness, soundness, and entanglement contract.

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
def:typed-sampler gt-06-types.tex	A type graph selects a pair of CL maps sharing one seed	TypedSampler	TypedSampler, sample, pad_level	CONSTRUCTED base	C4b TESTED
sec:ld-compiler gt-10-answer-reduction.tex	Six low-degree copies form an 18-type family	CLStep	_pcp_cl_map, pcp_sampler, intrinsic_pcp_levels	local CHECKED	C4b TESTED
sec:ld-compiler gt-10-answer-reduction.tex	Product with the oracularized sampler has 54 types	Checked{TypedSampler}	oracularize_sampler, typed_sampler_product	CHECKED shape; theorem cited	C4b TESTED

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
`fig:decider-pcp` `gt-10-answer-reduction.tex`	Five guarded checks connect input, proof, low degree, and PCP	`TypedAnswer ReducedDecider`	`typed_answer_reduced_decider`	CHECKED finite paths	C9 TESTED
`lem:detyping-verifiers` `gt-06-types.tex`	Remove types at cost $+2$ levels and 16^{54} soundness factor	`CitedDetyped Verifier`	`detype`	CITED	no current row
`thm:ar` `gt-10-answer-reduction.tex`	Conditional complexity, completeness, soundness, entanglement contract	`TypedAnswer ReducedVerifier`	`answer_reduce_pcp`, `AnswerReduce`	CHECKED/ CITED	no current row

14.5 Midpoint diagnostic (TB0.5)

Figure 103 isolates the executable recurrence behind the two proved midpoint rows.

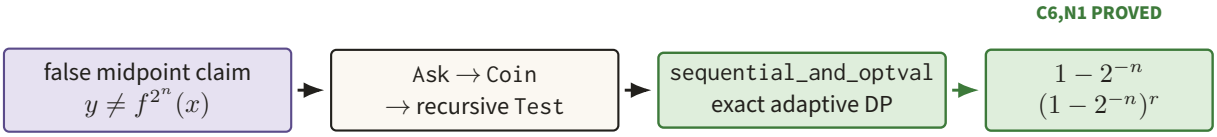


Figure 103. The `Ask/Coin/Test` recurrence is evaluated by an exact adaptive dynamic program. Its one-copy and sequential-AND values are the proved C6 and N1 formulas, not claims about quantum parallel repetition.

Analytic anchor	Analytic statement	IR sort	Implementation	Grade	Claim
`handoff.md` midpoint diagnostic	False level-n claim has value $1 - 2^{-n}$ under the stated orbit condition	`Test/Ask/Coin` terms	`toys/midpoint`	proof plus exact tests	C6 PROVED
`toys/midpoint/PROOF.md`	Sequential r-copy AND value is $(1 - 2^{-n})^r$; constant gap costs $\Theta(2^n)$ copies	exact dynamic program	`sequential_and_optval`	proof plus exact tests	N1 PROVED

14.6 Compression and fixed point (TB4–TB7)

Rungs TB0–TB2 are landed and have been through critic rounds; the four rungs of this subsection are not. What exists for them is a converged *design*: `docs/DESIGN.md` §9 fixes the sampler-description API, §10 anchoring and repetition (TB5), §11 the Pauli test and introspection (TB6), §12 the end-to-end Compress and the halting fixed point (TB7). Four ratchet rows were opened for that design and all four stand at CONJECTURE: C12 for the description-level closure laws, C13 for TB5, C14 for TB6, C15 for TB7. No line of `src/` implements them at this writing, so every grade below is CITED and every claim column is CONJECTURE: the rows record a design commitment, not evidence.

Figure 104 keeps the fixed-point mechanism distinct from the three cited compression transformations.

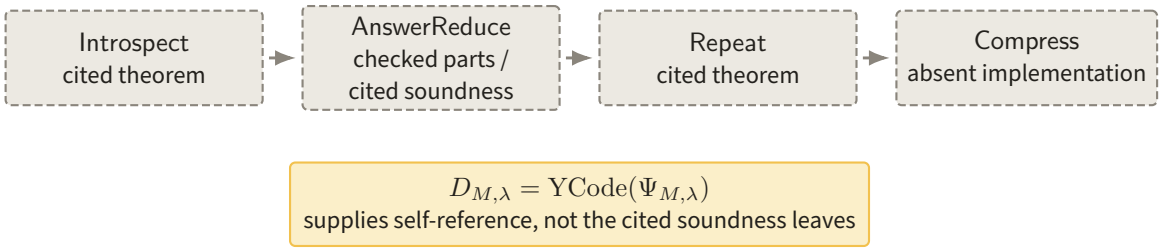


Figure 104. Introspection, answer reduction, and repetition compose syntactically into the Compress row, but their soundness leaves remain dashed and no implementation exists. The amber fixed point supplies self-reference only; it does not discharge those analytic contracts.

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
`def:sampler` queries `gt-04-cl.tex`	Six description-level transformations are definable from field/level headers and the four sampler queries alone	`SamplerDescription`	absent; designed in DESIGN §9	CITED	C12 CONJ.
`prop:anchoring`, `thm:repetition` `gt-11-parallel-repetition.tex`	Anchoring plus $k(n)$-fold parallel repetition; levels and dimensions $1, 1 \to 3, 9 \to 3, 729$ on the TB5 fixture	`AnchoredVerifier`	absent; designed in DESIGN §10 (TB5)	CITED	C13 CONJ.
`thm:pauli`, `thm:introspection` `gt-08-introspection.tex`	Introspection takes a level-9 verifier to a level-5 one; TB6 type/edge counts $34/116$ and $38/128$, detyped level 5	`IntroVerifier`	absent; designed in DESIGN §11 (TB6)	CITED	C14 CONJ.
`fig:compress` `gt-12-compression.tex`	Repeat ∘ AnswerReduce ∘ Introspect, in that order, on $(\overline{V}, \lambda)$	`ClosedProgram` `Compressor`	absent; designed in DESIGN §12 (TB7)	CITED	C15 CONJ.
`thm:compression` `gt-12-compression.tex`	Nine-level gap-preserving compression, sampler independent of $\overline{V}$	contract tree	no current implementation	CITED	C15 CONJ.
`fig:halt_f` `gt-12-compression.tex`	Self-application, Kleene, and YCode coincide by Theorems 10.2–10.4	`PartialProgram`, `Quoted`	`Hole`, `Specialize`, `Fix`/`YCode`	CITED; absent	C15 CONJ.; C18 SKETCH

14.7 The TB7 fixture and its two non-executed layers

Figure 105 is the design's own instantiation of the constants of Figure 43, together with the level and dimension chains it predicts and the two layers it declares it does *not* execute. It is printed here for one reason: it is the only place where the document's compression chapter carries numbers that can be falsified. Nothing in it has been run.

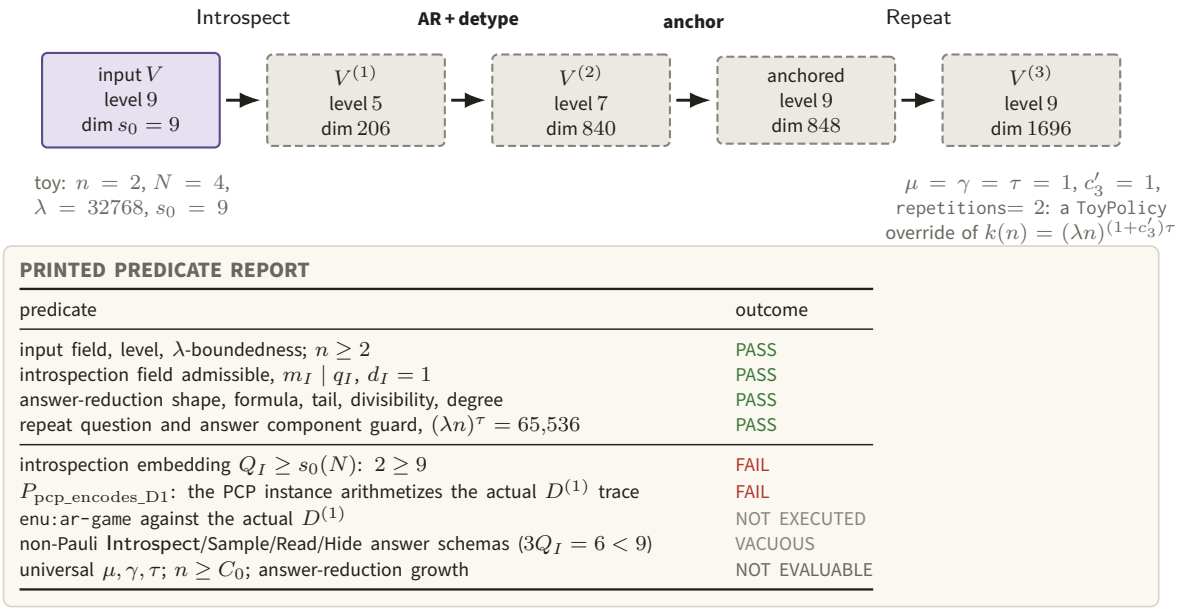


Figure 105. The toy instantiation at $n = 2$, $\lambda = 32768$, $s_0 = 9$: four constructed levels, four dimensions, and the two predicates whose failure is the reason two layers of the chain are declared unexecuted. Printing the failures beside the chain is what keeps a green construction from reading as a discharged theorem.

The two named non-executed layers are exactly those of C15. First, `enu:ar-game` against the actual $D^{(1)}$: the PCP instance supplied at $(m_A, s_A) = (1, 6)$ cannot arithmetize a trace of length $T = (2^{\lambda n})^\mu$, so $P_{\text{pcp\_encodes\_D1}}$ is `FAIL` and the game branch is `NOT_EXECUTED`. Second, the non-Pauli introspection answer schemas: with $M_I = 2$ the introspection register carries $Q_I = 2$ bits while the input verifier needs $s_0(N) = 9$, and $3Q_I = 6 < 9$, so Introspect, Sample, Read and Hide are `VACUOUS` in both roles and only the Pauli-typed predicates would execute. Neither failure is folded into a pass.

15 What is analytic and what is computed

15.1 The evidence boundary

The analytic content is the sequence of transformations and the implications attached to them. The computer currently realizes a strict middle segment. Figure 106 locates that middle segment inside the full transformation pipeline.

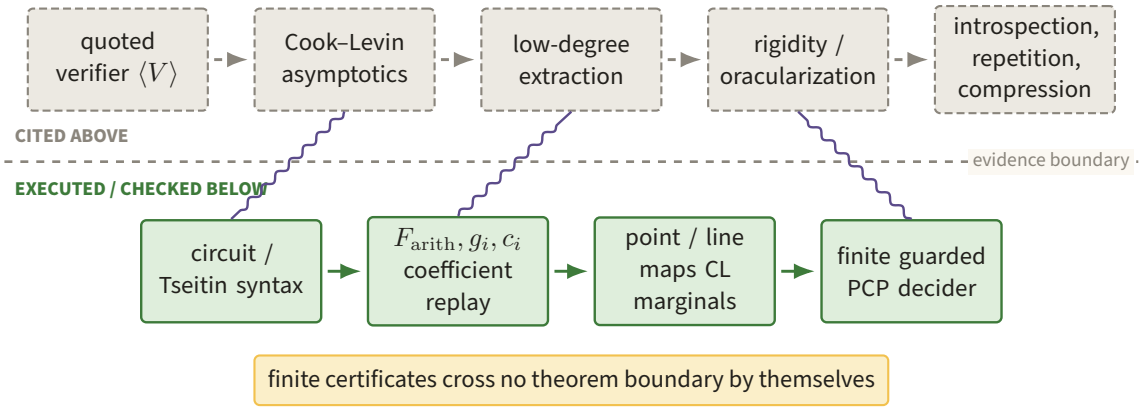


Figure 106. Green construction and replay stages sit below the boundary, while Cook–Levin asymptotics, low-degree extraction, rigidity, introspection, repetition, and compression remain dashed cited leaves above it. Violet crossings indicate interfaces, not a promotion of finite certificates into analytic theorems.

Class	Present content	What would be required to move the boundary
Executed constructions	Circuit/Tseitin syntax; formula arithmetization; multilinear extensions; c_0; zero-basis quotients; point/line maps; typed products; guarded TB2 decider	Scale-independent front end for quoted programs and a constructor-faithful lazy CL stage representation for the PCP sampler
Certified identities	Normalized coefficient equality; zero remainder; structural and actual degrees; dependency blocks; CL marginals; TB1 exact histograms; finite parser and branch checks	General semantic certificates connecting trace, SAT, Tseitin, and arithmetization, plus promotion through adversarial verdicts
Cited analytic leaves	Cook–Levin asymptotics; general Tseitin semantics; PCP soundness; low-degree extraction; oracularization; detyping; introspection; rigidity; anchored repetition; compression	Formal proofs over quantified machines, distributions, Hilbert spaces, measurements, approximation metrics, and asymptotic bounds—not more finite samples

Lean or another proof assistant could check the algebraic core once finite fields, sparse polynomials, interpolation, and polynomial division are formalized. That would move the general zero-basis identity and degree lemmas from prose to proof terms. It would not by itself certify `lem:ld-soundness`: that leaf requires a library for finite-dimensional Hilbert spaces, POVMs, state-dependent distances, tensor-code tests, and the quantitative extraction argument. Rigidity and oracularization require the same operator layer. Detyping additionally needs a formal probability and rejection-sampling argument with the $16^{|\mathrm{Type}|}$ loss.

Anchoring and parallel repetition are combined in Repeat and are theorem-backed rather than executed here. They are, however, no longer undesigned: `docs/DESIGN.md` §§9–12 fixes the description-level API and the TB5, TB6 and TB7 constructions, and C12–C15 record that design at Conjecture (Section 14.6). What is missing is implementation and adversarial verification, not a plan. Introspection and compression would additionally require a still broader object: a formal semantics for quoted verifier descriptions, resource bounds under specialization and universal evaluation, and a proof that the directed value and entanglement implications compose. The relevant final statements are `fig:compress` and `thm:compression` in `ground-truth/gt-12-compression.tex`.

Figure 107 plots the collapsing one-copy gap and the exact first six repetition counts used in the next

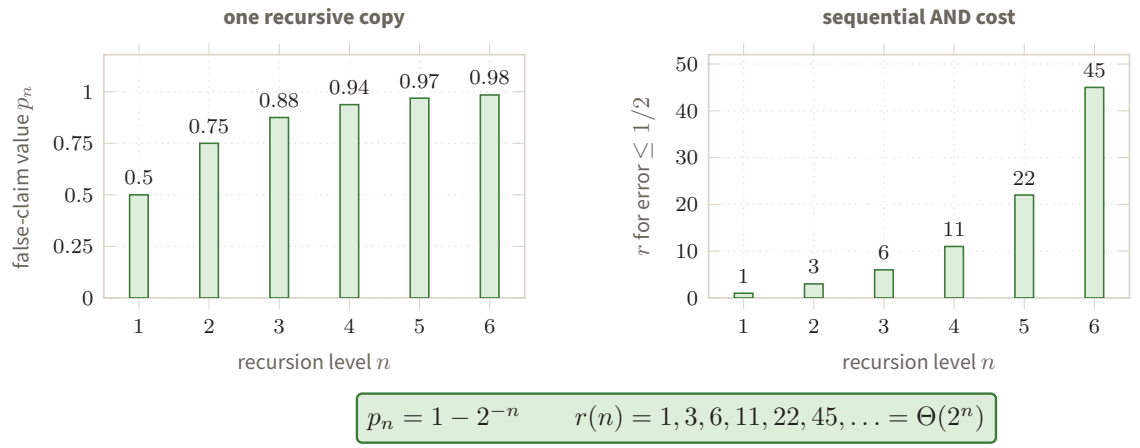


Figure 107. The left bars approach value one as $p_n = 1 - 2^{-n}$, while the right bars give the exact repetition counts $1, 3, 6, 11, 22, 45$ needed to recover error at most one half. The paired growth makes the $\Theta(n2^n)$ sequential transcript cost visible and diagnoses why representation alone is insufficient.

diagnostic.

15.2 The midpoint diagnostic

The fresh-coin split in Figure 108 is the complete local move of the recursive verifier.

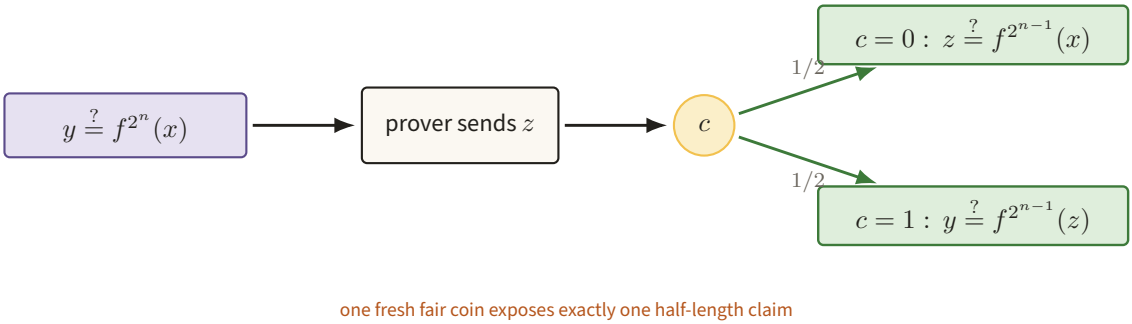


Figure 108. After the prover supplies z, one fresh fair coin selects the left or right half-length orbit claim. This single branching move yields the exact false-claim recurrence behind $p_n = 1 - 2^{-n}$.

The recursive midpoint verifier for $y = f^{2^n}(x)$ asks for z and checks one of

$$z = f^{2^{n-1}}(x), \qquad y = f^{2^{n-1}}(z)$$

with a fresh fair coin. It has a compact fixed-point description and perfect completeness. Under the orbit-prefix condition stated in C6, the exact optimum for a false claim is nevertheless

$$p_n = 1 - 2^{-n}.$$

Claim C6 is PROVED. For adaptive sequential AND repetition, N1 is also PROVED and gives value $(1-2^{-n})^r$; reaching error at most $1/2$ requires $r = \Theta(2^n)$, hence $\Theta(n2^n)$ transcript cost in the stated unit-cost model. Neither claim concerns quantum parallel repetition or the actual Compress operator.

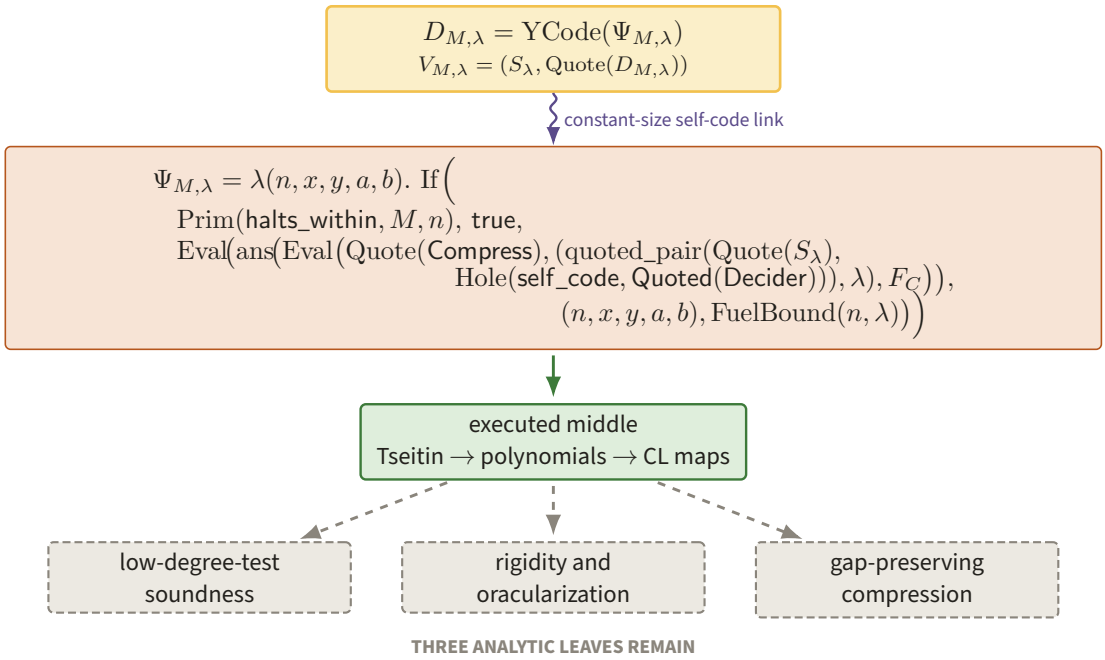


Figure 109. The actual term $\Psi_{M,\lambda}$ places its self-code hole inside the quoted call to Compress, and the amber fixed point $D_{M,\lambda} = \mathrm{YCode}(\Psi_{M,\lambda})$ closes that hole with constant overhead. The executable algebraic middle does not discharge the three dashed leaves: low-degree soundness, rigidity/oracularization, and gap-preserving compression.

The diagnostic isolates the point of the whole construction. A recursive term can represent an exponentially long computation, and a fixed point can make that term self-referential, while the acceptance gap collapses too quickly for iteration. Representability is easy; an iteratable, gap-preserving analytic transformation is the hard requirement.

Figure 109 closes the account at the fixed point itself, with the three analytic leaves kept visibly dashed.

15.3 Final accounting

The lambda reformulation removes none of the mathematically difficult leaves. It makes quotation, hard-wiring, description size, timeout, and self-reference explicit in one syntax. The polynomial IR makes the Cook–Levin-to-PCP middle visible as actual algebra, and the CL datatype exposes several sampler closure laws. What remains unchanged is the proof that high success in the typed quantum game yields consistent low-degree polynomial measurements, survives oracularization and detyping with the stated quantitative loss, survives introspection and anchored parallel repetition, and composes into the gap-preserving compression theorem. Those are analytic theorems, presently CITED; they are neither consequences of homoiconicity nor conclusions of the finite computations reported here.

Figure 110 leaves the document with the fixed point closed and the three analytic obligations still visibly open.

Figure 110. The amber description $D_{M,\lambda}$ completes the syntactic self-reference, but each dashed arrow still crosses into a cited analytic theorem. Closing those low-degree, rigidity/oracularization, and gap-preservation leaves—not another finite sample—is the remaining proof task.